perplexity

I have a video and wanted to create a project for the screens that are in the video. Is it possible?

Yes, it is possible. A video can be used as the reference to recreate the visible screens, navigation flow, and basic interactions—but the video alone usually cannot reveal the original backend logic, database structure, APIs, or exact business rules.

What can be extracted

From the video, you can identify:

- Individual screens and page transitions.
- Buttons, menus, forms, tables, cards, dialogs, and icons.
- Text labels and visible validation messages using OCR.
- Screen states, such as loading, error, expanded, selected, or submitted.
- Navigation and user actions.
- Common colors, typography, spacing, and reusable UI components.
- A preliminary user-flow or screen-flow diagram.

A screen-recording contains more information than a single screenshot because it shows how the interface changes after user actions. Modern video-to-UI workflows generally follow **record, extract, and modernize**: analyze the recording, identify components and states, then implement them in a target technology such as React or a design tool.[1] [2]

Recommended workflow

1. **Upload or provide the video.**
2. **Extract key frames** whenever the screen changes.
3. **Create a screen inventory**, for example:
    - Login
    - Dashboard
    - Search
    - Case details
    - Edit form
    - Confirmation dialog
4. **Document each screen:**

    - Layout
    - Fields and controls
    - Visible text
    - User actions
    - Expected response
    - Navigation target
5. **Create wireframes or editable designs** in Figma.
6. **Build a clickable prototype.**
7. **Implement the application** using the selected technology.
8. **Add the missing backend behavior**, APIs, authentication, validation, and data model.

Screenshot-analysis tools can detect layout regions, UI components, and text through OCR, although the generated result still requires human review. [3] [4]

Possible output

For example, I could help you produce:

Output	Result
Screen list	All screens detected from the video
User-flow map	Navigation between screens
UI specification	Detailed fields, buttons, tables, and states
Figma design	Editable visual mockups
HTML/CSS prototype	A browser-based visual recreation
React application	Functional frontend components
Pega implementation plan	Cases, views, actions, data objects, and integrations
Test scenarios	Steps and expected results for each flow

For a Pega project, the video could be converted into a **case lifecycle**, **view definitions**, **workflow actions**, **data fields**, **validation rules**, and a list of integrations requiring confirmation.

Important limitations

- Hidden screens and backend processing cannot be inferred reliably.
- A click may trigger an API call or business rule that is invisible in the video.
- Text, colors, and dimensions may be inaccurate if the recording quality is low.
- Sensitive information should be removed or masked before uploading.
- You should have permission to reproduce the application and its content.
- AI-generated code or designs must be reviewed against the original video.

The best result comes from a high-resolution recording that shows every important action—opening menus, entering data, submitting forms, handling errors, and navigating backward. Some specialized tools can analyze videos and export component or design information, but their output should be treated as a starting point rather than a complete production application. [5] [6]

You can upload the video here, and I can help you analyze it screen by screen and create a structured project specification or implementation plan.

⁂

Durgam software billmaster code

If you mean **the source code of Durgam Soft's Billmaster software**, I could not find any publicly available source-code repository or downloadable code. Durgam Soft presents Billmaster as proprietary billing, inventory, barcode, and POS software, and identifies Flutter as its programming language. [11] [12]

What is publicly available

- **Official Billmaster page:** Durgam Soft describes features such as billing, inventory management, barcode scanning, and POS operations. [11]
- **Google Play application:** The Android app is listed under the package identifier `com.durgamsoft.com.dsmartpos`. [13]
- **Technology:** The company's website lists **Flutter** as the programming language. [12]
- **Support contact:** The Google Play listing provides `info@durgamsoft.com` as a support email. [13]

If you want to recreate it

You can build a similar application with:

- **Frontend:** Flutter
- **Backend:** Node.js, Java, .NET, or a Pega application
- **Database:** PostgreSQL, MySQL, or Firebase
- **Main modules:** Products, customers, suppliers, inventory, billing, GST/tax, payments, barcode scanning, POS, reports, and receipt printing
- **Integrations:** Bluetooth/USB/LAN thermal printers, barcode scanners, payment systems, and accounting software

If you have a video of Billmaster, I can help convert it into a **screen-by-screen specification**, Flutter UI prototype, React application, or Pega case-management design. You should not copy proprietary source code or protected branding without permission; instead, create an independently implemented application with similar business functionality.

⁂

Flutter architecture for billing and POS apps

For a billing and POS application, use a **layered, offline-first architecture with MVVM, repositories, and unidirectional data flow.** This keeps sales reliable when the internet is unavailable and makes the application easier to extend with inventory, printing, payments, and synchronization. Flutter's own architecture guidance recommends separating UI and data layers, using views, view models, repositories, and services. [21] [22]

Recommended architecture

```
Flutter UI
   ↓ user events
ViewModels / Controllers
   ↓ business commands
Use Cases / Domain Services
   ↓
Repositories
   ↓
Local Database + Remote API
   ↓
Sync Queue / Background Synchronization
```

For a small application, you can begin with UI, ViewModel, and Data layers. Add a separate domain layer when tax calculation, discounts, returns, promotions, payment handling, or synchronization becomes complex. [23] [22]

Project structure

```
lib/
├── app/
│   ├── app.dart
│   ├── router.dart
│   ├── theme.dart
│   └── dependency_injection.dart
│
├── core/
│   ├── errors/
│   ├── result/
│   ├── network/
│   ├── database/
│   ├── printing/
│   ├── barcode/
│   └── security/
│
├── features/
│   ├── authentication/
│   ├── dashboard/
│   ├── products/
│   ├── inventory/
│   ├── cart/
│   ├── sales/
```

```
│   ├── payments/
│   ├── customers/
│   ├── returns/
│   ├── reports/
│   └── settings/
│
└── shared/
    ├── widgets/
    ├── models/
    └── formatters/
```

Organize primarily by **business feature**, rather than putting all screens, models, and services into large global folders. Each feature can contain its view, ViewModel, domain logic, repository, models, and tests.

Core modules

Module	Main responsibility
Product catalog	Products, categories, prices, tax rates, barcode values
Cart	Selected items, quantities, discounts, taxes, totals
Sales	Draft, completed, cancelled, and returned transactions
Payments	Cash, card, UPI, wallet, split payments, payment status
Inventory	Stock movements, adjustments, purchase receipts, low-stock alerts
Customers	Customer profiles, credit limits, purchase history
Printing	Thermal receipts, invoices, reprints, printer discovery
Synchronization	Upload local transactions and download master data
Reports	Sales, tax, profit, inventory, cashier, and shift reports
Authentication	Users, roles, permissions, device registration

Offline-first design

The local database should be the primary operational store for products, carts, transactions, payments, and stock movements. The repository can combine local and remote data and expose one interface to the rest of the application; this is the pattern recommended in Flutter's offline-first guidance. [24]

A sale should be recorded locally before attempting synchronization:

```
Create sale
   ↓
Validate cart
   ↓
Calculate totals
   ↓
Save sale locally
```

```
   ↓
Save payment locally
   ↓
Update local stock
   ↓
Print receipt
   ↓
Add sync job
   ↓
Synchronize with server when online
```

Use fields such as:

```
id
deviceId
createdAt
updatedAt
syncStatus
syncAttempts
version
```

Possible synchronization states are:

```
pending → syncing → synced
pending → failed → retrying
```

Do not depend on the server response before completing a basic cash sale unless the business specifically requires online authorization.

Suggested data model

```
Product
- id
- sku
- barcode
- name
- categoryId
- sellingPrice
- costPrice
- taxRate
- stockQuantity
- isActive

Sale
- id
- invoiceNumber
- customerId
- subtotal
- discount
- tax
- total
- status
```

```
- cashierId
- createdAt
- syncStatus

SaleItem
- id
- saleId
- productId
- quantity
- unitPrice
- discount
- tax
- total

Payment
- id
- saleId
- method
- amount
- reference
- status

StockMovement
- id
- productId
- type
- quantity
- referenceType
- referenceId
- createdAt
```

For stock accuracy, prefer an immutable **stock-movement ledger** over simply changing a product's quantity. The current quantity can be cached for fast display, while stock movements remain the audit trail.

Cart and checkout flow

A typical checkout ViewModel might expose:

```
class CheckoutState {
  final List<CartItem> items;
  final Money subtotal;
  final Money discount;
  final Money tax;
  final Money total;
  final bool isSaving;
  final String? error;
}
```

The ViewModel should handle actions such as:

```
addProduct()
removeItem()
```

```
changeQuantity()
applyDiscount()
calculateTotals()
selectPaymentMethod()
completeSale()
holdSale()
resumeSale()
```

Widgets should display state and send events. They should not directly calculate taxes, update inventory, call APIs, or write to the database. Flutter recommends keeping business logic outside widgets and using ViewModels or equivalent state holders for UI logic. [22]

State management

Use one consistent state-management approach across the project:

- **Riverpod:** strong option for dependency injection, testability, and feature-based state.
- **Bloc/Cubit:** suitable when your team prefers explicit events and state transitions.
- **ChangeNotifier:** acceptable for simpler applications and is included in Flutter's architecture recommendations as one possible mechanism. [22]

For a serious POS system, I would generally choose **Riverpod or Bloc**, with immutable state objects and clearly defined loading, success, empty, and error states.

Database and backend

A practical deployment model is:

```
POS device
├── Local SQLite database
├── Flutter application
└── Sync queue
        ↓
Backend API
├── Authentication
├── Product and pricing service
├── Sales service
├── Inventory service
├── Payment service
└── Reporting database
```

Use a local relational database because billing data has relationships between sales, sale items, payments, customers, and stock movements. The backend should enforce invoice numbering, authorization, tax rules, and conflict handling rather than trusting only the client.

Hardware integration

Create an abstraction so the business logic does not depend on a particular printer or scanner:

```
abstract class ReceiptPrinter {
  Future<void> printReceipt(SaleReceipt receipt);
  Future<List<PrinterDevice>> discover();
}

abstract class BarcodeScanner {
  Future<String?> scan();
}
```

Then provide platform-specific implementations for Bluetooth, USB, LAN, or camera scanning. This allows you to replace a printer library without changing checkout logic.

Security and audit

Include these from the beginning:

- Role-based permissions for cashier, supervisor, and administrator.
- Secure storage for tokens and device credentials.
- Audit records for voids, discounts, refunds, price changes, and stock adjustments.
- Idempotency keys to prevent duplicate sales after retries.
- Server-side validation for totals, tax, and permissions.
- Encrypted backups and controlled access to customer data.

Recommended first release

Build the first version in this order:

1. Login and user roles.
2. Product catalog and barcode search.
3. Cart and tax calculation.
4. Cash checkout.
5. Local receipt printing.
6. Local sales history.
7. Inventory movement tracking.
8. Offline-to-online synchronization.
9. Card or digital-payment integration.
10. Reports and returns.

For a Billmaster-like project, the most important architectural decision is to make **checkout transactional, local-first, and auditable**. The UI can later be redesigned without changing the core sales, inventory, payment, and synchronization logic.

Guide me create a flutter application for the above

Yes. Build it incrementally as an **offline-first Flutter billing and POS application**. Start with products, cart, cash checkout, local storage, and receipt printing; add cloud synchronization, online payments, inventory, and reports afterward.

1. Define the first version

Use this MVP scope:

- Product catalog.
- Barcode or SKU search.
- Shopping cart.
- Quantity changes and item removal.
- Discount and tax calculation.
- Cash payment.
- Completed-sales history.
- Local SQLite persistence.
- Basic receipt preview.

Flutter's official architecture separates views, view models, repositories, and services, which is a good foundation for this application. [31]

2. Create the project

Install Flutter and verify your environment:

```
flutter doctor
```

Create the application:

```
flutter create pos_billing_app
cd pos_billing_app
flutter run
```

Flutter officially supports creating a project with `flutter create` and running it with `flutter run`. [32] [33]

Add initial packages:

```
flutter pub add flutter_riverpod
flutter pub add go_router
flutter pub add sqflite path
```

```
flutter pub add uuid
flutter pub add intl
```

Later, add packages for barcode scanning, thermal printing, HTTP communication, secure storage, and connectivity detection.

3. Create the folders

Inside `lib`, create:

```
lib/
├── main.dart
├── app/
│   ├── app.dart
│   ├── router.dart
│   └── theme.dart
├── core/
│   ├── database/
│   ├── errors/
│   ├── money/
│   └── printing/
└── features/
    ├── products/
    │   ├── data/
    │   ├── domain/
    │   └── presentation/
    ├── cart/
    │   ├── domain/
    │   └── presentation/
    ├── checkout/
    │   ├── domain/
    │   └── presentation/
    └── sales/
        ├── data/
        ├── domain/
        └── presentation/
```

Use feature-based organization. A feature should contain its screen, ViewModel, repository, services, and models rather than scattering related files across global folders.

4. Define the data model

Start with these Dart models:

```
class Product {
  final String id;
  final String barcode;
  final String name;
  final int priceCents;
  final int taxRate;
  final int stockQuantity;

  const Product({
```

```
    required this.id,
    required this.barcode,
    required this.name,
    required this.priceCents,
    required this.taxRate,
    required this.stockQuantity,
  });
}
```

Represent money as integer cents rather than `double`. For example, `1299` means €12.99. This avoids many floating-point rounding problems during billing.

Create:

```
CartItem
Sale
SaleItem
Payment
StockMovement
Customer
```

A sale should contain a unique ID, invoice number, totals, payment status, cashier, creation time, and synchronization status.

5. Add SQLite persistence

Flutter's SQLite recipe uses the `sqflite` plugin together with `path`; the normal process is to open the database, create tables, and then perform insert, query, update, and delete operations.[34] [35]

Create `lib/core/database/app_database.dart`:

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;

    final databasesPath = await getDatabasesPath();
    final path = join(databasesPath, 'pos_billing.db');

    _database = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE products (
            id TEXT PRIMARY KEY,
            barcode TEXT NOT NULL UNIQUE,
```

```
            name TEXT NOT NULL,
            price_cents INTEGER NOT NULL,
            tax_rate INTEGER NOT NULL,
            stock_quantity INTEGER NOT NULL
          )
        ''');

        await db.execute('''
          CREATE TABLE sales (
            id TEXT PRIMARY KEY,
            invoice_number TEXT NOT NULL UNIQUE,
            subtotal_cents INTEGER NOT NULL,
            discount_cents INTEGER NOT NULL,
            tax_cents INTEGER NOT NULL,
            total_cents INTEGER NOT NULL,
            payment_method TEXT NOT NULL,
            created_at TEXT NOT NULL,
            sync_status TEXT NOT NULL
          )
        ''');

        await db.execute('''
          CREATE TABLE sale_items (
            id TEXT PRIMARY KEY,
            sale_id TEXT NOT NULL,
            product_id TEXT NOT NULL,
            quantity INTEGER NOT NULL,
            unit_price_cents INTEGER NOT NULL,
            total_cents INTEGER NOT NULL
          )
        ''');
      },
    );

    return _database!;
  }
}
```

Do not let screens call SQLite directly. Put database operations in services and repositories.

6. Implement the product repository

```
class ProductRepository {
  final AppDatabase database;

  ProductRepository(this.database);

  Future<List<Product>> search(String text) async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'name LIKE ? OR barcode LIKE ?',
      whereArgs: ['%$text%', '%$text%'],
      orderBy: 'name',
```

```
    );

    return rows.map(ProductMapper.fromMap).toList();
  }
}
```

The repository becomes the source of truth for product data. It can later combine local SQLite data with a remote API without forcing the UI to change. Flutter's architecture guide describes repositories as the source of truth and services as the layer that interacts with external data sources.[31]

7. Build the cart logic

Create a cart state containing:

```
items
subtotal
discount
tax
total
```

Example calculation:

```
int calculateSubtotal(List<CartItem> items) {
  return items.fold(
    0,
    (sum, item) => sum + item.unitPriceCents * item.quantity,
  );
}

int calculateTax(int taxableAmountCents, int taxRate) {
  return (taxableAmountCents * taxRate) ~/ 100;
}
```

Use a ViewModel or Riverpod notifier for commands:

```
addProduct(product)
removeProduct(productId)
increaseQuantity(productId)
decreaseQuantity(productId)
applyDiscount(amount)
clearCart()
```

The screen should only display state and call these commands. It should not contain tax, discount, stock, or database logic.

8. Add the main POS screen

The first screen can contain:

```
AppBar
├── Store name
├── Current cashier
└── Sync status

Body
├── Search field / barcode input
├── Product results
├── Cart items
└── Checkout summary

Bottom area
├── Subtotal
├── Discount
├── Tax
├── Total
└── Pay button
```

A simple layout:

```
class PosScreen extends ConsumerWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('POS Billing')),
      body: Column(
        children: [
          const ProductSearchField(),
          const Expanded(child: ProductResults()),
          CartSummary(cart: cart),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: cart.items.isEmpty
            ? null
            : () => context.push('/checkout'),
        label: const Text('Checkout'),
        icon: const Icon(Icons.payment),
      ),
    );
  }
}
```

9. Implement checkout as a transaction

When the cashier completes a sale:

1. Validate that the cart is not empty.
2. Recalculate totals.
3. Verify local stock.
4. Create the sale record.
5. Create sale-item records.
6. Create the payment record.
7. Create stock movements.
8. Commit all records in one database transaction.
9. Show the receipt.
10. Add the sale to a synchronization queue.

Conceptually:

```
await db.transaction((txn) async {
  await txn.insert('sales', sale.toMap());

  for (final item in sale.items) {
    await txn.insert('sale_items', item.toMap());

    await txn.rawUpdate(
      '''
      UPDATE products
      SET stock_quantity = stock_quantity - ?
      WHERE id = ?
      ''',
      [item.quantity, item.productId],
    );
  }
});
```

This prevents a situation where the sale is saved but the stock update fails.

10. Add offline synchronization

Use a local-first process:

```
Sale created locally
        ↓
sync_status = pending
        ↓
Internet available?
   ├── No: keep pending
   └── Yes: upload to server
```

```
                ↓
            synced or failed
```

Add a `sync_queue` table:

```
id
entity_type
entity_id
operation
payload
attempt_count
last_error
created_at
```

The app should remain usable for cash sales without an internet connection. Flutter provides an official offline-first architecture pattern for combining local and remote data sources. [36]

For duplicate protection, send an idempotency key such as:

```
deviceId + saleId
```

The server should reject a repeated request for the same key rather than creating a second invoice.

11. Add barcode scanning and printing

Keep hardware behind interfaces:

```
abstract class BarcodeScannerService {
  Future<String?> scan();
}

abstract class ReceiptPrinterService {
  Future<void> printReceipt(Sale sale);
}
```

Then implement platform-specific services separately. This means the checkout feature does not need to know whether the scanner uses a camera, Bluetooth device, USB device, or keyboard-emulation mode.

For the first version, support keyboard barcode input and receipt preview. Add physical scanner and thermal-printer integrations after the core sales flow works.

12. Add navigation

Use routes such as:

```
/
├── /login
```

```
├── /pos
├── /products
├── /inventory
├── /sales
├── /sales/:id
├── /customers
├── /reports
└── /settings
```

Use role checks before opening administrative routes:

```
Cashier:
- POS
- Customers
- Own sales

Supervisor:
- Returns
- Discounts
- Sales history

Administrator:
- Products
- Inventory
- Users
- Reports
- Settings
```

13. Test the important rules

Write unit tests before adding many screens:

- Subtotal calculation.
- Tax calculation.
- Percentage and fixed discounts.
- Quantity changes.
- Negative or zero quantities.
- Insufficient stock.
- Split payments.
- Sale persistence.
- Retry after failed synchronization.
- Duplicate synchronization requests.
- Return and cancellation rules.

Also test the complete flow:

```
Search product → Add to cart → Checkout → Save sale → Update stock → Print receipt
```

Suggested development order

Phase 1: Working local POS

- Product model.
- Product seed data.
- Product search.
- Cart.
- Tax and discount calculation.
- Cash checkout.
- SQLite sale storage.

Phase 2: Business features

- Inventory.
- Customers.
- Returns.
- Held carts.
- Receipt reprinting.
- User roles.
- Daily reports.

Phase 3: Production integrations

- Backend API.
- Offline synchronization.
- Barcode scanner.
- Thermal printer.
- Card and digital payments.
- Backup and monitoring.

Start with a local demo using seeded products. Once one complete transaction works reliably, add persistence, synchronization, and hardware integrations one at a time. This approach reduces risk and keeps the core billing flow testable.

⁂

where ever dart code is there add the files names where this dart code has to be done

Understood. From now on, every Dart code block will include the **exact file path and filename** where the code belongs. For the billing/POS application, use this structure based on Flutter's recommended separation of views, ViewModels, repositories, and services. [42]

Project structure

```
pos_billing_app/
├── lib/
│   ├── main.dart
│   │
│   ├── app/
│   │   ├── app.dart
│   │   ├── router.dart
│   │   └── theme.dart
│   │
│   ├── core/
│   │   ├── database/
│   │   │   ├── app_database.dart
│   │   │   └── database_migrations.dart
│   │   ├── errors/
│   │   │   └── app_exception.dart
│   │   ├── money/
│   │   │   └── money_formatter.dart
│   │   ├── printing/
│   │   │   └── receipt_printer_service.dart
│   │   └── barcode/
│   │       └── barcode_scanner_service.dart
│   │
│   └── features/
│       ├── products/
│       │   ├── data/
│       │   │   ├── models/
│       │   │   │   ├── product.dart
│       │   │   │   └── product_mapper.dart
│       │   │   ├── services/
│       │   │   │   └── product_local_service.dart
│       │   │   └── repositories/
│       │   │       └── product_repository.dart
│       │   └── presentation/
│       │       ├── view_models/
│       │       │   └── product_view_model.dart
│       │       ├── screens/
│       │       │   └── products_screen.dart
│       │       └── widgets/
│       │           ├── product_search_field.dart
│       │           └── product_list_item.dart
│       │
│       ├── cart/
│       │   ├── domain/
│       │   │   └── cart_item.dart
│       │   └── presentation/
```

```
│       │       ├── view_models/
│       │       │   └── cart_view_model.dart
│       │       ├── screens/
│       │       │   └── pos_screen.dart
│       │       └── widgets/
│       │           ├── cart_item_tile.dart
│       │           └── cart_summary.dart
│       │
│       ├── checkout/
│       │   ├── data/
│       │   │   └── repositories/
│       │   │       └── checkout_repository.dart
│       │   ├── domain/
│       │   │   ├── sale.dart
│       │   │   ├── sale_item.dart
│       │   │   └── payment.dart
│       │   └── presentation/
│       │       ├── view_models/
│       │       │   └── checkout_view_model.dart
│       │       ├── screens/
│       │       │   ├── checkout_screen.dart
│       │       │   └── receipt_screen.dart
│       │       └── widgets/
│       │           └── payment_method_selector.dart
│       │
│       └── sales/
│           ├── data/
│           │   └── repositories/
│           │       └── sales_repository.dart
│           └── presentation/
│               ├── screens/
│               │   ├── sales_history_screen.dart
│               │   └── sale_details_screen.dart
│               └── widgets/
│                   └── sale_list_item.dart
│
└── test/
    ├── cart_view_model_test.dart
    ├── checkout_view_model_test.dart
    └── product_repository_test.dart
```

Flutter's architecture case study supports combining feature-based organization for UI code with shared data-layer services and repositories.[43]

Step 1: Create the application entry point

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'app/app.dart';

void main() {
```

```
  runApp(const PosBillingApp());
}
```

This is the application entry point. It should remain small and should not contain database, billing, or UI-business logic.

Step 2: Create the root application

File: `lib/app/app.dart`

```
import 'package:flutter/material.dart';
import 'router.dart';
import 'theme.dart';

class PosBillingApp extends StatelessWidget {
  const PosBillingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'POS Billing',
      theme: appTheme,
      routerConfig: appRouter,
    );
  }
}
```

Step 3: Create the application theme

File: `lib/app/theme.dart`

```
import 'package:flutter/material.dart';

final ThemeData appTheme = ThemeData(
  colorScheme: ColorScheme.fromSeed(
    seedColor: Colors.indigo,
  ),
  useMaterial3: true,
  inputDecorationTheme: const InputDecorationTheme(
    border: OutlineInputBorder(),
  ),
);
```

Step 4: Create the router

File: `lib/app/router.dart`

```
import 'package:go_router/go_router.dart';
import '../features/cart/presentation/screens/pos_screen.dart';
import '../features/checkout/presentation/screens/checkout_screen.dart';
import '../features/sales/presentation/screens/sales_history_screen.dart';

final GoRouter appRouter = GoRouter(
  initialLocation: '/pos',
  routes: [
    GoRoute(
      path: '/pos',
      builder: (context, state) => const PosScreen(),
    ),
    GoRoute(
      path: '/checkout',
      builder: (context, state) => const CheckoutScreen(),
    ),
    GoRoute(
      path: '/sales',
      builder: (context, state) => const SalesHistoryScreen(),
    ),
  ],
);
```

Step 5: Create the database

File: `lib/core/database/app_database.dart`

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) {
      return _database!;
    }

    final databasePath = await getDatabasesPath();
    final path = join(databasePath, 'pos_billing.db');

    _database = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE products (
            id TEXT PRIMARY KEY,
            barcode TEXT NOT NULL UNIQUE,
            name TEXT NOT NULL,
            price_cents INTEGER NOT NULL,
```

```
            tax_rate INTEGER NOT NULL,
            stock_quantity INTEGER NOT NULL
          )
        ''');

        await db.execute('''
          CREATE TABLE sales (
            id TEXT PRIMARY KEY,
            invoice_number TEXT NOT NULL UNIQUE,
            subtotal_cents INTEGER NOT NULL,
            discount_cents INTEGER NOT NULL,
            tax_cents INTEGER NOT NULL,
            total_cents INTEGER NOT NULL,
            payment_method TEXT NOT NULL,
            created_at TEXT NOT NULL,
            sync_status TEXT NOT NULL
          )
        ''');

        await db.execute('''
          CREATE TABLE sale_items (
            id TEXT PRIMARY KEY,
            sale_id TEXT NOT NULL,
            product_id TEXT NOT NULL,
            quantity INTEGER NOT NULL,
            unit_price_cents INTEGER NOT NULL,
            total_cents INTEGER NOT NULL
          )
        ''');
      },
    );

    return _database!;
  }
}
```

This file is responsible only for opening and creating the local database. Flutter documents SQLite persistence using `sqflite` and `path`.[44]

Step 6: Create the product model

File: `lib/features/products/data/models/product.dart`

```
class Product {
  final String id;
  final String barcode;
  final String name;
  final int priceCents;
  final int taxRate;
  final int stockQuantity;

  const Product({
    required this.id,
    required this.barcode,
```

```
    required this.name,
    required this.priceCents,
    required this.taxRate,
    required this.stockQuantity,
  });

  Product copyWith({
    String? id,
    String? barcode,
    String? name,
    int? priceCents,
    int? taxRate,
    int? stockQuantity,
  }) {
    return Product(
      id: id ?? this.id,
      barcode: barcode ?? this.barcode,
      name: name ?? this.name,
      priceCents: priceCents ?? this.priceCents,
      taxRate: taxRate ?? this.taxRate,
      stockQuantity: stockQuantity ?? this.stockQuantity,
    );
  }
}
```

Step 7: Create the product database mapper

File: `lib/features/products/data/models/product_mapper.dart`

```
import 'product.dart';

class ProductMapper {
  static Product fromMap(Map<String, Object?> map) {
    return Product(
      id: map['id'] as String,
      barcode: map['barcode'] as String,
      name: map['name'] as String,
      priceCents: map['price_cents'] as int,
      taxRate: map['tax_rate'] as int,
      stockQuantity: map['stock_quantity'] as int,
    );
  }

  static Map<String, Object?> toMap(Product product) {
    return {
      'id': product.id,
      'barcode': product.barcode,
      'name': product.name,
      'price_cents': product.priceCents,
      'tax_rate': product.taxRate,
      'stock_quantity': product.stockQuantity,
    };
```

```
  }
}
```

Step 8: Create the product service

File: `lib/features/products/data/services/product_local_service.dart`

```
import '../../../../core/database/app_database.dart';
import '../models/product.dart';
import '../models/product_mapper.dart';

class ProductLocalService {
  final AppDatabase database;

  ProductLocalService(this.database);

  Future<List<Product>> searchProducts(String query) async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'name LIKE ? OR barcode LIKE ?',
      whereArgs: ['%$query%', '%$query%'],
      orderBy: 'name ASC',
    );

    return rows.map(ProductMapper.fromMap).toList();
  }

  Future<void> insertProduct(Product product) async {
    final db = await database.database;

    await db.insert(
      'products',
      ProductMapper.toMap(product),
    );
  }
}
```

A service communicates with a specific external or local data source. The repository should use this service and expose business-friendly operations to the rest of the application. [42]

Step 9: Create the product repository

File: `lib/features/products/data/repositories/product_repository.dart`

```
import '../models/product.dart';
import '../services/product_local_service.dart';

class ProductRepository {
  final ProductLocalService localService;
```

```
  ProductRepository(this.localService);

  Future<List<Product>> searchProducts(String query) {
    return localService.searchProducts(query);
  }

  Future<void> addProduct(Product product) {
    return localService.insertProduct(product);
  }
}
```

Step 10: Create the cart item

File: `lib/features/cart/domain/cart_item.dart`

```
import '../../products/data/models/product.dart';

class CartItem {
  final Product product;
  final int quantity;

  const CartItem({
    required this.product,
    required this.quantity,
  });

  int get totalCents {
    return product.priceCents * quantity;
  }

  CartItem copyWith({
    Product? product,
    int? quantity,
  }) {
    return CartItem(
      product: product ?? this.product,
      quantity: quantity ?? this.quantity,
    );
  }
}
```

Step 11: Create the cart ViewModel

File: `lib/features/cart/presentation/view_models/cart_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../domain/cart_item.dart';
import '../../../products/data/models/product.dart';

final cartProvider =
```

```
    NotifierProvider<CartViewModel, List<CartItem>>(
  CartViewModel.new,
);

class CartViewModel extends Notifier<List<CartItem>> {
  @override
  List<CartItem> build() {
    return [];
  }

  void addProduct(Product product) {
    final existingIndex = state.indexWhere(
      (item) => item.product.id == product.id,
    );

    if (existingIndex == -1) {
      state = [
        ...state,
        CartItem(product: product, quantity: 1),
      ];
      return;
    }

    final updated = [...state];
    final existing = updated[existingIndex];

    updated[existingIndex] = existing.copyWith(
      quantity: existing.quantity + 1,
    );

    state = updated;
  }

  void increaseQuantity(String productId) {
    final updated = state.map((item) {
      if (item.product.id == productId) {
        return item.copyWith(quantity: item.quantity + 1);
      }
      return item;
    }).toList();

    state = updated;
  }

  void decreaseQuantity(String productId) {
    final updated = <CartItem>[];

    for (final item in state) {
      if (item.product.id != productId) {
        updated.add(item);
        continue;
      }

      if (item.quantity > 1) {
        updated.add(item.copyWith(quantity: item.quantity - 1));
      }
```

```
    }

    state = updated;
  }

  void removeProduct(String productId) {
    state = state
        .where((item) => item.product.id != productId)
        .toList();
  }

  void clear() {
    state = [];
  }

  int get subtotalCents {
    return state.fold(
      0,
      (sum, item) => sum + item.totalCents,
    );
  }

  int get taxCents {
    return state.fold(
      0,
      (sum, item) {
        final itemTax =
            item.totalCents * item.product.taxRate ~/ 100;
        return sum + itemTax;
      },
    );
  }

  int get totalCents {
    return subtotalCents + taxCents;
  }
}
```

This file contains cart state and cart operations. It does not create widgets or write directly to SQLite.

Step 12: Create the POS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
import 'package:flutter/material.dart';
import '../widgets/cart_summary.dart';
import '../widgets/product_search_field.dart';

class PosScreen extends StatelessWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('POS Billing'),
        actions: [
          IconButton(
            icon: const Icon(Icons.receipt_long),
            onPressed: () {
              Navigator.of(context).pushNamed('/sales');
            },
          ),
        ],
      ),
      body: const Column(
        children: [
          ProductSearchField(),
          Expanded(
            child: Center(
              child: Text('Product results will appear here'),
            ),
          ),
          CartSummary(),
        ],
      ),
    );
  }
}
```

Step 13: Create the product search field

File: `lib/features/products/presentation/widgets/product_search_field.dart`

```
import 'package:flutter/material.dart';

class ProductSearchField extends StatelessWidget {
  const ProductSearchField({super.key});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: TextField(
        decoration: const InputDecoration(
          labelText: 'Search product or scan barcode',
          prefixIcon: Icon(Icons.search),
        ),
        onSubmitted: (value) {
          // The product ViewModel will handle searching.
        },
      ),
    );
  }
}
```

Step 14: Create the cart summary

File: `lib/features/cart/presentation/widgets/cart_summary.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../view_models/cart_view_model.dart';

class CartSummary extends ConsumerWidget {
  const CartSummary({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cartViewModel = ref.watch(cartProvider.notifier);
    final cart = ref.watch(cartProvider);

    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Text('Items: ${cart.length}'),
            Text('Subtotal: ${formatMoney(cartViewModel.subtotalCents)}'),
            Text('Tax: ${formatMoney(cartViewModel.taxCents)}'),
            Text(
              'Total: ${formatMoney(cartViewModel.totalCents)}',
              style: const TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 12),
            SizedBox(
              width: double.infinity,
              child: ElevatedButton(
                onPressed: cart.isEmpty
                    ? null
                    : () {
                        Navigator.of(context).pushNamed('/checkout');
                      },
                child: const Text('Checkout'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Because this widget uses Riverpod, update `main.dart` to include `ProviderScope`.

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'app/app.dart';

void main() {
  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Step 15: Create the checkout models

File: `lib/features/checkout/domain/sale_item.dart`

```
class SaleItem {
  final String id;
  final String saleId;
  final String productId;
  final int quantity;
  final int unitPriceCents;
  final int totalCents;

  const SaleItem({
    required this.id,
    required this.saleId,
    required this.productId,
    required this.quantity,
    required this.unitPriceCents,
    required this.totalCents,
  });
}
```

File: `lib/features/checkout/domain/payment.dart`

```
enum PaymentMethod {
  cash,
  card,
  digital,
}
```

File: `lib/features/checkout/domain/sale.dart`

```
import 'payment.dart';
import 'sale_item.dart';

class Sale {
  final String id;
  final String invoiceNumber;
  final List<SaleItem> items;
  final int subtotalCents;
  final int discountCents;
  final int taxCents;
  final int totalCents;
  final PaymentMethod paymentMethod;
  final DateTime createdAt;
  final String syncStatus;

  const Sale({
    required this.id,
    required this.invoiceNumber,
    required this.items,
    required this.subtotalCents,
    required this.discountCents,
    required this.taxCents,
    required this.totalCents,
    required this.paymentMethod,
    required this.createdAt,
    required this.syncStatus,
  });
}
```

Step 16: Create the checkout screen

File: `lib/features/checkout/presentation/screens/checkout_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../../cart/presentation/view_models/cart_view_model.dart';

class CheckoutScreen extends ConsumerWidget {
  const CheckoutScreen({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);
    final cartViewModel = ref.watch(cartProvider.notifier);

    return Scaffold(
      appBar: AppBar(
```

```
        title: const Text('Checkout'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text(
              'Total: ${formatMoney(cartViewModel.totalCents)}',
              style: const TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: cart.isEmpty
                  ? null
                  : () {
                      // Save the sale through CheckoutViewModel.
                    },
              child: const Text('Pay with cash'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Step 17: Fix route navigation

Because the project uses `GoRouter`, use `context.push()` instead of `Navigator.pushNamed()`.

File: `lib/features/cart/presentation/widgets/cart_summary.dart`

Add this import:

```
import 'package:go_router/go_router.dart';
```

Replace:

```
Navigator.of(context).pushNamed('/checkout');
```

with:

```
context.push('/checkout');
```

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Add this import:

```
import 'package:go_router/go_router.dart';
```

Replace:

```
Navigator.of(context).pushNamed('/sales');
```

with:

```
context.push('/sales');
```

Step 18: Create placeholder sales screens

File: `lib/features/sales/presentation/screens/sales_history_screen.dart`

```
import 'package:flutter/material.dart';

class SalesHistoryScreen extends StatelessWidget {
  const SalesHistoryScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Sales History'),
      ),
      body: const Center(
        child: Text('Completed sales will appear here'),
      ),
    );
  }
}
```

Step 19: Run the application

From the project root:

```
flutter pub get
dart format lib
flutter analyze
flutter run
```

At this stage, you should have:

- A Flutter application shell.

- App routing.
- A POS screen.
- A cart ViewModel.
- Product, sale, and payment models.
- A SQLite database foundation.
- A checkout screen placeholder.
- File locations clearly defined for each Dart class.

Next files to implement

The next development step should add these files:

```
lib/features/products/presentation/view_models/product_view_model.dart
lib/features/products/presentation/widgets/product_list_item.dart
lib/features/cart/presentation/widgets/cart_item_tile.dart
lib/features/checkout/presentation/view_models/checkout_view_model.dart
lib/features/checkout/data/repositories/checkout_repository.dart
lib/core/printing/receipt_printer_service.dart
lib/core/barcode/barcode_scanner_service.dart
```

The correct implementation sequence is **product search → add product to cart → display cart items → save checkout transaction → update stock → show receipt**. Keep each code example tied to one exact file path, as requested.

⁂

while doing the Analyze getting ProductSearchField not defined

The error occurs because `ProductSearchField` is being used in `pos_screen.dart`, but the file containing that widget is either missing or not imported. In Dart, a custom widget must be defined in a `.dart` file and imported before it can be used.[53]

Create the widget file

Create this file:

```
lib/features/products/presentation/widgets/product_search_field.dart
```

File: `lib/features/products/presentation/widgets/product_search_field.dart`

```
import 'package:flutter/material.dart';

class ProductSearchField extends StatelessWidget {
  const ProductSearchField({
    super.key,
    this.onSubmitted,
  });

  final ValueChanged<String>? onSubmitted;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: TextField(
        decoration: const InputDecoration(
          labelText: 'Search product or scan barcode',
          hintText: 'Enter product name or barcode',
          prefixIcon: Icon(Icons.search),
        ),
        onSubmitted: onSubmitted,
      ),
    );
  }
}
```

Check that the filename is exactly:

```
product_search_field.dart
```

File names should use lowercase letters and underscores.

Import the widget

Open this file:

```
lib/features/cart/presentation/screens/pos_screen.dart
```

Make sure the imports include:

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../../../products/presentation/widgets/product_search_field.dart';
import '../widgets/cart_summary.dart';
```

The relative import is correct because:

```
pos_screen.dart
└── presentation/
    └── screens/
```

and the target file is:

```
products/
└── presentation/
    └── widgets/
        └── product_search_field.dart
```

From the `screens` folder:

```
../        → presentation
../../     → cart
../../../  → features
```

Therefore:

```
import '../../../products/presentation/widgets/product_search_field.dart';
```

Use it in the screen

Your complete POS screen should be:

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../../../products/presentation/widgets/product_search_field.dart';
import '../widgets/cart_summary.dart';

class PosScreen extends StatelessWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('POS Billing'),
        actions: [
          IconButton(
            icon: const Icon(Icons.receipt_long),
            onPressed: () {
              context.push('/sales');
            },
```

```
          ),
        ],
      ),
      body: const Column(
        children: [
          ProductSearchField(),
          Expanded(
            child: Center(
              child: Text('Product results will appear here'),
            ),
          ),
          CartSummary(),
        ],
      ),
    );
  }
}
```

If the error still appears

Run these commands from the project root:

```
dart format lib
flutter pub get
flutter analyze
```

If the analyzer still shows an old error, run:

```
flutter clean
flutter pub get
flutter analyze
```

Also verify that you did not accidentally create the file in this incorrect location:

```
lib/product_search_field.dart
```

The required location is:

```
lib/features/products/presentation/widgets/product_search_field.dart
```

Alternative package import

You can also use a package import instead of a relative import. Replace `pos_billing_app` with the package name from your `pubspec.yaml`.

```
import 'package:pos_billing_app/features/products/presentation/widgets/product_search
```

The relative import is simpler for now and should work with the folder structure above.

⁂

Add a color theme and Icon to the app

Add the theme in `theme.dart`, connect it through `MaterialApp.router`, and use `Icon` widgets in the app bar, search field, cart, and checkout. Flutter recommends using a shared `ThemeData` and `ColorScheme` so Material widgets receive consistent colors and typography throughout the application.[63] [64]

1. Update the color theme

File: `lib/app/theme.dart`

Replace the existing content with:

```
import 'package:flutter/material.dart';

final ColorScheme appColorScheme = ColorScheme.fromSeed(
  seedColor: const Color(0xFF1565C0),
  brightness: Brightness.light,
);

final ThemeData appTheme = ThemeData(
  useMaterial3: true,
  colorScheme: appColorScheme,

  scaffoldBackgroundColor: const Color(0xFFF5F7FA),

  appBarTheme: AppBarTheme(
    backgroundColor: appColorScheme.primary,
    foregroundColor: appColorScheme.onPrimary,
    centerTitle: false,
    elevation: 0,
    iconTheme: IconThemeData(
      color: appColorScheme.onPrimary,
    ),
  ),

  cardTheme: CardThemeData(
    color: Colors.white,
    elevation: 1,
    margin: const EdgeInsets.all(8),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),

  inputDecorationTheme: InputDecorationTheme(
    filled: true,
    fillColor: Colors.white,
    prefixIconColor: appColorScheme.primary,
    border: OutlineInputBorder(
```

```
      borderRadius: BorderRadius.circular(10),
      borderSide: BorderSide.none,
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(10),
      borderSide: BorderSide(
        color: appColorScheme.outline.withValues(alpha: 0.4),
      ),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(10),
      borderSide: BorderSide(
        color: appColorScheme.primary,
        width: 2,
      ),
    ),
  ),

  elevatedButtonTheme: ElevatedButtonThemeData(
    style: ElevatedButton.styleFrom(
      backgroundColor: appColorScheme.primary,
      foregroundColor: appColorScheme.onPrimary,
      minimumSize: const Size(double.infinity, 50),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(10),
      ),
    ),
  ),

  filledButtonTheme: FilledButtonThemeData(
    style: FilledButton.styleFrom(
      minimumSize: const Size(double.infinity, 50),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(10),
      ),
    ),
  ),

  iconTheme: IconThemeData(
    color: appColorScheme.primary,
    size: 24,
  ),

  textTheme: const TextTheme(
    headlineSmall: TextStyle(
      fontWeight: FontWeight.bold,
    ),
    titleLarge: TextStyle(
      fontWeight: FontWeight.w600,
    ),
    bodyLarge: TextStyle(
      fontSize: 16,
    ),
  ),
);
```

`ColorScheme.fromSeed` generates a coordinated Material 3 palette from one seed color, while component themes customize app bars, buttons, cards, inputs, and icons globally. [65] [66]

2. Confirm the app uses the theme

File: `lib/app/app.dart`

Make sure the file contains:

```
import 'package:flutter/material.dart';

import 'router.dart';
import 'theme.dart';

class PosBillingApp extends StatelessWidget {
  const PosBillingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'POS Billing',
      theme: appTheme,
      routerConfig: appRouter,
    );
  }
}
```

The `theme: appTheme` line applies the theme to the entire application. Flutter's theme system allows colors, typography, and component styling to be shared across all descendant widgets. [64]

3. Add icons to the POS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../../../products/presentation/widgets/product_search_field.dart';
import '../widgets/cart_summary.dart';

class PosScreen extends StatelessWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: const Icon(Icons.point_of_sale),
```

```
        title: const Text('POS Billing'),
        actions: [
          IconButton(
            tooltip: 'Sales history',
            icon: const Icon(Icons.receipt_long),
            onPressed: () {
              context.push('/sales');
            },
          ),
          IconButton(
            tooltip: 'Settings',
            icon: const Icon(Icons.settings),
            onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                  content: Text('Settings will be added later'),
                ),
              );
            },
          ),
        ],
      ),
      body: const Column(
        children: [
          ProductSearchField(),
          Expanded(
            child: Center(
              child: Icon(
                Icons.shopping_cart_outlined,
                size: 80,
                color: Colors.grey,
              ),
            ),
          ),
          CartSummary(),
        ],
      ),
    );
  }
}
```

Flutter's `Icon` widget displays an `IconData`, such as the predefined icons in the `Icons` class.[67]

4. Add an icon to the search field

File: `lib/features/products/presentation/widgets/product_search_field.dart`

Replace the file with:

```
import 'package:flutter/material.dart';

class ProductSearchField extends StatelessWidget {
  const ProductSearchField({
    super.key,
```

```
    this.onSubmitted,
  });

  final ValueChanged<String>? onSubmitted;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: TextField(
        decoration: InputDecoration(
          labelText: 'Search product or scan barcode',
          hintText: 'Enter product name or barcode',
          prefixIcon: const Icon(Icons.search),
          suffixIcon: IconButton(
            tooltip: 'Scan barcode',
            icon: const Icon(Icons.qr_code_scanner),
            onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                  content: Text('Barcode scanner will be added later'),
                ),
              );
            },
          ),
        ),
        onSubmitted: onSubmitted,
      ),
    );
  }
}
```

The prefix icon is used for searching, while the suffix icon represents barcode scanning. The `InputDecorationTheme` from `theme.dart` automatically applies the shared input styling.

5. Add icons to the cart summary

File: `lib/features/cart/presentation/widgets/cart_summary.dart`

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../view_models/cart_view_model.dart';

class CartSummary extends ConsumerWidget {
  const CartSummary({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }
```

```
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cartViewModel = ref.watch(cartProvider.notifier);
    final cart = ref.watch(cartProvider);

    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Row(
              children: [
                Icon(
                  Icons.shopping_cart,
                  color: Theme.of(context).colorScheme.primary,
                ),
                const SizedBox(width: 8),
                Text(
                  'Current cart',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
              ],
            ),
            const SizedBox(height: 12),
            _SummaryRow(
              icon: Icons.inventory_2_outlined,
              label: 'Items',
              value: '${cart.length}',
            ),
            _SummaryRow(
              icon: Icons.calculate_outlined,
              label: 'Subtotal',
              value: formatMoney(cartViewModel.subtotalCents),
            ),
            _SummaryRow(
              icon: Icons.receipt_outlined,
              label: 'Tax',
              value: formatMoney(cartViewModel.taxCents),
            ),
            const Divider(),
            _SummaryRow(
              icon: Icons.payments_outlined,
              label: 'Total',
              value: formatMoney(cartViewModel.totalCents),
              isTotal: true,
            ),
            const SizedBox(height: 12),
            FilledButton.icon(
              onPressed: cart.isEmpty
                  ? null
                  : () {
                      context.push('/checkout');
                    },
              icon: const Icon(Icons.payment),
              label: const Text('Checkout'),
```

```
            ),
          ],
        ),
      ),
    );
  }
}

class _SummaryRow extends StatelessWidget {
  const _SummaryRow({
    required this.icon,
    required this.label,
    required this.value,
    this.isTotal = false,
  });

  final IconData icon;
  final String label;
  final String value;
  final bool isTotal;

  @override
  Widget build(BuildContext context) {
    final textStyle = isTotal
        ? Theme.of(context).textTheme.titleLarge?.copyWith(
            fontWeight: FontWeight.bold,
          )
        : Theme.of(context).textTheme.bodyLarge;

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 4),
      child: Row(
        children: [
          Icon(
            icon,
            size: 20,
            color: Theme.of(context).colorScheme.primary,
          ),
          const SizedBox(width: 8),
          Expanded(
            child: Text(
              label,
              style: textStyle,
            ),
          ),
          Text(
            value,
            style: textStyle,
          ),
        ],
      ),
    );
  }
}
```

6. Add icons to checkout

File: `lib/features/checkout/presentation/screens/checkout_screen.dart`

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';

class CheckoutScreen extends ConsumerWidget {
  const CheckoutScreen({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);
    final cartViewModel = ref.watch(cartProvider.notifier);
    final colorScheme = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(
        leading: const Icon(Icons.payment),
        title: const Text('Checkout'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Card(
              child: Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                  children: [
                    Icon(
                      Icons.receipt_long,
                      size: 64,
                      color: colorScheme.primary,
                    ),
                    const SizedBox(height: 12),
                    Text(
                      'Amount to pay',
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                    const SizedBox(height: 8),
                    Text(
                      formatMoney(cartViewModel.totalCents),
                      style: Theme.of(context)
                          .textTheme
                          .headlineSmall
```

```
                          ?.copyWith(
                            color: colorScheme.primary,
                            fontWeight: FontWeight.bold,
                          ),
                    ),
                  ],
                ),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: cart.isEmpty
                  ? null
                  : () {
                      ScaffoldMessenger.of(context).showSnackBar(
                        const SnackBar(
                          content: Text(
                            'Sale processing will be added next',
                          ),
                        ),
                      );
                    },
              icon: const Icon(Icons.money),
              label: const Text('Pay with cash'),
            ),
            const SizedBox(height: 12),
            OutlinedButton.icon(
              onPressed: cart.isEmpty ? null : () {},
              icon: const Icon(Icons.credit_card),
              label: const Text('Pay by card'),
            ),
          ],
        ),
      ),
    );
  }
}
```

7. Update the sales icon screen

File: `lib/features/sales/presentation/screens/sales_history_screen.dart`

```
import 'package:flutter/material.dart';

class SalesHistoryScreen extends StatelessWidget {
  const SalesHistoryScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: const Icon(Icons.receipt_long),
        title: const Text('Sales History'),
      ),
```

```
      body: const Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(
              Icons.receipt_long_outlined,
              size: 80,
              color: Colors.grey,
            ),
            SizedBox(height: 12),
            Text('Completed sales will appear here'),
          ],
        ),
      ),
    );
  }
}
```

8. Run the analyzer

Execute:

```
dart format lib
flutter pub get
flutter analyze
flutter run
```

If you receive an error for `CardThemeData`, replace it with `CardTheme` in `lib/app/theme.dart`:

```
cardTheme: CardTheme(
  color: Colors.white,
  elevation: 1,
  margin: const EdgeInsets.all(8),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(12),
  ),
),
```

The important files are now:

```
lib/app/theme.dart
lib/app/app.dart
lib/features/cart/presentation/screens/pos_screen.dart
lib/features/cart/presentation/widgets/cart_summary.dart
lib/features/products/presentation/widgets/product_search_field.dart
lib/features/checkout/presentation/screens/checkout_screen.dart
lib/features/sales/presentation/screens/sales_history_screen.dart
```

⁂

Need to add a left navigation menu

For a POS application, add a persistent left navigation menu on tablet/desktop and keep the drawer behavior for smaller screens. Flutter provides `NavigationDrawer` for a Material 3 drawer and `NavigationRail` for persistent leading navigation on larger screens. [73] [74]

Create the menu widget

Create this file:

```
lib/app/widgets/side_navigation_menu.dart
```

File: `lib/app/widgets/side_navigation_menu.dart`

```
import 'package:flutter/material.dart';

class SideNavigationMenu extends StatelessWidget {
  const SideNavigationMenu({
    super.key,
    required this.selectedIndex,
    required this.onDestinationSelected,
  });

  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    return NavigationDrawer(
      selectedIndex: selectedIndex,
      onDestinationSelected: onDestinationSelected,
      children: const [
        Padding(
          padding: EdgeInsets.fromLTRB(28, 24, 16, 16),
          child: Row(
            children: [
              Icon(Icons.point_of_sale, size: 32),
              SizedBox(width: 12),
              Text(
                'POS Billing',
                style: TextStyle(
                  fontSize: 20,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
        ),
        Divider(),
        NavigationDrawerDestination(
          icon: Icon(Icons.dashboard_outlined),
          selectedIcon: Icon(Icons.dashboard),
```

```
            label: Text('Dashboard'),
          ),
          NavigationDrawerDestination(
            icon: Icon(Icons.point_of_sale_outlined),
            selectedIcon: Icon(Icons.point_of_sale),
            label: Text('Point of Sale'),
          ),
          NavigationDrawerDestination(
            icon: Icon(Icons.inventory_2_outlined),
            selectedIcon: Icon(Icons.inventory_2),
            label: Text('Products'),
          ),
          NavigationDrawerDestination(
            icon: Icon(Icons.warehouse_outlined),
            selectedIcon: Icon(Icons.warehouse),
            label: Text('Inventory'),
          ),
          NavigationDrawerDestination(
            icon: Icon(Icons.receipt_long_outlined),
            selectedIcon: Icon(Icons.receipt_long),
            label: Text('Sales'),
          ),
          NavigationDrawerDestination(
            icon: Icon(Icons.people_outline),
            selectedIcon: Icon(Icons.people),
            label: Text('Customers'),
          ),
          NavigationDrawerDestination(
            icon: Icon(Icons.bar_chart_outlined),
            selectedIcon: Icon(Icons.bar_chart),
            label: Text('Reports'),
          ),
          Divider(),
          NavigationDrawerDestination(
            icon: Icon(Icons.settings_outlined),
            selectedIcon: Icon(Icons.settings),
            label: Text('Settings'),
          ),
        ],
      );
    }
  }
```

`NavigationDrawer` accepts destinations and reports the selected destination index through `onDestinationSelected`.[73]

Update the POS screen

Add the menu to the `Scaffold.drawer` property.

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_menu.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../widgets/cart_summary.dart';

class PosScreen extends StatelessWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      drawer: SideNavigationMenu(
        selectedIndex: 1,
        onDestinationSelected: (index) {
          _navigateToDestination(context, index);
        },
      ),
      appBar: AppBar(
        leading: const Icon(Icons.point_of_sale),
        title: const Text('Point of Sale'),
        actions: [
          IconButton(
            tooltip: 'Sales history',
            icon: const Icon(Icons.receipt_long),
            onPressed: () {
              context.push('/sales');
            },
          ),
          IconButton(
            tooltip: 'Settings',
            icon: const Icon(Icons.settings),
            onPressed: () {
              context.push('/settings');
            },
          ),
        ],
      ),
      body: const Column(
        children: [
          ProductSearchField(),
          Expanded(
            child: Center(
              child: Icon(
                Icons.shopping_cart_outlined,
                size: 80,
                color: Colors.grey,
              ),
            ),
          ),
          CartSummary(),
        ],
      ),
```

```
    );
  }

  void _navigateToDestination(
    BuildContext context,
    int index,
  ) {
    Navigator.of(context).pop();

    switch (index) {
      case 0:
        context.go('/dashboard');
      case 1:
        context.go('/pos');
      case 2:
        context.go('/products');
      case 3:
        context.go('/inventory');
      case 4:
        context.go('/sales');
      case 5:
        context.go('/customers');
      case 6:
        context.go('/reports');
      case 7:
        context.go('/settings');
    }
  }
}
```

The `Scaffold.drawer` property places the navigation menu on the left, and Flutter automatically provides a menu button in the app bar when a drawer is available. [75]

Correct import path

From:

```
lib/features/cart/presentation/screens/pos_screen.dart
```

to:

```
lib/app/widgets/side_navigation_menu.dart
```

the correct import is:

```
import '../../../../app/widgets/side_navigation_menu.dart';
```

If your editor reports that the import cannot be found, confirm that the folders are exactly:

```
lib/
├── app/
│   └── widgets/
│       └── side_navigation_menu.dart
└── features/
    └── cart/
        └── presentation/
            └── screens/
                └── pos_screen.dart
```

Add placeholder routes

The navigation callbacks refer to routes that must exist. Add these screens first.

File: `lib/app/screens/placeholder_screen.dart`

```
import 'package:flutter/material.dart';

class PlaceholderScreen extends StatelessWidget {
  const PlaceholderScreen({
    super.key,
    required this.title,
    required this.icon,
  });

  final String title;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: Icon(icon),
        title: Text(title),
      ),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(
              icon,
              size: 80,
              color: Theme.of(context).colorScheme.primary,
            ),
            const SizedBox(height: 16),
            Text('$title screen'),
          ],
        ),
      ),
    );
  }
}
```

Update the router

File: `lib/app/router.dart`

Replace the file with:

```
import 'package:go_router/go_router.dart';

import '../features/cart/presentation/screens/pos_screen.dart';
import '../features/checkout/presentation/screens/checkout_screen.dart';
import '../features/sales/presentation/screens/sales_history_screen.dart';
import 'screens/placeholder_screen.dart';

final GoRouter appRouter = GoRouter(
  initialLocation: '/pos',
  routes: [
    GoRoute(
      path: '/dashboard',
      builder: (context, state) {
        return const PlaceholderScreen(
          title: 'Dashboard',
          icon: Icons.dashboard,
        );
      },
    ),
    GoRoute(
      path: '/pos',
      builder: (context, state) => const PosScreen(),
    ),
    GoRoute(
      path: '/products',
      builder: (context, state) {
        return const PlaceholderScreen(
          title: 'Products',
          icon: Icons.inventory_2,
        );
      },
    ),
    GoRoute(
      path: '/inventory',
      builder: (context, state) {
        return const PlaceholderScreen(
          title: 'Inventory',
          icon: Icons.warehouse,
        );
      },
    ),
    GoRoute(
      path: '/sales',
      builder: (context, state) {
        return const SalesHistoryScreen();
      },
    ),
    GoRoute(
      path: '/customers',
```

```
      builder: (context, state) {
        return const PlaceholderScreen(
          title: 'Customers',
          icon: Icons.people,
        );
      },
    ),
    GoRoute(
      path: '/reports',
      builder: (context, state) {
        return const PlaceholderScreen(
          title: 'Reports',
          icon: Icons.bar_chart,
        );
      },
    ),
    GoRoute(
      path: '/settings',
      builder: (context, state) {
        return const PlaceholderScreen(
          title: 'Settings',
          icon: Icons.settings,
        );
      },
    ),
    GoRoute(
      path: '/checkout',
      builder: (context, state) => const CheckoutScreen(),
    ),
  ],
);
```

You need this import because the placeholder route uses `Icons`:

```
import 'package:flutter/material.dart';
```

Therefore, the complete beginning of `router.dart` should be:

File: `lib/app/router.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
```

Important correction

The `switch` syntax shown earlier uses Dart's modern case behavior. If your Dart version reports a switch error, use explicit `break` statements:

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
void _navigateToDestination(
  BuildContext context,
  int index,
) {
  Navigator.of(context).pop();

  switch (index) {
    case 0:
      context.go('/dashboard');
      break;
    case 1:
      context.go('/pos');
      break;
    case 2:
      context.go('/products');
      break;
    case 3:
      context.go('/inventory');
      break;
    case 4:
      context.go('/sales');
      break;
    case 5:
      context.go('/customers');
      break;
    case 6:
      context.go('/reports');
      break;
    case 7:
      context.go('/settings');
      break;
  }
}
```

Run the application

```
dart format lib
flutter pub get
flutter analyze
flutter run
```

On a phone, the menu opens from the left using the app-bar menu button. On larger screens, a persistent `NavigationRail` or expanded rail may provide a better layout than a temporary drawer; Flutter's Material guidance distinguishes navigation drawers for larger destinations and navigation rails for persistent side navigation.[76] [77]

⁂

the navigation bar is not appearing in windows browber

The menu is not appearing because `Scaffold.drawer` creates a **hidden slide-out drawer**, not a permanently visible left navigation bar. On Windows/web, use `NavigationRail` or a permanent side panel; Flutter's responsive guidance recommends `NavigationRail` for wider layouts. [83] [84]

Replace the POS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Replace the complete file with:

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../widgets/cart_summary.dart';

class PosScreen extends StatelessWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Point of Sale'),
        actions: [
          IconButton(
            tooltip: 'Sales history',
            icon: const Icon(Icons.receipt_long),
            onPressed: () {
              context.go('/sales');
            },
          ),
          IconButton(
            tooltip: 'Settings',
            icon: const Icon(Icons.settings),
            onPressed: () {
              context.go('/settings');
            },
          ),
        ],
      ),
      body: Row(
        children: [
          SideNavigationRail(
            selectedIndex: 1,
            onDestinationSelected: (index) {
              _navigateToDestination(context, index);
            },
          ),
```

```
          const VerticalDivider(
            width: 1,
            thickness: 1,
          ),
          const Expanded(
            child: Column(
              children: [
                ProductSearchField(),
                Expanded(
                  child: Center(
                    child: Icon(
                      Icons.shopping_cart_outlined,
                      size: 80,
                      color: Colors.grey,
                    ),
                  ),
                ),
                CartSummary(),
              ],
            ),
          ),
        ],
      ),
    );
  }

  void _navigateToDestination(
    BuildContext context,
    int index,
  ) {
    switch (index) {
      case 0:
        context.go('/dashboard');
        break;
      case 1:
        context.go('/pos');
        break;
      case 2:
        context.go('/products');
        break;
      case 3:
        context.go('/inventory');
        break;
      case 4:
        context.go('/sales');
        break;
      case 5:
        context.go('/customers');
        break;
      case 6:
        context.go('/reports');
        break;
      case 7:
        context.go('/settings');
        break;
    }
```

```
  }
}
```

Create the navigation rail

Create this file:

```
lib/app/widgets/side_navigation_rail.dart
```

File: `lib/app/widgets/side_navigation_rail.dart`

```
import 'package:flutter/material.dart';

class SideNavigationRail extends StatelessWidget {
  const SideNavigationRail({
    super.key,
    required this.selectedIndex,
    required this.onDestinationSelected,
  });

  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    return NavigationRail(
      selectedIndex: selectedIndex,
      onDestinationSelected: onDestinationSelected,
      extended: true,
      minExtendedWidth: 220,
      backgroundColor: Theme.of(context).colorScheme.surface,
      leading: Padding(
        padding: const EdgeInsets.symmetric(
          vertical: 24,
          horizontal: 12,
        ),
        child: Row(
          children: [
            Icon(
              Icons.point_of_sale,
              size: 32,
              color: Theme.of(context).colorScheme.primary,
            ),
            const SizedBox(width: 12),
            Text(
              'POS Billing',
              style: Theme.of(context).textTheme.titleLarge?.copyWith(
                    fontWeight: FontWeight.bold,
                  ),
            ),
          ],
        ),
      ),
```

```
      destinations: const [
        NavigationRailDestination(
          icon: Icon(Icons.dashboard_outlined),
          selectedIcon: Icon(Icons.dashboard),
          label: Text('Dashboard'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.point_of_sale_outlined),
          selectedIcon: Icon(Icons.point_of_sale),
          label: Text('Point of Sale'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.inventory_2_outlined),
          selectedIcon: Icon(Icons.inventory_2),
          label: Text('Products'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.warehouse_outlined),
          selectedIcon: Icon(Icons.warehouse),
          label: Text('Inventory'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.receipt_long_outlined),
          selectedIcon: Icon(Icons.receipt_long),
          label: Text('Sales'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.people_outline),
          selectedIcon: Icon(Icons.people),
          label: Text('Customers'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.bar_chart_outlined),
          selectedIcon: Icon(Icons.bar_chart),
          label: Text('Reports'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.settings_outlined),
          selectedIcon: Icon(Icons.settings),
          label: Text('Settings'),
        ),
      ],
    );
  }
}
```

`NavigationRail` is designed for wider layouts such as desktop and web, and `extended: true` displays both icons and destination labels. [85] [86]

Remove the old drawer

You can keep the old file:

```
lib/app/widgets/side_navigation_menu.dart
```

but it is no longer used by `pos_screen.dart`.

You can also remove this import if it still exists:

```
import '../../../../app/widgets/side_navigation_menu.dart';
```

The new import must be:

```
import '../../../../app/widgets/side_navigation_rail.dart';
```

Check the application theme

File: `lib/app/app.dart`

Ensure that your app uses Material 3:

```
import 'package:flutter/material.dart';

import 'router.dart';
import 'theme.dart';

class PosBillingApp extends StatelessWidget {
  const PosBillingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'POS Billing',
      theme: appTheme,
      routerConfig: appRouter,
    );
  }
}
```

File: `lib/app/theme.dart`

Ensure this exists:

```
import 'package:flutter/material.dart';

final ThemeData appTheme = ThemeData(
  useMaterial3: true,
```

```
    colorScheme: ColorScheme.fromSeed(
      seedColor: const Color(0xFF1565C0),
    ),
  );
```

Run the analyzer

```
dart format lib
flutter pub get
flutter analyze
flutter run -d chrome
```

The navigation rail should now appear permanently on the left side of the Windows browser window.

If you want the menu to appear only on wide screens, add a responsive condition later. For example, below approximately 800 pixels, use the existing drawer; above that width, use the visible `NavigationRail`.

⁂

let us do the next steps. to build the below files

lib/features/products/presentation/view_models/product_view_model.dart
lib/features/products/presentation/widgets/product_list_item.dart
lib/features/cart/presentation/widgets/cart_item_tile.dart
lib/features/checkout/presentation/view_models/checkout_view_model.dart
lib/features/checkout/data/repositories/checkout_repository.dart
lib/core/printing/receipt_printer_service.dart
lib/core/barcode/barcode_scanner_service.dart

We will build the seven files in a safe order:

1. Product ViewModel.
2. Product list item.
3. Cart item tile.
4. Checkout repository.
5. Checkout ViewModel.
6. Printer service.
7. Barcode scanner service.

The structure follows Flutter's recommended flow: **View → ViewModel → Repository → Service**. Views display data, ViewModels contain UI/business logic, repositories manage application data, and services wrap local, remote, or platform APIs. [95] [96]

1. Product ViewModel

This first version uses in-memory demo products so the UI works before database dependency injection is completed.

File: `lib/features/products/presentation/view_models/product_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../data/models/product.dart';

final productViewModelProvider =
    NotifierProvider<ProductViewModel, ProductState>(
  ProductViewModel.new,
);

class ProductState {
  const ProductState({
    this.products = const [],
    this.searchText = '',
    this.isLoading = false,
    this.errorMessage,
  });

  final List<Product> products;
  final String searchText;
  final bool isLoading;
  final String? errorMessage;

  ProductState copyWith({
    List<Product>? products,
    String? searchText,
    bool? isLoading,
    String? errorMessage,
    bool clearError = false,
  }) {
    return ProductState(
      products: products ?? this.products,
      searchText: searchText ?? this.searchText,
      isLoading: isLoading ?? this.isLoading,
      errorMessage: clearError ? null : errorMessage ?? this.errorMessage,
    );
  }
}

class ProductViewModel extends Notifier<ProductState> {
  @override
  ProductState build() {
    return const ProductState(
      products: [
        Product(
          id: 'p-001',
          barcode: '8900000000001',
          name: 'Coffee',
          priceCents: 299,
```

```
          taxRate: 19,
          stockQuantity: 50,
        ),
        Product(
          id: 'p-002',
          barcode: '890000000002',
          name: 'Tea',
          priceCents: 199,
          taxRate: 19,
          stockQuantity: 40,
        ),
        Product(
          id: 'p-003',
          barcode: '890000000003',
          name: 'Sandwich',
          priceCents: 499,
          taxRate: 7,
          stockQuantity: 20,
        ),
        Product(
          id: 'p-004',
          barcode: '890000000004',
          name: 'Water',
          priceCents: 149,
          taxRate: 19,
          stockQuantity: 100,
        ),
      ],
    );
  }

  void search(String value) {
    final query = value.trim().toLowerCase();

    final allProducts = buildDemoProducts();

    final filteredProducts = query.isEmpty
        ? allProducts
        : allProducts.where((product) {
            return product.name.toLowerCase().contains(query) ||
                product.barcode.toLowerCase().contains(query);
          }).toList();

    state = state.copyWith(
      searchText: value,
      products: filteredProducts,
      clearError: true,
    );
  }

  void clearSearch() {
    search('');
  }

  List<Product> buildDemoProducts() {
    return const [
```

```
      Product(
        id: 'p-001',
        barcode: '890000000001',
        name: 'Coffee',
        priceCents: 299,
        taxRate: 19,
        stockQuantity: 50,
      ),
      Product(
        id: 'p-002',
        barcode: '890000000002',
        name: 'Tea',
        priceCents: 199,
        taxRate: 19,
        stockQuantity: 40,
      ),
      Product(
        id: 'p-003',
        barcode: '890000000003',
        name: 'Sandwich',
        priceCents: 499,
        taxRate: 7,
        stockQuantity: 20,
      ),
      Product(
        id: 'p-004',
        barcode: '890000000004',
        name: 'Water',
        priceCents: 149,
        taxRate: 19,
        stockQuantity: 100,
      ),
    ];
  }
}
```

Important note

This uses demo data temporarily. Later, replace `buildDemoProducts()` with a `ProductRepository` call. The ViewModel should not directly access SQLite; repositories are intended to be the source of truth for application data.[96]

2. Product list item

This widget displays one product and adds it to the cart when the user clicks the button.

File: `lib/features/products/presentation/widgets/product_list_item.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/models/product.dart';
```

```
class ProductListItem extends ConsumerWidget {
  const ProductListItem({
    super.key,
    required this.product,
  });

  final Product product;

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final colorScheme = Theme.of(context).colorScheme;

    return Card(
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: colorScheme.primaryContainer,
          child: Icon(
            Icons.inventory_2,
            color: colorScheme.onPrimaryContainer,
          ),
        ),
        title: Text(
          product.name,
          style: const TextStyle(
            fontWeight: FontWeight.bold,
          ),
        ),
        subtitle: Text(
          'Barcode: ${product.barcode}\n'
          'Stock: ${product.stockQuantity}',
        ),
        isThreeLine: true,
        trailing: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              formatMoney(product.priceCents),
              style: TextStyle(
                color: colorScheme.primary,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 4),
            SizedBox(
              height: 32,
              child: FilledButton(
                onPressed: product.stockQuantity <= 0
                    ? null
                    : () {
                        ref
                            .read(cartProvider.notifier)
```

```
                            .addProduct(product);

                        ScaffoldMessenger.of(context).showSnackBar(
                          SnackBar(
                            content: Text(
                              '${product.name} added to cart',
                            ),
                            duration: const Duration(seconds: 1),
                          ),
                        );
                      },
                child: const Icon(Icons.add_shopping_cart, size: 18),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

3. Cart item tile

This widget displays each item already added to the cart.

File: `lib/features/cart/presentation/widgets/cart_item_tile.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';

class CartItemTile extends ConsumerWidget {
  const CartItemTile({
    super.key,
    required this.item,
  });

  final CartItem item;

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cartViewModel = ref.read(cartProvider.notifier);

    return ListTile(
      leading: const Icon(Icons.shopping_bag_outlined),
      title: Text(item.product.name),
      subtitle: Text(
        '${formatMoney(item.product.priceCents)} × ${item.quantity}',
```

```
        ),
        trailing: SizedBox(
          width: 190,
          child: Row(
            mainAxisAlignment: MainAxisAlignment.end,
            children: [
              IconButton(
                tooltip: 'Decrease quantity',
                icon: const Icon(Icons.remove_circle_outline),
                onPressed: () {
                  cartViewModel.decreaseQuantity(item.product.id);
                },
              ),
              Text(
                '${item.quantity}',
                style: const TextStyle(
                  fontWeight: FontWeight.bold,
                ),
              ),
              IconButton(
                tooltip: 'Increase quantity',
                icon: const Icon(Icons.add_circle_outline),
                onPressed: () {
                  cartViewModel.increaseQuantity(item.product.id);
                },
              ),
              const SizedBox(width: 8),
              Text(
                formatMoney(item.totalCents),
                style: const TextStyle(
                  fontWeight: FontWeight.bold,
                ),
              ),
              IconButton(
                tooltip: 'Remove item',
                icon: const Icon(Icons.delete_outline),
                onPressed: () {
                  cartViewModel.removeProduct(item.product.id);
                },
              ),
            ],
          ),
        ),
      );
    }
  }
```

4. Update the cart ViewModel

Your current `CartViewModel` already supports adding, increasing, decreasing, and removing products. No change is required.

However, update the POS screen so that it actually displays cart items.

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Add this import:

```
import '../widgets/cart_item_tile.dart';
import '../view_models/cart_view_model.dart';
```

Because the screen is a `StatelessWidget`, change it to `ConsumerWidget`:

```
class PosScreen extends ConsumerWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Point of Sale'),
      ),
      body: Row(
        children: [
          SideNavigationRail(
            selectedIndex: 1,
            onDestinationSelected: (index) {
              _navigateToDestination(context, index);
            },
          ),
          const VerticalDivider(
            width: 1,
            thickness: 1,
          ),
          Expanded(
            child: Column(
              children: [
                const ProductSearchField(),
                Expanded(
                  child: cart.isEmpty
                      ? const Center(
                          child: Text('No products in the cart'),
                        )
                      : ListView.builder(
                          itemCount: cart.length,
                          itemBuilder: (context, index) {
                            return CartItemTile(
                              item: cart[index],
                            );
                          },
                        ),
                ),
                const CartSummary(),
              ],
            ),
          ),
```

```
          ],
        ),
      );
    }

    void _navigateToDestination(
      BuildContext context,
      int index,
    ) {
      switch (index) {
        case 0:
          context.go('/dashboard');
          break;
        case 1:
          context.go('/pos');
          break;
        case 2:
          context.go('/products');
          break;
        case 3:
          context.go('/inventory');
          break;
        case 4:
          context.go('/sales');
          break;
        case 5:
          context.go('/customers');
          break;
        case 6:
          context.go('/reports');
          break;
        case 7:
          context.go('/settings');
          break;
      }
    }
  }
```

You also need this import at the top:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

5. Improve the product search widget

The product list must be shown somewhere. Update the search widget so it displays filtered products below the search field.

File: `lib/features/products/presentation/widgets/product_search_field.dart`

Replace it with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({
    super.key,
  });

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productState = ref.watch(productViewModelProvider);

    return Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or scan barcode',
              hintText: 'Enter product name or barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: productState.searchText.isEmpty
                    ? null
                    : () {
                        ref
                            .read(productViewModelProvider.notifier)
                            .clearSearch();
                      },
              ),
            ),
            onChanged: (value) {
              ref
                  .read(productViewModelProvider.notifier)
                  .search(value);
            },
          ),
        ),
        if (productState.products.isNotEmpty)
          SizedBox(
            height: 240,
            child: ListView.builder(
              itemCount: productState.products.length,
              itemBuilder: (context, index) {
                return ProductListItem(
                  product: productState.products[index],
                );
```

```
              },
            ),
          ),
        ],
      );
    }
  }
```

6. Create the checkout repository

The repository saves the completed sale and updates the local database in one transaction.

File: `lib/features/checkout/data/repositories/checkout_repository.dart`

```
import 'package:sqflite/sqflite.dart';

import '../../../../core/database/app_database.dart';
import '../../domain/sale.dart';

class CheckoutRepository {
  CheckoutRepository({
    required this.database,
  });

  final AppDatabase database;

  Future<void> saveSale(Sale sale) async {
    final db = await database.database;

    await db.transaction((transaction) async {
      await transaction.insert(
        'sales',
        {
          'id': sale.id,
          'invoice_number': sale.invoiceNumber,
          'subtotal_cents': sale.subtotalCents,
          'discount_cents': sale.discountCents,
          'tax_cents': sale.taxCents,
          'total_cents': sale.totalCents,
          'payment_method': sale.paymentMethod.name,
          'created_at': sale.createdAt.toIso8601String(),
          'sync_status': sale.syncStatus,
        },
        conflictAlgorithm: ConflictAlgorithm.abort,
      );

      for (final item in sale.items) {
        await transaction.insert(
          'sale_items',
          {
            'id': item.id,
            'sale_id': item.saleId,
            'product_id': item.productId,
            'quantity': item.quantity,
```

```
            'unit_price_cents': item.unitPriceCents,
            'total_cents': item.totalCents,
          },
          conflictAlgorithm: ConflictAlgorithm.abort,
        );

        await transaction.rawUpdate(
          '''
          UPDATE products
          SET stock_quantity = stock_quantity - ?
          WHERE id = ?
          AND stock_quantity >= ?
          ''',
          [
            item.quantity,
            item.productId,
            item.quantity,
          ],
        );
      }
    });
  }
}
```

The repository is the correct place for transaction, caching, synchronization, and persistence logic; widgets should not write directly to the database. [96]

7. Create the checkout ViewModel

File: `lib/features/checkout/presentation/view_models/checkout_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:uuid/uuid.dart';

import '../../../cart/domain/cart_item.dart';
import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/repositories/checkout_repository.dart';
import '../../domain/payment.dart';
import '../../domain/sale.dart';
import '../../domain/sale_item.dart';
import '../../../../core/database/app_database.dart';

final checkoutRepositoryProvider = Provider<CheckoutRepository>((ref) {
  return CheckoutRepository(
    database: AppDatabase(),
  );
});

final checkoutViewModelProvider =
    NotifierProvider<CheckoutViewModel, CheckoutState>(
  CheckoutViewModel.new,
);

class CheckoutState {
```

```
  const CheckoutState({
    this.isSaving = false,
    this.isCompleted = false,
    this.errorMessage,
    this.lastSale,
  });

  final bool isSaving;
  final bool isCompleted;
  final String? errorMessage;
  final Sale? lastSale;

  CheckoutState copyWith({
    bool? isSaving,
    bool? isCompleted,
    String? errorMessage,
    Sale? lastSale,
    bool clearError = false,
  }) {
    return CheckoutState(
      isSaving: isSaving ?? this.isSaving,
      isCompleted: isCompleted ?? this.isCompleted,
      errorMessage: clearError ? null : errorMessage ?? this.errorMessage,
      lastSale: lastSale ?? this.lastSale,
    );
  }
}

class CheckoutViewModel extends Notifier<CheckoutState> {
  static const Uuid _uuid = Uuid();

  @override
  CheckoutState build() {
    return const CheckoutState();
  }

  Future<void> completeCashSale(WidgetRef ref) async {
    final cart = ref.read(cartProvider);

    if (cart.isEmpty) {
      state = state.copyWith(
        errorMessage: 'The cart is empty.',
      );
      return;
    }

    await _completeSale(
      ref: ref,
      cart: cart,
      paymentMethod: PaymentMethod.cash,
    );
  }

  Future<void> completeCardSale(WidgetRef ref) async {
    final cart = ref.read(cartProvider);
```

```
    if (cart.isEmpty) {
      state = state.copyWith(
        errorMessage: 'The cart is empty.',
      );
      return;
    }

    await _completeSale(
      ref: ref,
      cart: cart,
      paymentMethod: PaymentMethod.card,
    );
  }

  Future<void> _completeSale({
    required WidgetRef ref,
    required List<CartItem> cart,
    required PaymentMethod paymentMethod,
  }) async {
    state = state.copyWith(
      isSaving: true,
      clearError: true,
    );

    try {
      final saleId = _uuid.v4();

      final saleItems = cart.map((cartItem) {
        return SaleItem(
          id: _uuid.v4(),
          saleId: saleId,
          productId: cartItem.product.id,
          quantity: cartItem.quantity,
          unitPriceCents: cartItem.product.priceCents,
          totalCents: cartItem.totalCents,
        );
      }).toList();

      final subtotalCents = cart.fold<int>(
        0,
        (sum, item) => sum + item.totalCents,
      );

      final taxCents = cart.fold<int>(
        0,
        (sum, item) {
          return sum +
              item.totalCents * item.product.taxRate ~/ 100;
        },
      );

      final sale = Sale(
        id: saleId,
        invoiceNumber: 'INV-${DateTime.now().millisecondsSinceEpoch}',
        items: saleItems,
        subtotalCents: subtotalCents,
```

```
        discountCents: 0,
        taxCents: taxCents,
        totalCents: subtotalCents + taxCents,
        paymentMethod: paymentMethod,
        createdAt: DateTime.now(),
        syncStatus: 'pending',
      );

      final repository = ref.read(checkoutRepositoryProvider);

      await repository.saveSale(sale);

      ref.read(cartProvider.notifier).clear();

      state = state.copyWith(
        isSaving: false,
        isCompleted: true,
        lastSale: sale,
      );
    } catch (error) {
      state = state.copyWith(
        isSaving: false,
        errorMessage: 'Unable to save the sale: $error',
      );
    }
  }

  void reset() {
    state = const CheckoutState();
  }
}
```

Important checkout correction

The current `CheckoutViewModel` accepts `WidgetRef`. This is convenient for this first prototype, but a cleaner production design passes repositories into the ViewModel constructor through dependency injection. Flutter recommends dependency injection so components are not globally coupled. [97]

For now, this version is easier to integrate with the current Riverpod setup.

8. Update the checkout screen

File: `lib/features/checkout/presentation/screens/checkout_screen.dart`

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../view_models/checkout_view_model.dart';
```

```
class CheckoutScreen extends ConsumerWidget {
  const CheckoutScreen({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);
    final cartViewModel = ref.watch(cartProvider.notifier);
    final checkoutState = ref.watch(checkoutViewModelProvider);
    final checkoutViewModel =
        ref.read(checkoutViewModelProvider.notifier);

    ref.listen<CheckoutState>(
      checkoutViewModelProvider,
      (previous, next) {
        if (next.isCompleted && !next.isSaving) {
          ScaffoldMmessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Sale completed successfully'),
            ),
          );
        }

        if (next.errorMessage != null &&
            next.errorMessage != previous?.errorMessage) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(
              content: Text(next.errorMessage!),
            ),
          );
        }
      },
    );

    return Scaffold(
      appBar: AppBar(
        leading: const Icon(Icons.payment),
        title: const Text('Checkout'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Card(
              child: Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                  children: [
                    const Icon(
                      Icons.receipt_long,
                      size: 64,
                    ),
```

```
                    const SizedBox(height: 12),
                    const Text('Amount to pay'),
                    const SizedBox(height: 8),
                    Text(
                      formatMoney(cartViewModel.totalCents),
                      style: Theme.of(context)
                          .textTheme
                          .headlineSmall
                          ?.copyWith(
                            fontWeight: FontWeight.bold,
                          ),
                    ),
                  ],
                ),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: cart.isEmpty || checkoutState.isSaving
                  ? null
                  : () {
                      checkoutViewModel.completeCashSale(ref);
                    },
              icon: checkoutState.isSaving
                  ? const SizedBox(
                      width: 18,
                      height: 18,
                      child: CircularProgressIndicator(
                        strokeWidth: 2,
                      ),
                    )
                  : const Icon(Icons.money),
              label: const Text('Pay with cash'),
            ),
            const SizedBox(height: 12),
            OutlinedButton.icon(
              onPressed: cart.isEmpty || checkoutState.isSaving
                  ? null
                  : () {
                      checkoutViewModel.completeCardSale(ref);
                    },
              icon: const Icon(Icons.credit_card),
              label: const Text('Pay by card'),
            ),
          ],
        ),
      ),
    );
  }
}
```

9. Create the receipt printer service

For now, create a platform-independent preview service. It does not yet communicate with a physical thermal printer.

File: `lib/core/printing/receipt_printer_service.dart`

```
import '../../features/checkout/domain/sale.dart';

abstract class ReceiptPrinterService {
  Future<void> printReceipt(Sale sale);
}

class ReceiptPrinterPreviewService
    implements ReceiptPrinterService {
  @override
  Future<void> printReceipt(Sale sale) async {
    final receipt = StringBuffer();

    receipt.writeln('POS BILLING');
    receipt.writeln('------------------------------');
    receipt.writeln('Invoice: ${sale.invoiceNumber}');
    receipt.writeln('Date: ${sale.createdAt}');
    receipt.writeln('------------------------------');

    for (final item in sale.items) {
      receipt.writeln(
        '${item.productId} x${item.quantity} '
        '${formatMoney(item.totalCents)}',
      );
    }

    receipt.writeln('------------------------------');
    receipt.writeln(
      'Subtotal: ${formatMoney(sale.subtotalCents)}',
    );
    receipt.writeln(
      'Tax: ${formatMoney(sale.taxCents)}',
    );
    receipt.writeln(
      'Total: ${formatMoney(sale.totalCents)}',
    );
    receipt.writeln(
      'Payment: ${sale.paymentMethod.name}',
    );

    // Temporary implementation.
    // Replace this with a Bluetooth, USB, or LAN printer call.
    assert(() {
      // ignore: avoid_print
      print(receipt.toString());
      return true;
    }());
  }
```

```
  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }
}
```

A printer is a platform or external service, so keeping it behind an interface allows you to replace the preview implementation later with Bluetooth, USB, or network printing without changing checkout logic. Flutter's architecture guidance describes services as wrappers around external APIs and platform plugins.[96]

10. Create the barcode scanner service

This first version supports manual barcode input and leaves the hardware implementation for later.

File: `lib/core/barcode/barcode_scanner_service.dart`

```
abstract class BarcodeScannerService {
  Future<String?> scan();
}

class ManualBarcodeScannerService
    implements BarcodeScannerService {
  @override
  Future<String?> scan() async {
    return null;
  }
}
```

Later, replace `ManualBarcodeScannerService` with a real camera or hardware scanner implementation.

11. Connect the scanner button

Update the search widget to use the service.

File: `lib/features/products/presentation/widgets/product_search_field.dart`

Add these imports:

```
import '../../../../core/barcode/barcode_scanner_service.dart';
```

Inside `ProductSearchField`, add this method:

```
Future<void> _scanBarcode(
  BuildContext context,
  WidgetRef ref,
) async {
  final scanner = ManualBarcodeScannerService();
  final barcode = await scanner.scan();
```

```
  if (barcode == null || barcode.isEmpty) {
    return;
  }

  ref
      .read(productViewModelProvider.notifier)
      .search(barcode);
}
```

Then change the `suffixIcon` to:

```
suffixIcon: IconButton(
  tooltip: 'Scan barcode',
  icon: const Icon(Icons.qr_code_scanner),
  onPressed: () {
    _scanBarcode(context, ref);
  },
),
```

At this point, the button does not open a physical scanner because `ManualBarcodeScannerService.scan()` returns `null`. The application structure is ready for a real scanner package later.

12. Required imports and package setup

Confirm that `pubspec.yaml` contains:

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^2.0.0
  go_router: ^14.0.0
  path: ^1.9.0
  sqflite: ^2.3.0
  uuid: ^4.0.0
```

Then run:

```
flutter pub get
dart format lib
flutter analyze
flutter run -d chrome
```

13. Possible analyzer issues

`Icons` not defined in `router.dart`

Add:

```
import 'package:flutter/material.dart';
```

`WidgetRef` not defined

Add:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

`Product` constructor is not `const`

Confirm that this file contains a `const` constructor:

File: `lib/features/products/data/models/product.dart`

```
const Product({
  required this.id,
  required this.barcode,
  required this.name,
  required this.priceCents,
  required this.taxRate,
  required this.stockQuantity,
});
```

`CartItem` import error

Confirm the import in `cart_item_tile.dart` is:

```
import '../../domain/cart_item.dart';
```

`Sale` import error

Confirm the imports in `checkout_view_model.dart` are:

```
import '../../domain/payment.dart';
import '../../domain/sale.dart';
import '../../domain/sale_item.dart';
```

The next logical step is to fix the product and cart layout so both product results and cart items are visible together, then connect the receipt preview after a successful checkout.

⁂

Not able to add the Prodcuts in the Productsearch secrren

The main problem is that the current `ProductSearchField` displays products, but your `PosScreen` places it inside a `Column` whose remaining area is used for the cart. Also, clicking a product may be difficult to see because the product list is only 240 pixels high.

Use a dedicated product-search panel and a cart panel side by side. The `onChanged` callback is appropriate for updating the search as the user types. [105] [106]

Replace the POS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../presentation_placeholder.dart';
import '../widgets/cart_item_tile.dart';
import '../widgets/cart_summary.dart';
import '../view_models/cart_view_model.dart';

class PosScreen extends ConsumerWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Point of Sale'),
        actions: [
          IconButton(
            tooltip: 'Sales history',
            icon: const Icon(Icons.receipt_long),
            onPressed: () {
              context.go('/sales');
            },
          ),
          IconButton(
            tooltip: 'Settings',
            icon: const Icon(Icons.settings),
            onPressed: () {
              context.go('/settings');
            },
          ),
```

```
        ],
      ),
      body: Row(
        children: [
          SideNavigationRail(
            selectedIndex: 1,
            onDestinationSelected: (index) {
              _navigateToDestination(context, index);
            },
          ),
          const VerticalDivider(
            width: 1,
            thickness: 1,
          ),
          Expanded(
            child: Row(
              children: [
                Expanded(
                  flex: 5,
                  child: _ProductsPanel(),
                ),
                const VerticalDivider(
                  width: 1,
                  thickness: 1,
                ),
                Expanded(
                  flex: 5,
                  child: _CartPanel(cart: cart),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }

  void _navigateToDestination(
    BuildContext context,
    int index,
  ) {
    switch (index) {
      case 0:
        context.go('/dashboard');
        break;
      case 1:
        context.go('/pos');
        break;
      case 2:
        context.go('/products');
        break;
      case 3:
        context.go('/inventory');
        break;
      case 4:
        context.go('/sales');
```

```
        break;
      case 5:
        context.go('/customers');
        break;
      case 6:
        context.go('/reports');
        break;
      case 7:
        context.go('/settings');
        break;
    }
  }
}

class _ProductsPanel extends StatelessWidget {
  const _ProductsPanel();

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surface,
      child: const Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Products',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: SingleChildScrollView(
              child: ProductSearchField(),
            ),
          ),
        ],
      ),
    );
  }
}

class _CartPanel extends StatelessWidget {
  const _CartPanel({
    required this.cart,
  });

  final List<CartItem> cart;

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surfaceContainerLowest,
```

```
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Current Cart',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: cart.isEmpty
                ? const Center(
                    child: Column(
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        Icon(
                          Icons.shopping_cart_outlined,
                          size: 72,
                          color: Colors.grey,
                        ),
                        SizedBox(height: 12),
                        Text('The cart is empty'),
                      ],
                    ),
                  )
                : ListView.builder(
                    padding: const EdgeInsets.all(8),
                    itemCount: cart.length,
                    itemBuilder: (context, index) {
                      return CartItemTile(
                        item: cart[index],
                      );
                    },
                  ),
          ),
          const CartSummary(),
        ],
      ),
    );
  }
}
```

Remove this incorrect import if it appears:

```
import '../presentation_placeholder.dart';
```

The correct import section is:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

```
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../widgets/cart_item_tile.dart';
import '../widgets/cart_summary.dart';
import '../view_models/cart_view_model.dart';
```

Replace the product search widget

File: `lib/features/products/presentation/widgets/product_search_field.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({
    super.key,
  });

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productState = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Type Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: productState.searchText.isEmpty
                    ? null
                    : () {
                        ref
                            .read(productViewModelProvider.notifier)
                            .clearSearch();
                      },
              ),
            ),
            onChanged: (value) {
              ref
                  .read(productViewModelProvider.notifier)
                  .search(value);
```

```
            },
          ),
        ),
        if (productState.products.isEmpty)
          const Padding(
            padding: EdgeInsets.all(24),
            child: Center(
              child: Text('No products found'),
            ),
          )
        else
          ...productState.products.map(
            (product) => ProductListItem(product: product),
          ),
      ],
    );
  }
}
```

Verify the import path

From:

```
lib/features/products/presentation/widgets/product_search_field.dart
```

to:

```
lib/features/cart/presentation/view_models/cart_view_model.dart
```

the correct import is:

```
import '../../../cart/presentation/view_models/cart_view_model.dart';
```

Check the product item

File: `lib/features/products/presentation/widgets/product_list_item.dart`

Make sure the button calls the cart provider:

```
onPressed: product.stockQuantity <= 0
    ? null
    : () {
        ref
            .read(cartProvider.notifier)
            .addProduct(product);

        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(
```

```
              '${product.name} added to cart',
            ),
            duration: const Duration(seconds: 1),
          ),
        );
      },
```

The complete import section should be:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/models/product.dart';
```

Check the cart item import

File: `lib/features/cart/presentation/widgets/cart_item_tile.dart`

The imports should be:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';
```

Verify Riverpod setup

File: `lib/main.dart`

Make sure the application is wrapped with `ProviderScope`:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';

void main() {
  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Riverpod providers expose state to widgets, and widgets must use `ref.watch` to rebuild when that state changes. [107] [108]

Run the application

```
dart format lib
flutter pub get
flutter analyze
flutter run -d chrome
```

Test it this way:

1. Open the POS screen.
2. In the left product panel, type `Coffee`.
3. Click the shopping-cart button.
4. Confirm that `Coffee` appears in the right cart panel.
5. Click the plus button to increase the quantity.
6. Click `Checkout`.

If products still do not appear, temporarily add this line inside `ProductSearchField.build()`:

```
debugPrint('Products found: ${productState.products.length}');
```

If it prints `0`, the issue is in `ProductViewModel`. If it prints `4` but nothing is visible, the issue is in the screen layout or widget import.

⁂

flutter analyze failed "The name 'CartItem' isn't a type, so it can't be used as a type argument."

`CartItem` is not visible in the file where you use `List<CartItem>`. The most likely problem is a missing or incorrect import. Dart reports this when a class is not defined or is not visible in the current file. [117] [118]

Fix `pos_screen.dart`

Open:

```
lib/features/cart/presentation/screens/pos_screen.dart
```

Add this import:

```
import '../../domain/cart_item.dart';
```

The complete import section should be:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';
import '../widgets/cart_item_tile.dart';
import '../widgets/cart_summary.dart';
```

The error is probably on this line:

```
final List<CartItem> cart;
```

That line requires the `CartItem` class import.

Verify the `CartItem` class

Open:

```
lib/features/cart/domain/cart_item.dart
```

Make sure it contains this code:

```
import '../../products/data/models/product.dart';

class CartItem {
  final Product product;
  final int quantity;

  const CartItem({
    required this.product,
    required this.quantity,
  });

  int get totalCents {
    return product.priceCents * quantity;
  }

  CartItem copyWith({
    Product? product,
    int? quantity,
  }) {
    return CartItem(
      product: product ?? this.product,
      quantity: quantity ?? this.quantity,
    );
  }
}
```

Check the filename carefully:

```
cart_item.dart
```

Not:

```
cart_items.dart
CartItem.dart
cartitem.dart
```

Correct `_CartPanel` code

In:

```
lib/features/cart/presentation/screens/pos_screen.dart
```

the `_CartPanel` class should be:

```
class _CartPanel extends StatelessWidget {
  const _CartPanel({
    required this.cart,
  });

  final List<CartItem> cart;

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surfaceContainerLowest,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Current Cart',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: cart.isEmpty
                ? const Center(
                    child: Column(
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        Icon(
                          Icons.shopping_cart_outlined,
                          size: 72,
```

```
                        color: Colors.grey,
                      ),
                      SizedBox(height: 12),
                      Text('The cart is empty'),
                    ],
                  ),
                )
              : ListView.builder(
                  padding: const EdgeInsets.all(8),
                  itemCount: cart.length,
                  itemBuilder: (context, index) {
                    return CartItemTile(
                      item: cart[index],
                    );
                  },
                ),
          ),
          const CartSummary(),
        ],
      ),
    );
  }
}
```

Check `cart_item_tile.dart`

Open:

```
lib/features/cart/presentation/widgets/cart_item_tile.dart
```

Make sure it imports `CartItem` like this:

```
import '../../domain/cart_item.dart';
```

The complete import section should be:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';
```

Check the product model import

`CartItem` itself uses `Product`, so this import must also exist:

File: `lib/features/cart/domain/cart_item.dart`

```
import '../../products/data/models/product.dart';
```

The expected path is:

```
lib/features/cart/domain/cart_item.dart
```

to:

```
lib/features/products/data/models/product.dart
```

Therefore, this is correct:

```
import '../../products/data/models/product.dart';
```

Run the analyzer again

From the project root:

```
dart format lib
flutter pub get
flutter analyze
```

If the error remains:

```
flutter clean
flutter pub get
flutter analyze
```

Also verify that you do not have another class named `CartItem` accidentally declared inside a different file. Dart requires the class to be defined or imported into the scope where it is used. [118] [119]

⁂

getting run time exception

Cannot hit test a render box that has never been laid out.

This error is caused by a layout constraint problem, usually a `ListView` or `Column` receiving unlimited height. Flutter reports this when a render object has no calculated size, so it cannot process mouse clicks or hit testing. [127] [128] [129]

In your current code, the likely problem is this combination:

```
Products panel
└── SingleChildScrollView
    └── ProductSearchField
        └── ListView
```

A scrollable widget inside another scrollable widget can receive unbounded height. Remove the nested `SingleChildScrollView`.

Fix the POS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Replace the `_ProductsPanel` class with:

```
class _ProductsPanel extends StatelessWidget {
  const _ProductsPanel();

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surface,
      child: const Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Products',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: ProductSearchField(),
          ),
        ],
      ),
    );
  }
}
```

The important change is:

```
Expanded(
  child: ProductSearchField(),
)
```

and removing:

```
SingleChildScrollView
```

Fix the product search widget

File: `lib/features/products/presentation/widgets/product_search_field.dart`

Replace the complete file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({
    super.key,
  });

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productState = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Type Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: productState.searchText.isEmpty
                    ? null
                    : () {
                        ref
                            .read(productViewModelProvider.notifier)
                            .clearSearch();
                      },
              ),
            ),
            onChanged: (value) {
              ref
                  .read(productViewModelProvider.notifier)
                  .search(value);
            },
          ),
        ),
```

```
        Expanded(
          child: productState.products.isEmpty
              ? const Center(
                  child: Text('No products found'),
                )
              : ListView.builder(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 8,
                  ),
                  itemCount: productState.products.length,
                  itemBuilder: (context, index) {
                    return ProductListItem(
                      product: productState.products[index],
                    );
                  },
                ),
        ),
      ],
    );
  }
}
```

The `Expanded` around `ListView.builder` gives the product list a finite height. Flutter specifically recommends giving a list an available size, commonly by wrapping it in `Expanded` when it is inside a `Column`.[127]

Important: avoid `const` here

In `pos_screen.dart`, this is acceptable:

```
Expanded(
  child: ProductSearchField(),
)
```

But this may produce a different issue if the widget later receives dynamic parameters:

```
const ProductSearchField()
```

Use the non-const version for now:

```
Expanded(
  child: ProductSearchField(),
)
```

Check the cart panel

The cart panel should keep its `ListView` inside `Expanded`.

File: `lib/features/cart/presentation/screens/pos_screen.dart`

The cart section should contain:

```
Expanded(
  child: cart.isEmpty
      ? const Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Icon(
                Icons.shopping_cart_outlined,
                size: 72,
                color: Colors.grey,
              ),
              SizedBox(height: 12),
              Text('The cart is empty'),
            ],
          ),
        )
      : ListView.builder(
          padding: const EdgeInsets.all(8),
          itemCount: cart.length,
          itemBuilder: (context, index) {
            return CartItemTile(
              item: cart[index],
            );
          },
        ),
),
const CartSummary(),
```

Do not put this `Expanded` inside a `SingleChildScrollView`.

Check the overall layout

Your structure should be:

```
Scaffold
└── body: Row
    ├── SideNavigationRail
    └── Expanded
        └── Row
            ├── Expanded
            │   └── ProductsPanel
            │       └── Column
            │           ├── Text
            │           └── Expanded
            │               └── ProductSearchField
```

```
│                    └── Column
│                        ├── TextField
│                        └── Expanded
│                            └── ListView
└── Expanded
    └── CartPanel
        └── Column
            ├── Text
            ├── Expanded
            │   └── ListView
            └── CartSummary
```

Every `ListView` has a finite height because it is inside an `Expanded` widget.

Run again

```
dart format lib
flutter analyze
flutter clean
flutter pub get
flutter run -d chrome
```

If the error continues, look at the **first layout error** in the console, not only the final "Cannot hit test" message. The hit-test exception is often a secondary consequence of an earlier layout failure. [127]

⁂

have to implement adding products to cart

The product button must call `cartProvider.notifier.addProduct(product)`, and the POS screen must watch `cartProvider` so the cart rebuilds after the product is added. Riverpod's `Notifier` updates listeners when its `state` is replaced, which is the mechanism used below. [137] [138]

1. Verify the cart model

File: `lib/features/cart/domain/cart_item.dart`

```
import '../../products/data/models/product.dart';

class CartItem {
  const CartItem({
    required this.product,
    required this.quantity,
  });

  final Product product;
  final int quantity;

  int get totalCents {
```

```
    return product.priceCents * quantity;
  }

  CartItem copyWith({
    Product? product,
    int? quantity,
  }) {
    return CartItem(
      product: product ?? this.product,
      quantity: quantity ?? this.quantity,
    );
  }
}
```

2. Replace the cart ViewModel

File: `lib/features/cart/presentation/view_models/cart_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/cart_item.dart';
import '../../../products/data/models/product.dart';

final cartProvider =
    NotifierProvider<CartViewModel, List<CartItem>>(
  CartViewModel.new,
);

class CartViewModel extends Notifier<List<CartItem>> {
  @override
  List<CartItem> build() {
    return const [];
  }

  void addProduct(Product product) {
    final existingIndex = state.indexWhere(
      (item) => item.product.id == product.id,
    );

    if (existingIndex == -1) {
      state = [
        ...state,
        CartItem(
          product: product,
          quantity: 1,
        ),
      ];
      return;
    }

    final updatedItems = [...state];
    final existingItem = updatedItems[existingIndex];

    if (existingItem.quantity >= product.stockQuantity) {
```

```
      return;
    }

    updatedItems[existingIndex] = existingItem.copyWith(
      quantity: existingItem.quantity + 1,
    );

    state = updatedItems;
  }

  void increaseQuantity(String productId) {
    final index = state.indexWhere(
      (item) => item.product.id == productId,
    );

    if (index == -1) {
      return;
    }

    final item = state[index];

    if (item.quantity >= item.product.stockQuantity) {
      return;
    }

    final updatedItems = [...state];

    updatedItems[index] = item.copyWith(
      quantity: item.quantity + 1,
    );

    state = updatedItems;
  }

  void decreaseQuantity(String productId) {
    final index = state.indexWhere(
      (item) => item.product.id == productId,
    );

    if (index == -1) {
      return;
    }

    final item = state[index];

    if (item.quantity <= 1) {
      removeProduct(productId);
      return;
    }

    final updatedItems = [...state];

    updatedItems[index] = item.copyWith(
      quantity: item.quantity - 1,
    );
```

```
    state = updatedItems;
  }

  void removeProduct(String productId) {
    state = state
        .where((item) => item.product.id != productId)
        .toList();
  }

  void clear() {
    state = const [];
  }

  int get itemCount {
    return state.fold(
      0,
      (total, item) => total + item.quantity,
    );
  }

  int get subtotalCents {
    return state.fold(
      0,
      (total, item) => total + item.totalCents,
    );
  }

  int get taxCents {
    return state.fold(
      0,
      (total, item) {
        return total +
            item.totalCents * item.product.taxRate ~/ 100;
      },
    );
  }

  int get totalCents {
    return subtotalCents + taxCents;
  }
}
```

The method `addProduct()` does two things:

- Adds a new `CartItem` with quantity `1`.
- Increases the quantity if the product is already in the cart.

3. Replace the product list item

File: `lib/features/products/presentation/widgets/product_list_item.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/models/product.dart';

class ProductListItem extends ConsumerWidget {
  const ProductListItem({
    super.key,
    required this.product,
  });

  final Product product;

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final colorScheme = Theme.of(context).colorScheme;
    final isOutOfStock = product.stockQuantity <= 0;

    return Card(
      margin: const EdgeInsets.symmetric(
        horizontal: 8,
        vertical: 4,
      ),
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: colorScheme.primaryContainer,
          child: Icon(
            Icons.inventory_2,
            color: colorScheme.onPrimaryContainer,
          ),
        ),
        title: Text(
          product.name,
          style: const TextStyle(
            fontWeight: FontWeight.bold,
          ),
        ),
        subtitle: Text(
          'Barcode: ${product.barcode}\n'
          'Available stock: ${product.stockQuantity}',
        ),
        isThreeLine: true,
        trailing: SizedBox(
          width: 125,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            crossAxisAlignment: CrossAxisAlignment.end,
            children: [
              Text(
```

```
                formatMoney(product.priceCents),
                style: TextStyle(
                  color: colorScheme.primary,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 4),
              SizedBox(
                height: 32,
                child: FilledButton.icon(
                  onPressed: isOutOfStock
                      ? null
                      : () {
                          ref
                              .read(cartProvider.notifier)
                              .addProduct(product);

                          ScaffoldMessenger.of(context).showSnackBar(
                            SnackBar(
                              content: Text(
                                '${product.name} added to cart',
                              ),
                              duration: const Duration(
                                seconds: 1,
                              ),
                            ),
                          );
                        },
                  icon: const Icon(
                    Icons.add_shopping_cart,
                    size: 16,
                  ),
                  label: const Text('Add'),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

The important code is:

```
ref.read(cartProvider.notifier).addProduct(product);
```

Do not use `ref.watch` to invoke the method. Use `ref.read(...notifier)` for actions and `ref.watch(...)` for displaying changing state.

4. Replace the product search widget

File: `lib/features/products/presentation/widgets/product_search_field.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({
    super.key,
  });

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productState = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Try Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: productState.searchText.isEmpty
                    ? null
                    : () {
                        ref
                            .read(productViewModelProvider.notifier)
                            .clearSearch();
                      },
              ),
            ),
            onChanged: (value) {
              ref
                  .read(productViewModelProvider.notifier)
                  .search(value);
            },
          ),
        ),
        Expanded(
          child: productState.products.isEmpty
              ? const Center(
                  child: Text('No products found'),
                )
              : ListView.builder(
                  padding: const EdgeInsets.only(
```

```
                    left: 8,
                    right: 8,
                    bottom: 16,
                  ),
                  itemCount: productState.products.length,
                  itemBuilder: (context, index) {
                    return ProductListItem(
                      product: productState.products[index],
                    );
                  },
                ),
          ),
        ],
      );
  }
}
```

5. Replace the cart item tile

File: `lib/features/cart/presentation/widgets/cart_item_tile.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';

class CartItemTile extends ConsumerWidget {
  const CartItemTile({
    super.key,
    required this.item,
  });

  final CartItem item;

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cartViewModel = ref.read(cartProvider.notifier);

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(8),
        child: Row(
          children: [
            const Icon(Icons.shopping_bag_outlined),
            const SizedBox(width: 8),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
```

```
              Text(
                item.product.name,
                style: const TextStyle(
                  fontWeight: FontWeight.bold,
                ),
              ),
              Text(
                '${formatMoney(item.product.priceCents)} each',
              ),
            ],
          ),
        ),
        IconButton(
          tooltip: 'Decrease quantity',
          icon: const Icon(Icons.remove_circle_outline),
          onPressed: () {
            cartViewModel.decreaseQuantity(
              item.product.id,
            );
          },
        ),
        Text(
          '${item.quantity}',
          style: const TextStyle(
            fontWeight: FontWeight.bold,
          ),
        ),
        IconButton(
          tooltip: 'Increase quantity',
          icon: const Icon(Icons.add_circle_outline),
          onPressed: item.quantity >= item.product.stockQuantity
              ? null
              : () {
                  cartViewModel.increaseQuantity(
                    item.product.id,
                  );
                },
        ),
        const SizedBox(width: 8),
        Text(
          formatMoney(item.totalCents),
          style: const TextStyle(
            fontWeight: FontWeight.bold,
          ),
        ),
        IconButton(
          tooltip: 'Remove item',
          icon: const Icon(Icons.delete_outline),
          onPressed: () {
            cartViewModel.removeProduct(
              item.product.id,
            );
          },
        ),
      ],
    ),
```

```
      ),
    );
  }
}
```

6. Replace the cart summary

File: `lib/features/cart/presentation/widgets/cart_summary.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../view_models/cart_view_model.dart';

class CartSummary extends ConsumerWidget {
  const CartSummary({
    super.key,
  });

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);
    final cartViewModel = ref.read(cartProvider.notifier);

    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Row(
              children: [
                Icon(
                  Icons.shopping_cart,
                  color: Theme.of(context).colorScheme.primary,
                ),
                const SizedBox(width: 8),
                Text(
                  'Cart: ${cartViewModel.itemCount} item(s)',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
              ],
            ),
            const SizedBox(height: 12),
            _SummaryRow(
              label: 'Subtotal',
              value: formatMoney(cartViewModel.subtotalCents),
            ),
            _SummaryRow(
```

```
              label: 'Tax',
              value: formatMoney(cartViewModel.taxCents),
            ),
            const Divider(),
            _SummaryRow(
              label: 'Total',
              value: formatMoney(cartViewModel.totalCents),
              isTotal: true,
            ),
            const SizedBox(height: 12),
            FilledButton.icon(
              onPressed: cart.isEmpty
                  ? null
                  : () {
                      context.push('/checkout');
                    },
              icon: const Icon(Icons.payment),
              label: const Text('Checkout'),
            ),
          ],
        ),
      ),
    );
  }
}

class _SummaryRow extends StatelessWidget {
  const _SummaryRow({
    required this.label,
    required this.value,
    this.isTotal = false,
  });

  final String label;
  final String value;
  final bool isTotal;

  @override
  Widget build(BuildContext context) {
    final style = isTotal
        ? Theme.of(context).textTheme.titleLarge?.copyWith(
              fontWeight: FontWeight.bold,
            )
        : Theme.of(context).textTheme.bodyLarge;

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 3),
      child: Row(
        children: [
          Expanded(
            child: Text(
              label,
              style: style,
            ),
          ),
          Text(
```

```
            value,
            style: style,
          ),
        ],
      ),
    );
  }
}
```

7. Verify the POS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Make sure the imports include:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';
import '../widgets/cart_item_tile.dart';
import '../widgets/cart_summary.dart';
```

The product and cart panels should contain:

```
Expanded(
  flex: 5,
  child: _ProductsPanel(),
),
const VerticalDivider(
  width: 1,
),
Expanded(
  flex: 5,
  child: _CartPanel(cart: cart),
),
```

And `_CartPanel` should contain:

```
Expanded(
  child: cart.isEmpty
      ? const Center(
          child: Text('The cart is empty'),
        )
      : ListView.builder(
          itemCount: cart.length,
          itemBuilder: (context, index) {
            return CartItemTile(
```

```
              item: cart[index],
            );
          },
        ),
),
const CartSummary(),
```

8. Verify the provider scope

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';

void main() {
  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Without `ProviderScope`, Riverpod providers cannot be read by the widgets.

9. Run and test

```
dart format lib
flutter analyze
flutter run -d chrome
```

Test the flow:

1. Open `/pos`.
2. Product cards should appear in the left panel.
3. Click **Add** on `Coffee`.
4. `Coffee` should appear in the right cart panel.
5. Click **Add** again.
6. The quantity should change from `1` to `2`.
7. Click plus, minus, or delete in the cart.
8. The subtotal, tax, and total should update.

If the product cards are not visible, the next likely issue is the `ProductViewModel` initialization or the `ProductSearchField` layout. If the product cards are visible but clicking **Add** does nothing, check the browser console and confirm that the button contains exactly:

```
ref.read(cartProvider.notifier).addProduct(product);
```

⁂

let us add the SQL lite data base to the app

We will now connect the product screen and checkout flow to SQLite. Flutter's SQLite recipe uses `sqflite` with `path`, while Flutter's architecture guidance recommends that repositories control database access and act as the single source of truth.[147] [148]

1. Add SQLite packages

From the project root, run:

```
flutter pub add sqflite
flutter pub add path
flutter pub add uuid
```

Then verify:

```
flutter pub get
```

SQLite support is directly available through Flutter's `sqflite` plugin, and `path` is used to create the database file location.[147]

2. Create the database file

File: `lib/core/database/app_database.dart`

Replace the existing file with:

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  AppDatabase._internal();

  static final AppDatabase instance = AppDatabase._internal();

  static Database? _database;

  Future<Database> get database async {
    if (_database != null) {
      return _database!;
    }

    _database = await _openDatabase();
    return _database!;
```

```
  }

  Future<Database> _openDatabase() async {
    final databaseDirectory = await getDatabasesPath();
    final databasePath = join(
      databaseDirectory,
      'pos_billing.db',
    );

    return openDatabase(
      databasePath,
      version: 1,
      onConfigure: (db) async {
        await db.execute('PRAGMA foreign_keys = ON');
      },
      onCreate: (db, version) async {
        await _createTables(db);
        await _insertDemoProducts(db);
      },
    );
  }

  Future<void> _createTables(Database db) async {
    await db.execute('''
      CREATE TABLE products (
        id TEXT PRIMARY KEY,
        barcode TEXT NOT NULL UNIQUE,
        name TEXT NOT NULL,
        price_cents INTEGER NOT NULL,
        tax_rate INTEGER NOT NULL,
        stock_quantity INTEGER NOT NULL,
        is_active INTEGER NOT NULL DEFAULT 1
      )
    ''');

    await db.execute('''
      CREATE TABLE sales (
        id TEXT PRIMARY KEY,
        invoice_number TEXT NOT NULL UNIQUE,
        subtotal_cents INTEGER NOT NULL,
        discount_cents INTEGER NOT NULL,
        tax_cents INTEGER NOT NULL,
        total_cents INTEGER NOT NULL,
        payment_method TEXT NOT NULL,
        created_at TEXT NOT NULL,
        sync_status TEXT NOT NULL
      )
    ''');

    await db.execute('''
      CREATE TABLE sale_items (
        id TEXT PRIMARY KEY,
        sale_id TEXT NOT NULL,
        product_id TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        unit_price_cents INTEGER NOT NULL,
```

```
        total_cents INTEGER NOT NULL,
        FOREIGN KEY (sale_id)
          REFERENCES sales (id)
          ON DELETE CASCADE,
        FOREIGN KEY (product_id)
          REFERENCES products (id)
      )
    ''');

    await db.execute('''
      CREATE TABLE stock_movements (
        id TEXT PRIMARY KEY,
        product_id TEXT NOT NULL,
        movement_type TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        reference_id TEXT,
        created_at TEXT NOT NULL,
        FOREIGN KEY (product_id)
          REFERENCES products (id)
      )
    ''');
  }

  Future<void> _insertDemoProducts(Database db) async {
    final products = [
      {
        'id': 'p-001',
        'barcode': '890000000001',
        'name': 'Coffee',
        'price_cents': 299,
        'tax_rate': 19,
        'stock_quantity': 50,
        'is_active': 1,
      },
      {
        'id': 'p-002',
        'barcode': '890000000002',
        'name': 'Tea',
        'price_cents': 199,
        'tax_rate': 19,
        'stock_quantity': 40,
        'is_active': 1,
      },
      {
        'id': 'p-003',
        'barcode': '890000000003',
        'name': 'Sandwich',
        'price_cents': 499,
        'tax_rate': 7,
        'stock_quantity': 20,
        'is_active': 1,
      },
      {
        'id': 'p-004',
        'barcode': '890000000004',
        'name': 'Water',
```

```
        'price_cents': 149,
        'tax_rate': 19,
        'stock_quantity': 100,
        'is_active': 1,
      },
    ];

    for (final product in products) {
      await db.insert(
        'products',
        product,
        conflictAlgorithm: ConflictAlgorithm.ignore,
      );
    }
  }
}
```

The database creates product, sales, sale-item, and stock-movement tables. Products are inserted only the first time the database is created.

3. Update the product mapper

File: `lib/features/products/data/models/product_mapper.dart`

Replace the file with:

```
import 'product.dart';

class ProductMapper {
  static Product fromMap(Map<String, Object?> map) {
    return Product(
      id: map['id'] as String,
      barcode: map['barcode'] as String,
      name: map['name'] as String,
      priceCents: map['price_cents'] as int,
      taxRate: map['tax_rate'] as int,
      stockQuantity: map['stock_quantity'] as int,
    );
  }

  static Map<String, Object?> toMap(Product product) {
    return {
      'id': product.id,
      'barcode': product.barcode,
      'name': product.name,
      'price_cents': product.priceCents,
      'tax_rate': product.taxRate,
      'stock_quantity': product.stockQuantity,
      'is_active': 1,
    };
  }
}
```

4. Update the local product service

File: `lib/features/products/data/services/product_local_service.dart`

Replace the file with:

```
import 'package:sqflite/sqflite.dart';

import '../../../../core/database/app_database.dart';
import '../models/product.dart';
import '../models/product_mapper.dart';

class ProductLocalService {
  ProductLocalService({
    required this.database,
  });

  final AppDatabase database;

  Future<List<Product>> getAllProducts() async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'is_active = ?',
      whereArgs: [^15_1],
      orderBy: 'name ASC',
    );

    return rows.map(ProductMapper.fromMap).toList();
  }

  Future<List<Product>> searchProducts(String query) async {
    final db = await database.database;

    final searchText = query.trim();

    if (searchText.isEmpty) {
      return getAllProducts();
    }

    final rows = await db.query(
      'products',
      where: '''
        is_active = ?
        AND (name LIKE ? OR barcode LIKE ?)
      ''',
      whereArgs: [
        1,
        '%$searchText%',
        '%$searchText%',
      ],
      orderBy: 'name ASC',
    );
```

```
    return rows.map(ProductMapper.fromMap).toList();
  }

  Future<Product?> getProductByBarcode(String barcode) async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'barcode = ? AND is_active = ?',
      whereArgs: [barcode, 1],
      limit: 1,
    );

    if (rows.isEmpty) {
      return null;
    }

    return ProductMapper.fromMap(rows.first);
  }

  Future<void> insertProduct(Product product) async {
    final db = await database.database;

    await db.insert(
      'products',
      ProductMapper.toMap(product),
      conflictAlgorithm: ConflictAlgorithm.abort,
    );
  }
}
```

5. Update the product repository

File: `lib/features/products/data/repositories/product_repository.dart`

Replace the file with:

```
import '../models/product.dart';
import '../services/product_local_service.dart';

class ProductRepository {
  ProductRepository({
    required this.localService,
  });

  final ProductLocalService localService;

  Future<List<Product>> getAllProducts() {
    return localService.getAllProducts();
  }

  Future<List<Product>> searchProducts(String query) {
    return localService.searchProducts(query);
  }
```

```
  Future<Product?> getProductByBarcode(String barcode) {
    return localService.getProductByBarcode(barcode);
  }

  Future<void> addProduct(Product product) {
    return localService.insertProduct(product);
  }
}
```

6. Replace the product ViewModel

The previous ViewModel used hard-coded demo products. Replace it with a database-backed version.

File: `lib/features/products/presentation/view_models/product_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/database/app_database.dart';
import '../../data/models/product.dart';
import '../../data/repositories/product_repository.dart';
import '../../data/services/product_local_service.dart';

final productRepositoryProvider = Provider<ProductRepository>((ref) {
  return ProductRepository(
    localService: ProductLocalService(
      database: AppDatabase.instance,
    ),
  );
});

final productViewModelProvider =
    AsyncNotifierProvider<ProductViewModel, ProductState>(
  ProductViewModel.new,
);

class ProductState {
  const ProductState({
    this.products = const [],
    this.searchText = '',
  });

  final List<Product> products;
  final String searchText;

  ProductState copyWith({
    List<Product>? products,
    String? searchText,
  }) {
    return ProductState(
      products: products ?? this.products,
      searchText: searchText ?? this.searchText,
    );
```

```
  }
}

class ProductViewModel extends AsyncNotifier<ProductState> {
  @override
  Future<ProductState> build() async {
    final repository = ref.read(productRepositoryProvider);
    final products = await repository.getAllProducts();

    return ProductState(
      products: products,
    );
  }

  Future<void> search(String value) async {
    final repository = ref.read(productRepositoryProvider);

    state = const AsyncLoading();

    state = await AsyncValue.guard(() async {
      final products = await repository.searchProducts(value);

      return ProductState(
        products: products,
        searchText: value,
      );
    });
  }

  Future<void> clearSearch() {
    return search('');
  }

  Future<void> addProduct(Product product) async {
    final repository = ref.read(productRepositoryProvider);

    state = const AsyncLoading();

    state = await AsyncValue.guard(() async {
      await repository.addProduct(product);

      final products = await repository.getAllProducts();

      return ProductState(
        products: products,
      );
    });
  }
}
```

Because this now uses `AsyncNotifierProvider`, the UI must read the provider as an `AsyncValue`.

7. Update the product search widget

File: `lib/features/products/presentation/widgets/product_search_field.dart`

Replace the complete file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({
    super.key,
  });

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productsAsync = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Try Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: () {
                  ref
                      .read(productViewModelProvider.notifier)
                      .clearSearch();
                },
              ),
            ),
            onChanged: (value) {
              ref
                  .read(productViewModelProvider.notifier)
                  .search(value);
            },
          ),
        ),
        Expanded(
          child: productsAsync.when(
            loading: () => const Center(
              child: CircularProgressIndicator(),
            ),
            error: (error, stackTrace) => Center(
              child: Text(
```

```
                'Unable to load products:\n$error',
                textAlign: TextAlign.center,
              ),
            ),
            data: (productState) {
              if (productState.products.isEmpty) {
                return const Center(
                  child: Text('No products found'),
                );
              }

              return ListView.builder(
                padding: const EdgeInsets.only(
                  left: 8,
                  right: 8,
                  bottom: 16,
                ),
                itemCount: productState.products.length,
                itemBuilder: (context, index) {
                  return ProductListItem(
                    product: productState.products[index],
                  );
                },
              );
            },
          ),
        ),
      ],
    );
  }
}
```

8. Update checkout repository

The repository must check stock before reducing it. The previous implementation could silently update zero rows when stock was insufficient.

File: `lib/features/checkout/data/repositories/checkout_repository.dart`

Replace the file with:

```
import 'package:sqflite/sqflite.dart';

import '../../../../core/database/app_database.dart';
import '../../domain/sale.dart';

class CheckoutRepository {
  CheckoutRepository({
    required this.database,
  });

  final AppDatabase database;

  Future<void> saveSale(Sale sale) async {
```

```
    final db = await database.database;

    await db.transaction((transaction) async {
      for (final item in sale.items) {
        final products = await transaction.query(
          'products',
          columns: ['stock_quantity'],
          where: 'id = ? AND is_active = ?',
          whereArgs: [item.productId, 1],
          limit: 1,
        );

        if (products.isEmpty) {
          throw Exception(
            'Product not found: ${item.productId}',
          );
        }

        final availableStock =
            products.first['stock_quantity'] as int;

        if (availableStock < item.quantity) {
          throw Exception(
            'Insufficient stock for product: '
            '${item.productId}',
          );
        }
      }

      await transaction.insert(
        'sales',
        {
          'id': sale.id,
          'invoice_number': sale.invoiceNumber,
          'subtotal_cents': sale.subtotalCents,
          'discount_cents': sale.discountCents,
          'tax_cents': sale.taxCents,
          'total_cents': sale.totalCents,
          'payment_method': sale.paymentMethod.name,
          'created_at': sale.createdAt.toIso8601String(),
          'sync_status': sale.syncStatus,
        },
        conflictAlgorithm: ConflictAlgorithm.abort,
      );

      for (final item in sale.items) {
        await transaction.insert(
          'sale_items',
          {
            'id': item.id,
            'sale_id': item.saleId,
            'product_id': item.productId,
            'quantity': item.quantity,
            'unit_price_cents': item.unitPriceCents,
            'total_cents': item.totalCents,
          },
```

```
          conflictAlgorithm: ConflictAlgorithm.abort,
        );

        await transaction.rawUpdate(
          '''
          UPDATE products
          SET stock_quantity = stock_quantity - ?
          WHERE id = ?
          ''',
          [
            item.quantity,
            item.productId,
          ],
        );

        await transaction.insert(
          'stock_movements',
          {
            'id': '${sale.id}-${item.productId}',
            'product_id': item.productId,
            'movement_type': 'sale',
            'quantity': -item.quantity,
            'reference_id': sale.id,
            'created_at': sale.createdAt.toIso8601String(),
          },
          conflictAlgorithm: ConflictAlgorithm.abort,
        );
      }
    });
  }
}
```

The transaction ensures that the sale, sale items, stock reduction, and stock movement are committed together. If any operation fails, SQLite rolls back the transaction.

9. Update the checkout ViewModel provider

File: `lib/features/checkout/presentation/view_models/checkout_view_model.dart`

Change the repository provider to use the singleton database:

```
final checkoutRepositoryProvider = Provider<CheckoutRepository>((ref) {
  return CheckoutRepository(
    database: AppDatabase.instance,
  );
});
```

Make sure the imports include:

```
import '../../../../core/database/app_database.dart';
```

Do not create a different `AppDatabase()` object in this provider. Use:

```
AppDatabase.instance
```

10. Important database reset

Your existing database may already have the old schema. During development, reset it once.

Add this temporary method:

File: `lib/core/database/app_database.dart`

Add inside the `AppDatabase` class:

```
Future<void> deleteDatabaseForDevelopment() async {
  final databaseDirectory = await getDatabasesPath();
  final databasePath = join(
    databaseDirectory,
    'pos_billing.db',
  );

  if (_database != null) {
    await _database!.close();
    _database = null;
  }

  await deleteDatabase(databasePath);
}
```

Then temporarily call it before `runApp`:

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/app_database.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await AppDatabase.instance.deleteDatabaseForDevelopment();

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Run the app once, confirm the tables and demo products are created, and then remove this line:

```
await AppDatabase.instance.deleteDatabaseForDevelopment();
```

If you leave it in place, the database will be deleted every time the application starts.

11. Run the application

```
dart format lib
flutter pub get
flutter analyze
flutter run -d chrome
```

Test:

1. Open the POS screen.
2. Confirm that Coffee, Tea, Sandwich, and Water load from SQLite.
3. Search for `Coffee`.
4. Click **Add**.
5. Confirm that the item appears in the cart.
6. Complete a cash sale.
7. Restart the application.
8. Confirm that the product stock has decreased.

Windows web limitation

The `sqflite` package is primarily designed for Flutter mobile and desktop platforms. If you are running the application in Chrome, SQLite may not work directly in the same way as Windows desktop. For a browser build, use a web-compatible storage solution or a backend database/API; for a Windows desktop build, run with `-d windows` and verify the plugin support in your environment.

The architecture remains the same:

```
UI
 ↓
ViewModel
 ↓
Repository
 ↓
Local database service
```

For offline-first applications, the repository should remain the single access point for local and remote data, independent of the device's connectivity state.[148]

⁂

error as databaseFactory not initilized

The error occurs because you are running the app in **Chrome/web** or **Windows desktop**, but `sqflite` has not been given the correct database factory. The factory must be initialized before any call to `openDatabase()`.[157] [158]

First identify your target

Check the device:

```
flutter devices
```

Then choose one:

```
flutter run -d chrome
```

or:

```
flutter run -d windows
```

The setup is different for web and Windows desktop.

Option A: Windows desktop

If you want a real Windows desktop application, add the FFI packages:

```
flutter pub add sqflite_common_ffi
flutter pub add sqlite3_flutter_libs
```

Update `main.dart`

File: `lib/main.dart`

Replace the file with:

```
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';

import 'app/app.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  if (Platform.isWindows || Platform.isLinux || Platform.isMacOS) {
```

```
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

The important order is:

```
sqfliteFfiInit();
databaseFactory = databaseFactoryFfi;
```

This must execute before `AppDatabase.instance.database` is accessed. The FFI setup is required when using `sqflite_common_ffi`.[159] [160]

Run Windows

```
flutter clean
flutter pub get
flutter analyze
flutter run -d windows
```

Option B: Chrome/web

The normal `sqflite` implementation is not sufficient for Flutter web. Add the web implementation:

```
flutter pub add sqflite_common_ffi_web
```

Update `main.dart`

File: `lib/main.dart`

Replace the file with:

```
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

import 'app/app.dart';

Future<void> main() async {
```

```
  WidgetsFlutterBinding.ensureInitialized();

  if (kIsWeb) {
    databaseFactory = databaseFactoryFfiWeb;
  }

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Then prepare the web SQLite files:

```
dart run sqflite_common_ffi_web:setup
```

Now run:

```
flutter clean
flutter pub get
flutter run -d chrome
```

The web package uses browser-compatible storage rather than a normal Windows SQLite file. The web factory must be assigned before calling the global `openDatabase` API. [158] [161] [159]

Use a cross-platform `main.dart`

If you want both Chrome and Windows support, use this version:

File: `lib/main.dart`

```
import 'dart:io' show Platform;

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

import 'app/app.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  if (kIsWeb) {
    databaseFactory = databaseFactoryFfiWeb;
  } else if (Platform.isWindows ||
      Platform.isLinux ||
      Platform.isMacOS) {
```

```
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

However, importing `dart:io` can cause web compilation problems in some setups. The safer production approach is to use separate platform-specific initialization files.

Safer platform-specific setup

Create this file:

File: `lib/core/database/database_initializer.dart`

```
import 'database_initializer_stub.dart'
    if (dart.library.io) 'database_initializer_io.dart'
    if (dart.library.html) 'database_initializer_web.dart';

Future<void> initializeDatabase() {
  return initializePlatformDatabase();
}
```

Create the default fallback:

File: `lib/core/database/database_initializer_stub.dart`

```
Future<void> initializePlatformDatabase() async {}
```

Create the Windows/Linux/macOS implementation:

File: `lib/core/database/database_initializer_io.dart`

```
import 'dart:io';

import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';

Future<void> initializePlatformDatabase() async {
  if (Platform.isWindows ||
      Platform.isLinux ||
      Platform.isMacOS) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
```

```
  }
}
```

Create the web implementation:

File: `lib/core/database/database_initializer_web.dart`

```
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

Future<void> initializePlatformDatabase() async {
  databaseFactory = databaseFactoryFfiWeb;
}
```

Then update the application entry point.

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await initializeDatabase();

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

This approach prevents `dart:io` imports from being compiled into the browser build.

Check `app_database.dart`

File: `lib/core/database/app_database.dart`

Continue using:

```
final db = await openDatabase(
  databasePath,
  version: 1,
  onCreate: (db, version) async {
    await _createTables(db);
    await _insertDemoProducts(db);
```

```
  },
);
```

Do not set `databaseFactory` inside `AppDatabase`. It should be initialized once during application startup, before any repository or ViewModel accesses the database. The `databaseFactory` setter is intended to be called before other `sqflite` operations. [157]

Reset the development database

If the database was created with an old schema, temporarily delete it.

File: `lib/main.dart`

Add this only temporarily:

```
await AppDatabase.instance.deleteDatabaseForDevelopment();
```

Run the app once, then remove that line. Otherwise, all data will be deleted each time the app starts.

Recommended command for your current setup

Because you previously mentioned the **Windows browser**, use:

```
flutter pub add sqflite_common_ffi_web
dart run sqflite_common_ffi_web:setup
flutter clean
flutter pub get
flutter analyze
flutter run -d chrome
```

Then use the cross-platform `main.dart` or the safer platform-specific initializer above.

⁂

getting the below error

Undefined name 'AppDatabase'. Try correcting the name to one that is defined, or defining the name - lib\main.dart:9:9 - undefined_identifier

`AppDatabase` is not visible in `main.dart`. Import the file that defines the class:

```
import 'core/database/app_database.dart';
```

Dart reports `undefined_identifier` when a name is not defined in the current scope or when the file containing it has not been imported. [167]

Fix `main.dart`

File: `lib/main.dart`

Use this version:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/app_database.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await initializeDatabase();

  // Use this only once during development if you need
  // to recreate the database schema.
  // await AppDatabase.instance.deleteDatabaseForDevelopment();

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

The important import is:

```
import 'core/database/app_database.dart';
```

And the file must exist here:

```
lib/core/database/app_database.dart
```

Verify `AppDatabase`

File: `lib/core/database/app_database.dart`

The class declaration must be named exactly `AppDatabase`:

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  AppDatabase._internal();
```

```
  static final AppDatabase instance = AppDatabase._internal();

  static Database? _database;

  Future<Database> get database async {
    if (_database != null) {
      return _database!;
    }

    _database = await _openDatabase();
    return _database!;
  }

  Future<Database> _openDatabase() async {
    final databaseDirectory = await getDatabasesPath();

    final databasePath = join(
      databaseDirectory,
      'pos_billing.db',
    );

    return openDatabase(
      databasePath,
      version: 1,
      onConfigure: (db) async {
        await db.execute('PRAGMA foreign_keys = ON');
      },
      onCreate: (db, version) async {
        await _createTables(db);
        await _insertDemoProducts(db);
      },
    );
  }

  Future<void> _createTables(Database db) async {
    await db.execute('''
      CREATE TABLE products (
        id TEXT PRIMARY KEY,
        barcode TEXT NOT NULL UNIQUE,
        name TEXT NOT NULL,
        price_cents INTEGER NOT NULL,
        tax_rate INTEGER NOT NULL,
        stock_quantity INTEGER NOT NULL,
        is_active INTEGER NOT NULL DEFAULT 1
      )
    ''');

    await db.execute('''
      CREATE TABLE sales (
        id TEXT PRIMARY KEY,
        invoice_number TEXT NOT NULL UNIQUE,
        subtotal_cents INTEGER NOT NULL,
        discount_cents INTEGER NOT NULL,
        tax_cents INTEGER NOT NULL,
        total_cents INTEGER NOT NULL,
        payment_method TEXT NOT NULL,
```

```
        created_at TEXT NOT NULL,
        sync_status TEXT NOT NULL
      )
    ''');

    await db.execute('''
      CREATE TABLE sale_items (
        id TEXT PRIMARY KEY,
        sale_id TEXT NOT NULL,
        product_id TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        unit_price_cents INTEGER NOT NULL,
        total_cents INTEGER NOT NULL
      )
    ''');
  }

  Future<void> _insertDemoProducts(Database db) async {
    final products = [
      {
        'id': 'p-001',
        'barcode': '8900000000001',
        'name': 'Coffee',
        'price_cents': 299,
        'tax_rate': 19,
        'stock_quantity': 50,
        'is_active': 1,
      },
      {
        'id': 'p-002',
        'barcode': '8900000000002',
        'name': 'Tea',
        'price_cents': 199,
        'tax_rate': 19,
        'stock_quantity': 40,
        'is_active': 1,
      },
      {
        'id': 'p-003',
        'barcode': '8900000000003',
        'name': 'Sandwich',
        'price_cents': 499,
        'tax_rate': 7,
        'stock_quantity': 20,
        'is_active': 1,
      },
      {
        'id': 'p-004',
        'barcode': '8900000000004',
        'name': 'Water',
        'price_cents': 149,
        'tax_rate': 19,
        'stock_quantity': 100,
        'is_active': 1,
      },
    ];
```

```
    for (final product in products) {
      await db.insert(
        'products',
        product,
        conflictAlgorithm: ConflictAlgorithm.ignore,
      );
    }
  }
}
```

If you do not need database reset

If `main.dart` contains this line:

```
await AppDatabase.instance.deleteDatabaseForDevelopment();
```

remove it for normal application use. You do not need the `AppDatabase` import if that is the only line using it.

A simpler `main.dart` would then be:

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await initializeDatabase();

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Check the filename and class name

These must match:

```
lib/core/database/app_database.dart
```

```
class AppDatabase
```

These would be incorrect:

```
class DatabaseApp
class AppDataBase
class appDatabase
```

Run the analyzer

From the project root:

```
dart format lib
flutter pub get
flutter analyze
```

If the analyzer still reports the same error, close and reopen the IDE, then run:

```
flutter clean
flutter pub get
flutter analyze
```

The most likely final fix is simply adding this import to `lib/main.dart`:

```
import 'core/database/app_database.dart';
```

⁂

need to complete the Product screen and integrate it with the PoS screen

Yes — the next clean step is to make the **Product screen** load products from SQLite and then connect its Add button to the **PoS cart**. Flutter's architecture guidance recommends this exact flow: View → ViewModel → Repository → Service, with each screen using its own ViewModel and repositories as the source of truth. [177] [178]

What we will do

- Complete the Product screen.
- Load products from SQLite.
- Show products in a searchable list.
- Add products to the cart from the Product screen.
- Reuse the same cart on the PoS screen.

1. Product ViewModel

File: `lib/features/products/presentation/view_models/product_view_model.dart`

Replace the file with:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/database/app_database.dart';
import '../../data/models/product.dart';
import '../../data/repositories/product_repository.dart';
import '../../data/services/product_local_service.dart';

final productRepositoryProvider = Provider<ProductRepository>((ref) {
  return ProductRepository(
    localService: ProductLocalService(
      database: AppDatabase.instance,
    ),
  );
});

final productViewModelProvider =
    AsyncNotifierProvider<ProductViewModel, ProductState>(
  ProductViewModel.new,
);

class ProductState {
  const ProductState({
    this.products = const [],
    this.searchText = '',
  });

  final List<Product> products;
  final String searchText;

  ProductState copyWith({
    List<Product>? products,
    String? searchText,
  }) {
    return ProductState(
      products: products ?? this.products,
      searchText: searchText ?? this.searchText,
    );
  }
}

class ProductViewModel extends AsyncNotifier<ProductState> {
  @override
  Future<ProductState> build() async {
    final repository = ref.read(productRepositoryProvider);
    final products = await repository.getAllProducts();

    return ProductState(products: products);
  }
```

```
  Future<void> search(String value) async {
    final repository = ref.read(productRepositoryProvider);

    state = const AsyncLoading();

    state = await AsyncValue.guard(() async {
      final products = await repository.searchProducts(value);
      return ProductState(
        products: products,
        searchText: value,
      );
    });
  }

  Future<void> clearSearch() {
    return search('');
  }
}
```

2. Product screen item

File: `lib/features/products/presentation/widgets/product_list_item.dart`

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/models/product.dart';

class ProductListItem extends ConsumerWidget {
  const ProductListItem({
    super.key,
    required this.product,
  });

  final Product product;

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final colorScheme = Theme.of(context).colorScheme;
    final isOutOfStock = product.stockQuantity <= 0;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: colorScheme.primaryContainer,
          child: Icon(
```

```
            Icons.inventory_2,
            color: colorScheme.onPrimaryContainer,
          ),
        ),
        title: Text(
          product.name,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Text(
          'Barcode: ${product.barcode}\nStock: ${product.stockQuantity}',
        ),
        isThreeLine: true,
        trailing: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              formatMoney(product.priceCents),
              style: TextStyle(
                color: colorScheme.primary,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 4),
            SizedBox(
              height: 32,
              child: FilledButton.icon(
                onPressed: isOutOfStock
                    ? null
                    : () {
                        ref
                            .read(cartProvider.notifier)
                            .addProduct(product);

                        ScaffoldMessenger.of(context).showSnackBar(
                          SnackBar(
                            content: Text('${product.name} added to cart'),
                            duration: const Duration(seconds: 1),
                          ),
                        );
                      },
                icon: const Icon(Icons.add_shopping_cart, size: 16),
                label: const Text('Add'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

3. Product search widget

File: `lib/features/products/presentation/widgets/product_search_field.dart`

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productsAsync = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Type Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: () {
                  ref.read(productViewModelProvider.notifier).clearSearch();
                },
              ),
            ),
            onChanged: (value) {
              ref.read(productViewModelProvider.notifier).search(value);
            },
          ),
        ),
        Expanded(
          child: productsAsync.when(
            loading: () => const Center(
              child: CircularProgressIndicator(),
            ),
            error: (error, stackTrace) => Center(
              child: Text(
                'Unable to load products:\n$error',
                textAlign: TextAlign.center,
              ),
            ),
            data: (productState) {
              if (productState.products.isEmpty) {
```

```
                return const Center(
                  child: Text('No products found'),
                );
              }

              return ListView.builder(
                padding: const EdgeInsets.only(
                  left: 8,
                  right: 8,
                  bottom: 16,
                ),
                itemCount: productState.products.length,
                itemBuilder: (context, index) {
                  return ProductListItem(
                    product: productState.products[index],
                  );
                },
              );
            },
          ),
        ),
      ],
    );
  }
}
```

4. PoS screen integration

File: `lib/features/cart/presentation/screens/pos_screen.dart`

Make sure the file uses both the product panel and the cart panel:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';
import '../widgets/cart_item_tile.dart';
import '../widgets/cart_summary.dart';

class PosScreen extends ConsumerWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Point of Sale'),
        actions: [
```

```
          IconButton(
            tooltip: 'Sales history',
            icon: const Icon(Icons.receipt_long),
            onPressed: () => context.go('/sales'),
          ),
          IconButton(
            tooltip: 'Settings',
            icon: const Icon(Icons.settings),
            onPressed: () => context.go('/settings'),
          ),
        ],
      ),
      body: Row(
        children: [
          SideNavigationRail(
            selectedIndex: 1,
            onDestinationSelected: (index) {
              _navigateToDestination(context, index);
            },
          ),
          const VerticalDivider(width: 1, thickness: 1),
          Expanded(
            child: Row(
              children: [
                const Expanded(
                  flex: 5,
                  child: _ProductsPanel(),
                ),
                const VerticalDivider(width: 1, thickness: 1),
                Expanded(
                  flex: 5,
                  child: _CartPanel(cart: cart),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }

  void _navigateToDestination(BuildContext context, int index) {
    switch (index) {
      case 0:
        context.go('/dashboard');
        break;
      case 1:
        context.go('/pos');
        break;
      case 2:
        context.go('/products');
        break;
      case 3:
        context.go('/inventory');
        break;
      case 4:
```

```
        context.go('/sales');
        break;
      case 5:
        context.go('/customers');
        break;
      case 6:
        context.go('/reports');
        break;
      case 7:
        context.go('/settings');
        break;
    }
  }
}

class _ProductsPanel extends StatelessWidget {
  const _ProductsPanel();

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surface,
      child: const Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Products',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: ProductSearchField(),
          ),
        ],
      ),
    );
  }
}

class _CartPanel extends StatelessWidget {
  const _CartPanel({
    required this.cart,
  });

  final List<CartItem> cart;

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surfaceContainerLowest,
      child: Column(
```

```
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Current Cart',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: cart.isEmpty
                ? const Center(
                    child: Text('The cart is empty'),
                  )
                : ListView.builder(
                    padding: const EdgeInsets.all(8),
                    itemCount: cart.length,
                    itemBuilder: (context, index) {
                      return CartItemTile(item: cart[index]);
                    },
                  ),
          ),
          const CartSummary(),
        ],
      ),
    );
  }
}
```

5. Cart add action

This is the key line that connects Product screen to PoS cart:

```
ref.read(cartProvider.notifier).addProduct(product);
```

It already exists in `ProductListItem`, so once the Product screen loads correctly, clicking **Add** will update the PoS cart immediately.

6. Important dependency check

Make sure these files still exist:

```
lib/features/cart/domain/cart_item.dart
lib/features/cart/presentation/view_models/cart_view_model.dart
lib/features/products/data/models/product.dart
lib/features/products/data/repositories/product_repository.dart
lib/features/products/data/services/product_local_service.dart
lib/core/database/app_database.dart
```

And make sure `main.dart` has `ProviderScope`.

7. Run the app

```
dart format lib
flutter pub get
flutter analyze
flutter run -d chrome
```

8. What you should see

- Left side: Product search field and product list.
- Right side: Current cart.
- Clicking **Add** on a product adds it to the cart.
- Searching filters the Product list.
- The PoS screen reflects the cart update immediately.

If you want, the next step can be to make the **Product screen a separate full page** in the left navigation menu instead of showing it inside the PoS split layout.

⁂

I lost the changes on my local machine for this application. wanted to add the git repository. guide me with the steps

Yes — the cleanest way is to **recreate the app folder, initialize Git, commit the recovered files, then push to a remote repository**. GitHub's workflow is: `git init`, stage files with `git add`, create a commit, add the remote with `git remote add origin <URL>`, then push with `git push -u origin main`.[187] [188] [189]

1. Check the project folder

Go to the root of your Flutter app:

```
cd path/to/your/flutter_project
```

Confirm the main Flutter files exist:

```
pubspec.yaml
lib/
android/
ios/
web/
windows/
```

If the project folder is missing or incomplete, stop and restore it from backup or the IDE local history first.

2. Recreate the Flutter `.gitignore`

Create this file in the project root:

File: `.gitignore`

```
.dart_tool/
.flutter-plugins
.flutter-plugins-dependencies
.packages
.pub-cache/
build/

.idea/
.vscode/
*.iml
*.ipr
*.iws

*.log
*.lock
.DS_Store

ios/Flutter/.last_build_id
android/app/debug
android/app/profile
android/app/release

lib/generated_plugin_registrant.dart
```

A Flutter `.gitignore` normally excludes build outputs, IDE metadata, and generated files.[190] [191]

3. Initialize Git

Run:

```
git init
```

This turns the current folder into a Git repository.[188]

4. Add and commit your files

Stage everything:

```
git add .
```

Create the first commit:

```
git commit -m "Initial commit"
```

If Git asks for your identity, configure it first:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

5. Create a remote repository

Create a new empty repository on GitHub, GitLab, or Bitbucket. Do **not** initialize it with a README if you already have files locally, unless you know how to merge histories.

Then copy the repository URL, for example:

```
https://github.com/yourname/pos_billing_app.git
```

6. Add the remote

Run:

```
git remote add origin https://github.com/yourname/pos_billing_app.git
```

Git uses `remote add` to connect your local repository to the online one.[192] [187]

7. Push to the remote

If your main branch is called `main`:

```
git branch -M main
git push -u origin main
```

GitHub documents `git push` as the command that uploads local branch commits to the remote repository.[189]

8. If the remote already has files

If the remote repo was not empty, first pull and merge:

```
git pull origin main --allow-unrelated-histories
```

Then push:

```
git push -u origin main
```

9. Recommended command sequence

Here is the full sequence in one place:

```
cd path/to/your/flutter_project
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/yourname/pos_billing_app.git
git push -u origin main
```

10. Good habit for recovery

Before pushing, run:

```
flutter pub get
flutter analyze
```

Then commit again if you fix issues:

```
git add .
git commit -m "Fix build and restore project"
git push
```

11. If you lost local changes

If the files are not present anymore, Git cannot recover what was never committed. In that case, check:

- IDE local history.
- Windows File History or backup.
- Recycle Bin.
- OneDrive / cloud backup.
- Previous clones on another machine.

If you want, I can give you a **safe recovery checklist** for Flutter projects, including the exact folder structure and files that should be present before you initialize Git.

⁂

Please give all the files that is developed until now. there are about 33 files indifferent folders.

Below is the consolidated file list we have developed so far, organized by folder. It follows the feature-first MVVM structure recommended by Flutter's architecture guidance.[197] [198]

Root files

- `.gitignore`
- `pubspec.yaml`
- `lib/main.dart`

App files

- `lib/app/app.dart`
- `lib/app/router.dart`
- `lib/app/theme.dart`
- `lib/app/widgets/side_navigation_rail.dart`

Core files

- `lib/core/database/app_database.dart`
- `lib/core/database/database_initializer.dart`
- `lib/core/database/database_initializer_stub.dart`
- `lib/core/database/database_initializer_io.dart`
- `lib/core/database/database_initializer_web.dart`
- `lib/core/barcode/barcode_scanner_service.dart`
- `lib/core/printing/receipt_printer_service.dart`

Product feature

- `lib/features/products/data/models/product.dart`
- `lib/features/products/data/models/product_mapper.dart`
- `lib/features/products/data/repositories/product_repository.dart`
- `lib/features/products/data/services/product_local_service.dart`
- `lib/features/products/presentation/view_models/product_view_model.dart`
- `lib/features/products/presentation/widgets/product_search_field.dart`
- `lib/features/products/presentation/widgets/product_list_item.dart`

Cart feature

- `lib/features/cart/domain/cart_item.dart`
- `lib/features/cart/presentation/screens/pos_screen.dart`
- `lib/features/cart/presentation/view_models/cart_view_model.dart`
- `lib/features/cart/presentation/widgets/cart_item_tile.dart`
- `lib/features/cart/presentation/widgets/cart_summary.dart`

Checkout feature

- `lib/features/checkout/domain/payment.dart`
- `lib/features/checkout/domain/sale.dart`
- `lib/features/checkout/domain/sale_item.dart`
- `lib/features/checkout/data/repositories/checkout_repository.dart`
- `lib/features/checkout/presentation/screens/checkout_screen.dart`
- `lib/features/checkout/presentation/view_models/checkout_view_model.dart`

Sales feature

- `lib/features/sales/presentation/screens/sales_history_screen.dart`

Placeholder / support files

- `lib/app/screens/placeholder_screen.dart`

Total count

That is **32 files** if we count only the files explicitly developed in the app codebase so far, plus the root `.gitignore` and `pubspec.yaml` makes it **34 items** total in the project list. Some of those support files were introduced as part of setup and platform initialization.

Notes

- The structure matches Flutter's recommended separation of views, view models, repositories, and services. [198] [197]
- The navigation rail is used for larger layouts, which is consistent with Material 3 guidance for side navigation on wider screens. [199] [200]

If you want, I can next give you the **exact content of each file in one-by-one copy/paste blocks** or create a **tree view with only the files that must currently exist.**

⁂

Give the final code for all of the files for by each group

Below is the **final code set, grouped by layer**, so you can recreate the whole app cleanly. This follows Flutter's recommended MVVM split: views and view models in the UI layer, repositories and services in the data layer. [207] [208] [209]

Root

File: `.gitignore`

```
.dart_tool/
.flutter-plugins
.flutter-plugins-dependencies
.packages
.pub-cache/
build/

.idea/
.vscode/
*.iml
*.ipr
*.iws

*.log
*.lock
.DS_Store

ios/Flutter/.last_build_id
android/app/debug
android/app/profile
android/app/release

lib/generated_plugin_registrant.dart
```

File: `pubspec.yaml`

Use your normal Flutter generated file, and ensure these dependencies are present:

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^2.5.1
  go_router: ^14.2.7
  path: ^1.9.0
  sqflite: ^2.3.3
  sqflite_common_ffi: ^2.3.3
  sqflite_common_ffi_web: ^0.4.5
  uuid: ^4.5.1
```

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeDatabase();

  runApp(
    const ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

App layer

File: `lib/app/app.dart`

```
import 'package:flutter/material.dart';

import 'router.dart';
import 'theme.dart';

class PosBillingApp extends StatelessWidget {
  const PosBillingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'POS Billing',
      theme: appTheme,
      routerConfig: appRouter,
    );
  }
}
```

File: `lib/app/router.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../features/cart/presentation/screens/pos_screen.dart';
import '../features/checkout/presentation/screens/checkout_screen.dart';
import '../features/sales/presentation/screens/sales_history_screen.dart';
import 'screens/placeholder_screen.dart';
```

```
final GoRouter appRouter = GoRouter(
  initialLocation: '/pos',
  routes: [
    GoRoute(
      path: '/dashboard',
      builder: (context, state) => const PlaceholderScreen(
        title: 'Dashboard',
        icon: Icons.dashboard,
      ),
    ),
    GoRoute(
      path: '/pos',
      builder: (context, state) => const PosScreen(),
    ),
    GoRoute(
      path: '/products',
      builder: (context, state) => const PlaceholderScreen(
        title: 'Products',
        icon: Icons.inventory_2,
      ),
    ),
    GoRoute(
      path: '/inventory',
      builder: (context, state) => const PlaceholderScreen(
        title: 'Inventory',
        icon: Icons.warehouse,
      ),
    ),
    GoRoute(
      path: '/sales',
      builder: (context, state) => const SalesHistoryScreen(),
    ),
    GoRoute(
      path: '/customers',
      builder: (context, state) => const PlaceholderScreen(
        title: 'Customers',
        icon: Icons.people,
      ),
    ),
    GoRoute(
      path: '/reports',
      builder: (context, state) => const PlaceholderScreen(
        title: 'Reports',
        icon: Icons.bar_chart,
      ),
    ),
    GoRoute(
      path: '/settings',
      builder: (context, state) => const PlaceholderScreen(
        title: 'Settings',
        icon: Icons.settings,
      ),
    ),
    GoRoute(
      path: '/checkout',
      builder: (context, state) => const CheckoutScreen(),
```

```
    ),
  ],
);
```

File: `lib/app/theme.dart`

```
import 'package:flutter/material.dart';

final ThemeData appTheme = ThemeData(
  useMaterial3: true,
  colorScheme: ColorScheme.fromSeed(
    seedColor: const Color(0xFF1565C0),
  ),
  scaffoldBackgroundColor: const Color(0xFFF5F7FA),
  appBarTheme: AppBarTheme(
    backgroundColor: ColorScheme.fromSeed(
      seedColor: const Color(0xFF1565C0),
    ).primary,
    foregroundColor: Colors.white,
    centerTitle: false,
    elevation: 0,
  ),
  inputDecorationTheme: InputDecorationTheme(
    filled: true,
    fillColor: Colors.white,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(10),
    ),
  ),
);
```

File: `lib/app/widgets/side_navigation_rail.dart`

```
import 'package:flutter/material.dart';

class SideNavigationRail extends StatelessWidget {
  const SideNavigationRail({
    super.key,
    required this.selectedIndex,
    required this.onDestinationSelected,
  });

  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    return NavigationRail(
      selectedIndex: selectedIndex,
      onDestinationSelected: onDestinationSelected,
      extended: true,
      minExtendedWidth: 220,
      backgroundColor: Theme.of(context).colorScheme.surface,
```

```
      leading: Padding(
        padding: const EdgeInsets.symmetric(vertical: 24, horizontal: 12),
        child: Row(
          children: [
            Icon(
              Icons.point_of_sale,
              size: 32,
              color: Theme.of(context).colorScheme.primary,
            ),
            const SizedBox(width: 12),
            Text(
              'POS Billing',
              style: Theme.of(context).textTheme.titleLarge?.copyWith(
                    fontWeight: FontWeight.bold,
                  ),
            ),
          ],
        ),
      ),
      destinations: const [
        NavigationRailDestination(
          icon: Icon(Icons.dashboard_outlined),
          selectedIcon: Icon(Icons.dashboard),
          label: Text('Dashboard'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.point_of_sale_outlined),
          selectedIcon: Icon(Icons.point_of_sale),
          label: Text('Point of Sale'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.inventory_2_outlined),
          selectedIcon: Icon(Icons.inventory_2),
          label: Text('Products'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.warehouse_outlined),
          selectedIcon: Icon(Icons.warehouse),
          label: Text('Inventory'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.receipt_long_outlined),
          selectedIcon: Icon(Icons.receipt_long),
          label: Text('Sales'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.people_outline),
          selectedIcon: Icon(Icons.people),
          label: Text('Customers'),
        ),
        NavigationRailDestination(
          icon: Icon(Icons.bar_chart_outlined),
          selectedIcon: Icon(Icons.bar_chart),
          label: Text('Reports'),
        ),
        NavigationRailDestination(
```

```
          icon: Icon(Icons.settings_outlined),
          selectedIcon: Icon(Icons.settings),
          label: Text('Settings'),
        ),
      ],
    );
  }
}
```

File: `lib/app/screens/placeholder_screen.dart`

```
import 'package:flutter/material.dart';

class PlaceholderScreen extends StatelessWidget {
  const PlaceholderScreen({
    super.key,
    required this.title,
    required this.icon,
  });

  final String title;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: Icon(icon),
        title: Text(title),
      ),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(icon, size: 80, color: Theme.of(context).colorScheme.primary),
            const SizedBox(height: 16),
            Text('$title screen'),
          ],
        ),
      ),
    );
  }
}
```

Core layer

File: `lib/core/database/database_initializer.dart`

```
import 'database_initializer_stub.dart'
    if (dart.library.io) 'database_initializer_io.dart'
    if (dart.library.html) 'database_initializer_web.dart';

Future<void> initializeDatabase() {
  return initializePlatformDatabase();
}
```

File: `lib/core/database/database_initializer_stub.dart`

```
Future<void> initializePlatformDatabase() async {}
```

File: `lib/core/database/database_initializer_io.dart`

```
import 'dart:io';

import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';

Future<void> initializePlatformDatabase() async {
  if (Platform.isWindows || Platform.isLinux || Platform.isMacOS) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }
}
```

File: `lib/core/database/database_initializer_web.dart`

```
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

Future<void> initializePlatformDatabase() async {
  databaseFactory = databaseFactoryFfiWeb;
}
```

File: `lib/core/database/app_database.dart`

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  AppDatabase._internal();

  static final AppDatabase instance = AppDatabase._internal();

  static Database? _database;
```

```
Future<Database> get database async {
  if (_database != null) return _database!;
  _database = await _openDatabase();
  return _database!;
}

Future<Database> _openDatabase() async {
  final databaseDirectory = await getDatabasesPath();
  final databasePath = join(databaseDirectory, 'pos_billing.db');

  return openDatabase(
    databasePath,
    version: 1,
    onConfigure: (db) async {
      await db.execute('PRAGMA foreign_keys = ON');
    },
    onCreate: (db, version) async {
      await _createTables(db);
      await _insertDemoProducts(db);
    },
  );
}

Future<void> _createTables(Database db) async {
  await db.execute('''
    CREATE TABLE products (
      id TEXT PRIMARY KEY,
      barcode TEXT NOT NULL UNIQUE,
      name TEXT NOT NULL,
      price_cents INTEGER NOT NULL,
      tax_rate INTEGER NOT NULL,
      stock_quantity INTEGER NOT NULL,
      is_active INTEGER NOT NULL DEFAULT 1
    )
  ''');

  await db.execute('''
    CREATE TABLE sales (
      id TEXT PRIMARY KEY,
      invoice_number TEXT NOT NULL UNIQUE,
      subtotal_cents INTEGER NOT NULL,
      discount_cents INTEGER NOT NULL,
      tax_cents INTEGER NOT NULL,
      total_cents INTEGER NOT NULL,
      payment_method TEXT NOT NULL,
      created_at TEXT NOT NULL,
      sync_status TEXT NOT NULL
    )
  ''');

  await db.execute('''
    CREATE TABLE sale_items (
      id TEXT PRIMARY KEY,
      sale_id TEXT NOT NULL,
      product_id TEXT NOT NULL,
```

```
        quantity INTEGER NOT NULL,
        unit_price_cents INTEGER NOT NULL,
        total_cents INTEGER NOT NULL,
        FOREIGN KEY (sale_id) REFERENCES sales (id) ON DELETE CASCADE,
        FOREIGN KEY (product_id) REFERENCES products (id)
      )
    ''');

    await db.execute('''
      CREATE TABLE stock_movements (
        id TEXT PRIMARY KEY,
        product_id TEXT NOT NULL,
        movement_type TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        reference_id TEXT,
        created_at TEXT NOT NULL,
        FOREIGN KEY (product_id) REFERENCES products (id)
      )
    ''');
  }

  Future<void> _insertDemoProducts(Database db) async {
    final products = [
      {
        'id': 'p-001',
        'barcode': '8900000000001',
        'name': 'Coffee',
        'price_cents': 299,
        'tax_rate': 19,
        'stock_quantity': 50,
        'is_active': 1,
      },
      {
        'id': 'p-002',
        'barcode': '8900000000002',
        'name': 'Tea',
        'price_cents': 199,
        'tax_rate': 19,
        'stock_quantity': 40,
        'is_active': 1,
      },
      {
        'id': 'p-003',
        'barcode': '8900000000003',
        'name': 'Sandwich',
        'price_cents': 499,
        'tax_rate': 7,
        'stock_quantity': 20,
        'is_active': 1,
      },
      {
        'id': 'p-004',
        'barcode': '8900000000004',
        'name': 'Water',
        'price_cents': 149,
        'tax_rate': 19,
```

```
        'stock_quantity': 100,
        'is_active': 1,
      },
    ];

    for (final product in products) {
      await db.insert(
        'products',
        product,
        conflictAlgorithm: ConflictAlgorithm.ignore,
      );
    }
  }

  Future<void> deleteDatabaseForDevelopment() async {
    final databaseDirectory = await getDatabasesPath();
    final databasePath = join(databaseDirectory, 'pos_billing.db');

    if (_database != null) {
      await _database!.close();
      _database = null;
    }

    await deleteDatabase(databasePath);
  }
}
```

File: `lib/core/barcode/barcode_scanner_service.dart`

```
abstract class BarcodeScannerService {
  Future<String?> scan();
}

class ManualBarcodeScannerService implements BarcodeScannerService {
  @override
  Future<String?> scan() async {
    return null;
  }
}
```

File: `lib/core/printing/receipt_printer_service.dart`

```
import '../../features/checkout/domain/sale.dart';

abstract class ReceiptPrinterService {
  Future<void> printReceipt(Sale sale);
}

class ReceiptPrinterPreviewService implements ReceiptPrinterService {
  @override
  Future<void> printReceipt(Sale sale) async {
    final receipt = StringBuffer();
```

```
    receipt.writeln('POS BILLING');
    receipt.writeln('------------------------------');
    receipt.writeln('Invoice: ${sale.invoiceNumber}');
    receipt.writeln('Date: ${sale.createdAt}');
    receipt.writeln('------------------------------');

    for (final item in sale.items) {
      receipt.writeln(
        '${item.productId} x${item.quantity} ${_formatMoney(item.totalCents)}',
      );
    }

    receipt.writeln('------------------------------');
    receipt.writeln('Subtotal: ${_formatMoney(sale.subtotalCents)}');
    receipt.writeln('Tax: ${_formatMoney(sale.taxCents)}');
    receipt.writeln('Total: ${_formatMoney(sale.totalCents)}');
    receipt.writeln('Payment: ${sale.paymentMethod.name}');

    // ignore: avoid_print
    print(receipt.toString());
  }

  String _formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }
}
```

Product feature

File: `lib/features/products/data/models/product.dart`

```
class Product {
  const Product({
    required this.id,
    required this.barcode,
    required this.name,
    required this.priceCents,
    required this.taxRate,
    required this.stockQuantity,
  });

  final String id;
  final String barcode;
  final String name;
  final int priceCents;
  final int taxRate;
  final int stockQuantity;
}
```

File: `lib/features/products/data/models/product_mapper.dart`

```
import 'product.dart';

class ProductMapper {
  static Product fromMap(Map<String, Object?> map) {
    return Product(
      id: map['id'] as String,
      barcode: map['barcode'] as String,
      name: map['name'] as String,
      priceCents: map['price_cents'] as int,
      taxRate: map['tax_rate'] as int,
      stockQuantity: map['stock_quantity'] as int,
    );
  }

  static Map<String, Object?> toMap(Product product) {
    return {
      'id': product.id,
      'barcode': product.barcode,
      'name': product.name,
      'price_cents': product.priceCents,
      'tax_rate': product.taxRate,
      'stock_quantity': product.stockQuantity,
      'is_active': 1,
    };
  }
}
```

File: `lib/features/products/data/services/product_local_service.dart`

```
import 'package:sqflite/sqflite.dart';

import '../../../../core/database/app_database.dart';
import '../models/product.dart';
import '../models/product_mapper.dart';

class ProductLocalService {
  ProductLocalService({
    required this.database,
  });

  final AppDatabase database;

  Future<List<Product>> getAllProducts() async {
    final db = await database.database;
    final rows = await db.query(
      'products',
      where: 'is_active = ?',
      whereArgs: [^21_1],
      orderBy: 'name ASC',
    );
    return rows.map(ProductMapper.fromMap).toList();
  }
```

```
  Future<List<Product>> searchProducts(String query) async {
    final db = await database.database;
    final searchText = query.trim();

    if (searchText.isEmpty) {
      return getAllProducts();
    }

    final rows = await db.query(
      'products',
      where: 'is_active = ? AND (name LIKE ? OR barcode LIKE ?)',
      whereArgs: [1, '%$searchText%', '%$searchText%'],
      orderBy: 'name ASC',
    );

    return rows.map(ProductMapper.fromMap).toList();
  }

  Future<Product?> getProductByBarcode(String barcode) async {
    final db = await database.database;
    final rows = await db.query(
      'products',
      where: 'barcode = ? AND is_active = ?',
      whereArgs: [barcode, 1],
      limit: 1,
    );

    if (rows.isEmpty) return null;
    return ProductMapper.fromMap(rows.first);
  }

  Future<void> insertProduct(Product product) async {
    final db = await database.database;
    await db.insert(
      'products',
      ProductMapper.toMap(product),
      conflictAlgorithm: ConflictAlgorithm.abort,
    );
  }
}
```

File: `lib/features/products/data/repositories/product_repository.dart`

```
import '../models/product.dart';
import '../services/product_local_service.dart';

class ProductRepository {
  ProductRepository({
    required this.localService,
  });

  final ProductLocalService localService;

  Future<List<Product>> getAllProducts() => localService.getAllProducts();
```

```
  Future<List<Product>> searchProducts(String query) =>
      localService.searchProducts(query);

  Future<Product?> getProductByBarcode(String barcode) =>
      localService.getProductByBarcode(barcode);

  Future<void> addProduct(Product product) => localService.insertProduct(product);
}
```

File: `lib/features/products/presentation/view_models/product_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/database/app_database.dart';
import '../../data/models/product.dart';
import '../../data/repositories/product_repository.dart';
import '../../data/services/product_local_service.dart';

final productRepositoryProvider = Provider<ProductRepository>((ref) {
  return ProductRepository(
    localService: ProductLocalService(
      database: AppDatabase.instance,
    ),
  );
});

final productViewModelProvider =
    AsyncNotifierProvider<ProductViewModel, ProductState>(
  ProductViewModel.new,
);

class ProductState {
  const ProductState({
    this.products = const [],
    this.searchText = '',
  });

  final List<Product> products;
  final String searchText;

  ProductState copyWith({
    List<Product>? products,
    String? searchText,
  }) {
    return ProductState(
      products: products ?? this.products,
      searchText: searchText ?? this.searchText,
    );
  }
}

class ProductViewModel extends AsyncNotifier<ProductState> {
  @override
  Future<ProductState> build() async {
```

```
    final repository = ref.read(productRepositoryProvider);
    final products = await repository.getAllProducts();
    return ProductState(products: products);
  }

  Future<void> search(String value) async {
    final repository = ref.read(productRepositoryProvider);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      final products = await repository.searchProducts(value);
      return ProductState(
        products: products,
        searchText: value,
      );
    });
  }

  Future<void> clearSearch() => search('');
}
```

File: `lib/features/products/presentation/widgets/product_search_field.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productsAsync = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Type Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: () {
                  ref.read(productViewModelProvider.notifier).clearSearch();
                },
              ),
            ),
            onChanged: (value) {
              ref.read(productViewModelProvider.notifier).search(value);
```

```
            },
          ),
        ),
        Expanded(
          child: productsAsync.when(
            loading: () => const Center(child: CircularProgressIndicator()),
            error: (error, stackTrace) => Center(
              child: Text(
                'Unable to load products:\n$error',
                textAlign: TextAlign.center,
              ),
            ),
            data: (productState) {
              if (productState.products.isEmpty) {
                return const Center(child: Text('No products found'));
              }

              return ListView.builder(
                padding: const EdgeInsets.only(
                  left: 8,
                  right: 8,
                  bottom: 16,
                ),
                itemCount: productState.products.length,
                itemBuilder: (context, index) {
                  return ProductListItem(
                    product: productState.products[index],
                  );
                },
              );
            },
          ),
        ),
      ],
    );
  }
}
```

File: `lib/features/products/presentation/widgets/product_list_item.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/models/product.dart';

class ProductListItem extends ConsumerWidget {
  const ProductListItem({
    super.key,
    required this.product,
  });

  final Product product;

  String formatMoney(int cents) {
```

```
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final colorScheme = Theme.of(context).colorScheme;
    final isOutOfStock = product.stockQuantity <= 0;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: colorScheme.primaryContainer,
          child: Icon(
            Icons.inventory_2,
            color: colorScheme.onPrimaryContainer,
          ),
        ),
        title: Text(
          product.name,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Text(
          'Barcode: ${product.barcode}\nStock: ${product.stockQuantity}',
        ),
        isThreeLine: true,
        trailing: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              formatMoney(product.priceCents),
              style: TextStyle(
                color: colorScheme.primary,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 4),
            SizedBox(
              height: 32,
              child: FilledButton.icon(
                onPressed: isOutOfStock
                    ? null
                    : () {
                        ref.read(cartProvider.notifier).addProduct(product);
                        ScaffoldMmessenger.of(context).showSnackBar(
                          SnackBar(
                            content: Text('${product.name} added to cart'),
                            duration: const Duration(seconds: 1),
                          ),
                        );
                      },
                icon: const Icon(Icons.add_shopping_cart, size: 16),
                label: const Text('Add'),
              ),
            ),
          ],
```

```
        ),
      ),
    );
  }
}
```

Cart feature

File: `lib/features/cart/domain/cart_item.dart`

```
import '../../products/data/models/product.dart';

class CartItem {
  const CartItem({
    required this.product,
    required this.quantity,
  });

  final Product product;
  final int quantity;

  int get totalCents => product.priceCents * quantity;

  CartItem copyWith({
    Product? product,
    int? quantity,
  }) {
    return CartItem(
      product: product ?? this.product,
      quantity: quantity ?? this.quantity,
    );
  }
}
```

File: `lib/features/cart/presentation/view_models/cart_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../features/products/data/models/product.dart';
import '../../domain/cart_item.dart';

final cartProvider = NotifierProvider<CartViewModel, List<CartItem>>(
  CartViewModel.new,
);

class CartViewModel extends Notifier<List<CartItem>> {
  @override
  List<CartItem> build() {
    return const [];
  }

  void addProduct(Product product) {
```

```
    final existingIndex = state.indexWhere(
      (item) => item.product.id == product.id,
    );

    if (existingIndex == -1) {
      state = [...state, CartItem(product: product, quantity: 1)];
      return;
    }

    final items = [...state];
    final existingItem = items[existingIndex];

    if (existingItem.quantity >= product.stockQuantity) return;

    items[existingIndex] = existingItem.copyWith(
      quantity: existingItem.quantity + 1,
    );
    state = items;
  }

  void increaseQuantity(String productId) {
    final index = state.indexWhere((item) => item.product.id == productId);
    if (index == -1) return;

    final item = state[index];
    if (item.quantity >= item.product.stockQuantity) return;

    final items = [...state];
    items[index] = item.copyWith(quantity: item.quantity + 1);
    state = items;
  }

  void decreaseQuantity(String productId) {
    final index = state.indexWhere((item) => item.product.id == productId);
    if (index == -1) return;

    final item = state[index];
    if (item.quantity <= 1) {
      removeProduct(productId);
      return;
    }

    final items = [...state];
    items[index] = item.copyWith(quantity: item.quantity - 1);
    state = items;
  }

  void removeProduct(String productId) {
    state = state.where((item) => item.product.id != productId).toList();
  }

  void clear() {
    state = const [];
  }

  int get itemCount => state.fold(0, (total, item) => total + item.quantity);
```

```
  int get subtotalCents =>
      state.fold(0, (total, item) => total + item.totalCents);

  int get taxCents => state.fold(
        0,
        (total, item) => total + (item.totalCents * item.product.taxRate ~/ 100),
      );

  int get totalCents => subtotalCents + taxCents;
}
```

File: `lib/features/cart/presentation/widgets/cart_item_tile.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';

class CartItemTile extends ConsumerWidget {
  const CartItemTile({
    super.key,
    required this.item,
  });

  final CartItem item;

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cartViewModel = ref.read(cartProvider.notifier);

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(8),
        child: Row(
          children: [
            const Icon(Icons.shopping_bag_outlined),
            const SizedBox(width: 8),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    item.product.name,
                    style: const TextStyle(fontWeight: FontWeight.bold),
                  ),
                  Text('${formatMoney(item.product.priceCents)} each'),
                ],
              ),
            ),
```

```
            IconButton(
              tooltip: 'Decrease quantity',
              icon: const Icon(Icons.remove_circle_outline),
              onPressed: () {
                cartViewModel.decreaseQuantity(item.product.id);
              },
            ),
            Text(
              '${item.quantity}',
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
            IconButton(
              tooltip: 'Increase quantity',
              icon: const Icon(Icons.add_circle_outline),
              onPressed: item.quantity >= item.product.stockQuantity
                  ? null
                  : () {
                      cartViewModel.increaseQuantity(item.product.id);
                    },
            ),
            const SizedBox(width: 8),
            Text(
              formatMoney(item.totalCents),
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
            IconButton(
              tooltip: 'Remove item',
              icon: const Icon(Icons.delete_outline),
              onPressed: () {
                cartViewModel.removeProduct(item.product.id);
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

File: `lib/features/cart/presentation/widgets/cart_summary.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../view_models/cart_view_model.dart';

class CartSummary extends ConsumerWidget {
  const CartSummary({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
```

```
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);
    final cartViewModel = ref.read(cartProvider.notifier);

    return Card(
      margin: const EdgeInsets.all(16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Row(
              children: [
                Icon(Icons.shopping_cart,
                    color: Theme.of(context).colorScheme.primary),
                const SizedBox(width: 8),
                Text(
                  'Cart: ${cartViewModel.itemCount} item(s)',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
              ],
            ),
            const SizedBox(height: 12),
            _SummaryRow(label: 'Subtotal', value: formatMoney(cartViewModel.subtotal(
            _SummaryRow(label: 'Tax', value: formatMoney(cartViewModel.taxCents)),
            const Divider(),
            _SummaryRow(
              label: 'Total',
              value: formatMoney(cartViewModel.totalCents),
              isTotal: true,
            ),
            const SizedBox(height: 12),
            FilledButton.icon(
              onPressed: cart.isEmpty
                  ? null
                  : () {
                      context.push('/checkout');
                    },
              icon: const Icon(Icons.payment),
              label: const Text('Checkout'),
            ),
          ],
        ),
      ),
    );
  }
}

class _SummaryRow extends StatelessWidget {
  const _SummaryRow({
    required this.label,
    required this.value,
    this.isTotal = false,
  });

  final String label;
  final String value;
```

```
  final bool isTotal;

  @override
  Widget build(BuildContext context) {
    final style = isTotal
        ? Theme.of(context).textTheme.titleLarge?.copyWith(
              fontWeight: FontWeight.bold,
            )
        : Theme.of(context).textTheme.bodyLarge;

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 3),
      child: Row(
        children: [
          Expanded(child: Text(label, style: style)),
          Text(value, style: style),
        ],
      ),
    );
  }
}
```

File: `lib/features/cart/presentation/screens/pos_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../../../products/presentation/widgets/product_search_field.dart';
import '../../domain/cart_item.dart';
import '../view_models/cart_view_model.dart';
import '../widgets/cart_item_tile.dart';
import '../widgets/cart_summary.dart';

class PosScreen extends ConsumerWidget {
  const PosScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cart = ref.watch(cartProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Point of Sale'),
        actions: [
          IconButton(
            tooltip: 'Sales history',
            icon: const Icon(Icons.receipt_long),
            onPressed: () => context.go('/sales'),
          ),
          IconButton(
            tooltip: 'Settings',
            icon: const Icon(Icons.settings),
            onPressed: () => context.go('/settings'),
```

```
          ),
        ],
      ),
      body: Row(
        children: [
          SideNavigationRail(
            selectedIndex: 1,
            onDestinationSelected: (index) {
              _navigateToDestination(context, index);
            },
          ),
          const VerticalDivider(width: 1, thickness: 1),
          Expanded(
            child: Row(
              children: [
                const Expanded(
                  flex: 5,
                  child: _ProductsPanel(),
                ),
                const VerticalDivider(width: 1, thickness: 1),
                Expanded(
                  flex: 5,
                  child: _CartPanel(cart: cart),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }

  void _navigateToDestination(BuildContext context, int index) {
    switch (index) {
      case 0:
        context.go('/dashboard');
        break;
      case 1:
        context.go('/pos');
        break;
      case 2:
        context.go('/products');
        break;
      case 3:
        context.go('/inventory');
        break;
      case 4:
        context.go('/sales');
        break;
      case 5:
        context.go('/customers');
        break;
      case 6:
        context.go('/reports');
        break;
      case 7:
```

```
        context.go('/settings');
        break;
    }
  }
}

class _ProductsPanel extends StatelessWidget {
  const _ProductsPanel();

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surface,
      child: const Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Products',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: ProductSearchField(),
          ),
        ],
      ),
    );
  }
}

class _CartPanel extends StatelessWidget {
  const _CartPanel({
    required this.cart,
  });

  final List<CartItem> cart;

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surfaceContainerLowest,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          const Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Current Cart',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
```

```
              ),
            ),
          ),
          Expanded(
            child: cart.isEmpty
                ? const Center(
                    child: Text('The cart is empty'),
                  )
                : ListView.builder(
                    padding: const EdgeInsets.all(8),
                    itemCount: cart.length,
                    itemBuilder: (context, index) {
                      return CartItemTile(item: cart[index]);
                    },
                  ),
          ),
          const CartSummary(),
        ],
      ),
    );
  }
}
```

Checkout feature

File: `lib/features/checkout/domain/payment.dart`

```
enum PaymentMethod {
  cash,
  card,
  digital,
}
```

File: `lib/features/checkout/domain/sale_item.dart`

```
class SaleItem {
  const SaleItem({
    required this.id,
    required this.saleId,
    required this.productId,
    required this.quantity,
    required this.unitPriceCents,
    required this.totalCents,
  });

  final String id;
  final String saleId;
  final String productId;
  final int quantity;
  final int unitPriceCents;
```

```
  final int totalCents;
}
```

File: `lib/features/checkout/domain/sale.dart`

```
import 'payment.dart';
import 'sale_item.dart';

class Sale {
  const Sale({
    required this.id,
    required this.invoiceNumber,
    required this.items,
    required this.subtotalCents,
    required this.discountCents,
    required this.taxCents,
    required this.totalCents,
    required this.paymentMethod,
    required this.createdAt,
    required this.syncStatus,
  });

  final String id;
  final String invoiceNumber;
  final List<SaleItem> items;
  final int subtotalCents;
  final int discountCents;
  final int taxCents;
  final int totalCents;
  final PaymentMethod paymentMethod;
  final DateTime createdAt;
  final String syncStatus;
}
```

File: `lib/features/checkout/data/repositories/checkout_repository.dart`

```
import 'package:sqflite/sqflite.dart';

import '../../../../core/database/app_database.dart';
import '../../domain/sale.dart';

class CheckoutRepository {
  CheckoutRepository({
    required this.database,
  });

  final AppDatabase database;

  Future<void> saveSale(Sale sale) async {
    final db = await database.database;

    await db.transaction((transaction) async {
      for (final item in sale.items) {
```

```
        final products = await transaction.query(
          'products',
          columns: ['stock_quantity'],
          where: 'id = ? AND is_active = ?',
          whereArgs: [item.productId, 1],
          limit: 1,
        );

        if (products.isEmpty) {
          throw Exception('Product not found: ${item.productId}');
        }

        final availableStock = products.first['stock_quantity'] as int;

        if (availableStock < item.quantity) {
          throw Exception('Insufficient stock for product: ${item.productId}');
        }
      }

      await transaction.insert(
        'sales',
        {
          'id': sale.id,
          'invoice_number': sale.invoiceNumber,
          'subtotal_cents': sale.subtotalCents,
          'discount_cents': sale.discountCents,
          'tax_cents': sale.taxCents,
          'total_cents': sale.totalCents,
          'payment_method': sale.paymentMethod.name,
          'created_at': sale.createdAt.toIso8601String(),
          'sync_status': sale.syncStatus,
        },
        conflictAlgorithm: ConflictAlgorithm.abort,
      );

      for (final item in sale.items) {
        await transaction.insert(
          'sale_items',
          {
            'id': item.id,
            'sale_id': item.saleId,
            'product_id': item.productId,
            'quantity': item.quantity,
            'unit_price_cents': item.unitPriceCents,
            'total_cents': item.totalCents,
          },
          conflictAlgorithm: ConflictAlgorithm.abort,
        );

        await transaction.rawUpdate(
          '''
          UPDATE products
          SET stock_quantity = stock_quantity - ?
          WHERE id = ?
          ''',
          [item.quantity, item.productId],
```

```
        );

        await transaction.insert(
          'stock_movements',
          {
            'id': '${sale.id}-${item.productId}',
            'product_id': item.productId,
            'movement_type': 'sale',
            'quantity': -item.quantity,
            'reference_id': sale.id,
            'created_at': sale.createdAt.toIso8601String(),
          },
          conflictAlgorithm: ConflictAlgorithm.abort,
        );
      }
    });
  }
}
```

File: `lib/features/checkout/presentation/view_models/checkout_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:uuid/uuid.dart';

import '../../../../core/database/app_database.dart';
import '../../../cart/domain/cart_item.dart';
import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/repositories/checkout_repository.dart';
import '../../domain/payment.dart';
import '../../domain/sale.dart';
import '../../domain/sale_item.dart';

final checkoutRepositoryProvider = Provider<CheckoutRepository>((ref) {
  return CheckoutRepository(
    database: AppDatabase.instance,
  );
});

final checkoutViewModelProvider =
    NotifierProvider<CheckoutViewModel, CheckoutState>(
  CheckoutViewModel.new,
);

class CheckoutState {
  const CheckoutState({
    this.isSaving = false,
    this.isCompleted = false,
    this.errorMessage,
    this.lastSale,
  });

  final bool isSaving;
  final bool isCompleted;
  final String? errorMessage;
  final Sale? lastSale;
```

```
  CheckoutState copyWith({
    bool? isSaving,
    bool? isCompleted,
    String? errorMessage,
    Sale? lastSale,
    bool clearError = false,
  }) {
    return CheckoutState(
      isSaving: isSaving ?? this.isSaving,
      isCompleted: isCompleted ?? this.isCompleted,
      errorMessage: clearError ? null : errorMessage ?? this.errorMessage,
      lastSale: lastSale ?? this.lastSale,
    );
  }
}

class CheckoutViewModel extends Notifier<CheckoutState> {
  static const Uuid _uuid = Uuid();

  @override
  CheckoutState build() {
    return const CheckoutState();
  }

  Future<void> completeCashSale(WidgetRef ref) async {
    final cart = ref.read(cartProvider);
    if (cart.isEmpty) {
      state = state.copyWith(errorMessage: 'The cart is empty.');
      return;
    }

    await _completeSale(
      ref: ref,
      cart: cart,
      paymentMethod: PaymentMethod.cash,
    );
  }

  Future<void> completeCardSale(WidgetRef ref) async {
    final cart = ref.read(cartProvider);
    if (cart.isEmpty) {
      state = state.copyWith(errorMessage: 'The cart is empty.');
      return;
    }

    await _completeSale(
      ref: ref,
      cart: cart,
      paymentMethod: PaymentMethod.card,
    );
  }

  Future<void> _completeSale({
    required WidgetRef ref,
    required List<CartItem> cart,
```

```
    required PaymentMethod paymentMethod,
  }) async {
    state = state.copyWith(isSaving: true, clearError: true);

    try {
      final saleId = _uuid.v4();

      final saleItems = cart.map((cartItem) {
        return SaleItem(
          id: _uuid.v4(),
          saleId: saleId,
          productId: cartItem.product.id,
          quantity: cartItem.quantity,
          unitPriceCents: cartItem.product.priceCents,
          totalCents: cartItem.totalCents,
        );
      }).toList();

      final subtotalCents =
          cart.fold<int>(0, (sum, item) => sum + item.totalCents);

      final taxCents = cart.fold<int>(
        0,
        (sum, item) => sum + (item.totalCents * item.product.taxRate ~/ 100),
      );

      final sale = Sale(
        id: saleId,
        invoiceNumber: 'INV-${DateTime.now().millisecondsSinceEpoch}',
        items: saleItems,
        subtotalCents: subtotalCents,
        discountCents: 0,
        taxCents: taxCents,
        totalCents: subtotalCents + taxCents,
        paymentMethod: paymentMethod,
        createdAt: DateTime.now(),
        syncStatus: 'pending',
      );

      final repository = ref.read(checkoutRepositoryProvider);
      await repository.saveSale(sale);

      ref.read(cartProvider.notifier).clear();

      state = state.copyWith(
        isSaving: false,
        isCompleted: true,
        lastSale: sale,
      );
    } catch (error) {
      state = state.copyWith(
        isSaving: false,
        errorMessage: 'Unable to save the sale: $error',
      );
    }
  }
```

```
  void reset() {
    state = const CheckoutState();
  }
}
```

File: `lib/features/checkout/presentation/screens/checkout_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../view_models/checkout_view_model.dart';

class CheckoutScreen extends ConsumerWidget {
  const CheckoutScreen({super.key});

  String formatMoney(int cents) {
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final cartViewModel = ref.watch(cartProvider.notifier);
    final cart = ref.watch(cartProvider);
    final checkoutState = ref.watch(checkoutViewModelProvider);
    final checkoutViewModel = ref.read(checkoutViewModelProvider.notifier);

    ref.listen<CheckoutState>(
      checkoutViewModelProvider,
      (previous, next) {
        if (next.isCompleted && !next.isSaving) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Sale completed successfully')),
          );
        }

        if (next.errorMessage != null &&
            next.errorMessage != previous?.errorMessage) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(next.errorMessage!)),
          );
        }
      },
    );

    return Scaffold(
      appBar: AppBar(
        leading: const Icon(Icons.payment),
        title: const Text('Checkout'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
          children: [
            Card(
              child: Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                  children: [
                    const Icon(Icons.receipt_long, size: 64),
                    const SizedBox(height: 12),
                    const Text('Amount to pay'),
                    const SizedBox(height: 8),
                    Text(
                      formatMoney(cartViewModel.totalCents),
                      style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                            fontWeight: FontWeight.bold,
                          ),
                    ),
                  ],
                ),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: cart.isEmpty || checkoutState.isSaving
                  ? null
                  : () {
                      checkoutViewModel.completeCashSale(ref);
                    },
              icon: checkoutState.isSaving
                  ? const SizedBox(
                      width: 18,
                      height: 18,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Icon(Icons.money),
              label: const Text('Pay with cash'),
            ),
            const SizedBox(height: 12),
            OutlinedButton.icon(
              onPressed: cart.isEmpty || checkoutState.isSaving
                  ? null
                  : () {
                      checkoutViewModel.completeCardSale(ref);
                    },
              icon: const Icon(Icons.credit_card),
              label: const Text('Pay by card'),
            ),
          ],
        ),
      ),
    );
  }
}
```

File: `lib/features/sales/presentation/screens/sales_history_screen.dart`

```
import 'package:flutter/material.dart';

class SalesHistoryScreen extends StatelessWidget {
  const SalesHistoryScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        leading: const Icon(Icons.receipt_long),
        title: const Text('Sales History'),
      ),
      body: const Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(Icons.receipt_long_outlined, size: 80, color: Colors.grey),
            SizedBox(height: 12),
            Text('Completed sales will appear here'),
          ],
        ),
      ),
    );
  }
}
```

What to do next

1. Create the files exactly as listed.
2. Run:

   ```
   flutter pub get
   dart format lib
   flutter analyze
   ```

3. Start the app:

   ```
   flutter run
   ```

If you want, I can next give you the **same code split into copy/paste batches by folder**, so you can add them file by file without scrolling through the whole answer.

⁂

resolve the error

error - Target of URI doesn't exist: '../../../features/products/data/models/product.dart'. Try creating the file referenced by the URI, or try using a URI
for a file that does exist - lib\features\cart\presentation\view_models\cart_view_model.dart:3:8 - uri_does_not_exist

The import path is wrong in `cart_view_model.dart`. The file is inside `lib/features/cart/presentation/view_models/`, so `../../../features/...` goes too far and points to a non-existing location. Dart raises `uri_does_not_exist` when the URI does not match an actual file path. [217] [218]

Fix the import

File: `lib/features/cart/presentation/view_models/cart_view_model.dart`

Replace this line:

```
import '../../../features/products/data/models/product.dart';
```

with:

```
import '../../../products/data/models/product.dart';
```

That is the correct relative path from:

```
lib/features/cart/presentation/view_models/cart_view_model.dart
```

to:

```
lib/features/products/data/models/product.dart
```

Correct imports for the file

File: `lib/features/cart/presentation/view_models/cart_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../products/data/models/product.dart';
import '../../domain/cart_item.dart';
```

Why this happened

From `cart_view_model.dart`, the path structure is:

```
cart/
  presentation/
    view_models/
```

To reach `products/data/models/product.dart`, go up three levels to `features/`, then into `products/data/models/`. So the correct path is:

```
../../../products/data/models/product.dart
```

not:

```
../../../features/products/data/models/product.dart
```

The analyzer's `uri_does_not_exist` diagnostic usually means the URI is mistyped or the file is missing.[217]

Run again

After fixing the import:

```
dart format lib
flutter analyze
```

If you still get import errors, check for similar wrong imports in these files too:

- `lib/features/cart/domain/cart_item.dart`
- `lib/features/products/presentation/widgets/product_list_item.dart`
- `lib/features/products/presentation/view_models/product_view_model.dart`

The safest approach is to keep imports consistent and relative from the current file location, or switch to `package:` imports everywhere if you prefer a more stable structure.

⁂

fix the error

error - Undefined class 'Product'. Try changing the name to the name of an existing class, or creating a class with the name 'Product' - lib\features\cart\presentation\view_models\cart_view_model.dart:16:19 - undefined_class

`Product` is not visible in `cart_view_model.dart`. The class exists, but the file is importing the wrong path or not importing it at all. Dart gives `undefined_class` when a class is outside the current scope and no correct import brings it in.[227]

Fix `cart_view_model.dart`

File: `lib/features/cart/presentation/view_models/cart_view_model.dart`

Replace the import section with:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../products/data/models/product.dart';
import '../../domain/cart_item.dart';
```

So the full file starts like this:

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../products/data/models/product.dart';
import '../../domain/cart_item.dart';

final cartProvider = NotifierProvider<CartViewModel, List<CartItem>>(
  CartViewModel.new,
);

class CartViewModel extends Notifier<List<CartItem>> {
  @override
  List<CartItem> build() {
    return const [];
  }

  void addProduct(Product product) {
    ...
  }
}
```

Check the Product file

File: `lib/features/products/data/models/product.dart`

Make sure it contains the class exactly:

```
class Product {
  const Product({
    required this.id,
    required this.barcode,
    required this.name,
    required this.priceCents,
    required this.taxRate,
    required this.stockQuantity,
  });

  final String id;
  final String barcode;
```

```
  final String name;
  final int priceCents;
  final int taxRate;
  final int stockQuantity;
}
```

Why this happened

From:

```
lib/features/cart/presentation/view_models/cart_view_model.dart
```

the correct relative import to the product model is:

```
../../../products/data/models/product.dart
```

not:

```
../../../features/products/data/models/product.dart
```

That wrong version points to a path that does not exist. The analyzer then sees `Product` as undefined. [228] [229]

Also check for duplicate or wrong imports

If you still get the error, verify there is no broken import elsewhere in the same file, and that the `Product` class has not been renamed. A class used in a type position must be imported from a file that actually exists. [227]

Run again

```
dart format lib
flutter analyze
```

If you want, I can now give you the **correct import map for all files** so these path errors do not happen again.

⁂

fix the error

error - Invalid constant value - lib\main.dart:11:37 - invalid_constant

The issue is that `main.dart` is using `const ProviderScope(...)`, but something inside that widget tree is not const-safe in your current setup. In Dart, `Invalid constant value` usually

means a `const` expression is being used with a non-const value or a non-const constructor. [238] [239] [240]

Fix `main.dart`

File: `lib/main.dart`

Change this:

```
runApp(
  const ProviderScope(
    child: PosBillingApp(),
  ),
);
```

to this:

```
runApp(
  ProviderScope(
    child: PosBillingApp(),
  ),
);
```

Removing `const` is the correct fix when the widget tree contains something that cannot be evaluated as a compile-time constant. [239] [240]

Full file

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeDatabase();

  runApp(
    ProviderScope(
      child: PosBillingApp(),
    ),
  );
}
```

Why this happens

`const` can only be used when every value in that expression is known at compile time. If any part of the constructor chain is not const, the analyzer reports `invalid_constant`.[241] [238]

Run again

```
dart format lib
flutter analyze
```

If the error still appears after this change, the next place to check is any other `const` widget that contains a runtime value, especially inside `MaterialApp.router` or any screen constructors.

⁂

fix the below errors

error - The function 'BillingGuru' isn't defined. Try importing the library that defines 'BillingGuru', correcting the name to the name of an existing function, or defining a function named 'BillingGuru' - lib\main.dart:11:37 - undefined_function error - The name 'BillingGuru' isn't a class. Try correcting the name to match an existing class - test\widget_test.dart:17:35 - creation_with_non_type

The app root widget was renamed to `BillingGuru`, but `main.dart` and `test/widget_test.dart` still reference it incorrectly. Update both files so they use the actual widget class name defined in `lib/app/app.dart`. Flutter's default widget test also must import and pump the real root widget, or you'll get `undefined_function` / `creation_with_non_type`.[248] [249] [250]

Fix `main.dart`

File: `lib/main.dart`

Use this:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeDatabase();

  runApp(
    ProviderScope(
      child: const PosBillingApp(),
    ),
```

```
  );
}
```

Make sure you are calling `PosBillingApp()`, not `BillingGuru()`. The name you pass to `runApp` must match the actual root widget class. [251] [248]

Fix `widget_test.dart`

File: `test/widget_test.dart`

Replace the content with:

```
import 'package:flutter_test/flutter_test.dart';
import 'package:your_app_name/app/app.dart';

void main() {
  testWidgets('App smoke test', (WidgetTester tester) async {
    await tester.pumpWidget(const PosBillingApp());

    expect(find.text('POS Billing'), findsOneWidget);
  });
}
```

Replace `your_app_name` with the package name from `pubspec.yaml`.

If you do not need the default widget test yet, you can also temporarily delete `test/widget_test.dart`. Flutter's testing docs note that widget tests are optional, and many projects replace the generated template when the root widget changes. [252] [250]

Why this happened

- `BillingGuru` is being used in `main.dart`, but no class with that name exists.
- `widget_test.dart` still tries to build `BillingGuru`, so the test fails too.
- The analyzer reports `undefined_function` when it sees something used like a function call but no such function exists. It reports `creation_with_non_type` when you try to instantiate something that is not a class. [251]

Run again

```
dart format lib test
flutter analyze
flutter test
```

If you want, I can also give you the **correct final contents of** `lib/app/app.dart` so the root widget name is fully aligned across the project.

⁂

while pushing to a remote repository getting the below exception

remote: Repository not found.

`remote: Repository not found` usually means one of three things: the remote URL is wrong, the repository does not exist, or your Git account does not have access to it. GitHub's troubleshooting guidance says to verify the exact clone/push URL and confirm you have permission to the repo.[258] [259]

Check the remote URL

Run:

```
git remote -v
```

Make sure the URL matches the repository exactly, including the owner, repo name, and case. If it is wrong, fix it:

```
git remote set-url origin https://github.com/<OWNER>/<REPO>.git
```

A wrong remote URL is one of the most common causes of this error.[260] [259]

Confirm the repository exists

Open the repository URL in your browser:

```
https://github.com/<OWNER>/<REPO>
```

If the page does not open, the repo may not exist yet, or the name may be misspelled. GitHub notes that the repo must exist and be accessible to your account.[258]

Check your access

If the repository is private, make sure your GitHub account has write access. If you are using HTTPS and two-factor authentication, use a **personal access token** instead of a password. GitHub and Git community guidance both point out that authentication or permission issues can surface as "Repository not found."[261] [262] [258]

If you use HTTPS

Update the remote like this:

```
git remote set-url origin https://github.com/<OWNER>/<REPO>.git
git push -u origin main
```

If prompted for credentials, use your GitHub username and a personal access token. GitHub no longer accepts normal account passwords for Git operations over HTTPS. [259] [261]

If you use SSH

Check the remote:

```
git remote -v
```

It should look like:

```
git@github.com:<OWNER>/<REPO>.git
```

Then test authentication:

```
ssh -T git@github.com
```

If SSH auth succeeds but the repo still fails, the issue is usually the repository path or access rights. [263] [259]

If the repo is new

If you created the repository on GitHub after your local project was already initialized, make sure you added the correct remote:

```
git remote remove origin
git remote add origin https://github.com/<OWNER>/<REPO>.git
git branch -M main
git push -u origin main
```

Fast checklist

- The repo exists on GitHub.
- The remote URL matches exactly.
- You are authenticated as the correct GitHub account.
- You have permission to push.
- You are using a PAT for HTTPS if needed. [264] [259] [258]

If you share the output of `git remote -v` and the repository URL format you used, I can tell you exactly what to change.

⁂

making a repository public in Github

To make a GitHub repository public, open the repository on GitHub, go to **Settings**, then scroll to the **Danger Zone** and use **Change repository visibility** to switch it to **Public**. GitHub notes that only users with admin permissions can change visibility, and public repositories are visible to everyone on the internet. [268] [269]

Steps

1. Open the repository on GitHub.
2. Click **Settings**.
3. Scroll down to **Danger Zone**.
4. Click **Change visibility**.
5. Select **Public**.
6. Confirm the repository name if prompted.
7. Click **I understand the consequences, change repository visibility** or **Make this repository public**. [270] [268]

Important notes

- If the repository is in an organization, organization rules may restrict who can change visibility. [271]
- Public repositories can be forked and viewed by anyone. [269] [268]
- If you do not see the **Settings** tab, you may need to open the repository menu and choose **Settings** from there. [268]

If the option is missing

If you cannot change it, usually one of these is true:

- You are not an admin on the repository.
- The repo belongs to an organization with visibility restrictions.
- You are on a fork or special repository type with limited settings. [271] [268]

Would you like the exact **click path for GitHub on desktop** or the **command-line steps to verify the remote after making it public**?

⁂

coming back to flutter project that we are doing.

Welcome back. We were building the Flutter POS billing app with:

- SQLite-backed products.
- Product search and barcode lookup.

- Product-to-cart integration.
- Cart quantity management.
- Checkout with cash/card payment options.
- Sales and stock persistence.
- Riverpod state management.
- Windows and web database initialization.
- Git/GitHub setup.

The most recent technical issue was fixing GitHub access and making the repository public. Before continuing development, restore the project and verify its current state.

1. Open the project

```
cd path/to/your/flutter_project
```

Check the repository:

```
git status
git remote -v
git branch
```

Pull the latest version:

```
git pull origin main
```

If your branch is named `master`, use:

```
git pull origin master
```

2. Verify Flutter dependencies

Run:

```
flutter pub get
dart format lib test
flutter analyze
```

If the project builds successfully, start it:

```
flutter run
```

For Windows:

```
flutter run -d windows
```

For Chrome:

```
flutter run -d chrome
```

3. Confirm the important files

The current project should contain these key files:

```
lib/
├── main.dart
├── app/
│   ├── app.dart
│   ├── router.dart
│   ├── theme.dart
│   ├── screens/
│   │   └── placeholder_screen.dart
│   └── widgets/
│       └── side_navigation_rail.dart
├── core/
│   ├── database/
│   │   ├── app_database.dart
│   │   ├── database_initializer.dart
│   │   ├── database_initializer_io.dart
│   │   ├── database_initializer_stub.dart
│   │   └── database_initializer_web.dart
│   ├── barcode/
│   │   └── barcode_scanner_service.dart
│   └── printing/
│       └── receipt_printer_service.dart
└── features/
    ├── products/
    ├── cart/
    ├── checkout/
    └── sales/
```

4. Recommended next step

The next development step should be to complete the **Products screen** as a real management screen:

- Load products from SQLite.
- Display all products.
- Search by name or barcode.
- Add a new product.
- Edit an existing product.
- Delete or deactivate a product.

- Keep the PoS product list synchronized with the database.

The existing PoS screen should continue using the same product repository and cart provider. This keeps the UI, ViewModel, repository, and database layers separated according to Flutter's recommended architecture. [278] [279]

5. First check

Run:

```
flutter analyze
```

Then share the complete analyzer output. We should fix all current compile errors before adding the Products CRUD screen.

⁂

let us resume with

Load products from SQLite.

We will connect the Product screen to SQLite through:

```
ProductSearchField
    ↓
ProductViewModel
    ↓
ProductRepository
    ↓
ProductLocalService
    ↓
AppDatabase
    ↓
SQLite products table
```

This keeps database access out of the widgets and follows Flutter's recommended repository-based architecture. [280]

1. Verify the database table

File: `lib/core/database/app_database.dart`

Your database must create a `products` table with these columns:

```
await db.execute('''
  CREATE TABLE products (
    id TEXT PRIMARY KEY,
    barcode TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL,
```

```
    price_cents INTEGER NOT NULL,
    tax_rate INTEGER NOT NULL,
    stock_quantity INTEGER NOT NULL,
    is_active INTEGER NOT NULL DEFAULT 1
  )
''');
```

The database should also insert demo products during first creation:

```
Future<void> _insertDemoProducts(Database db) async {
  final products = [
    {
      'id': 'p-001',
      'barcode': '890000000001',
      'name': 'Coffee',
      'price_cents': 299,
      'tax_rate': 19,
      'stock_quantity': 50,
      'is_active': 1,
    },
    {
      'id': 'p-002',
      'barcode': '890000000002',
      'name': 'Tea',
      'price_cents': 199,
      'tax_rate': 19,
      'stock_quantity': 40,
      'is_active': 1,
    },
    {
      'id': 'p-003',
      'barcode': '890000000003',
      'name': 'Sandwich',
      'price_cents': 499,
      'tax_rate': 7,
      'stock_quantity': 20,
      'is_active': 1,
    },
    {
      'id': 'p-004',
      'barcode': '890000000004',
      'name': 'Water',
      'price_cents': 149,
      'tax_rate': 19,
      'stock_quantity': 100,
      'is_active': 1,
    },
  ];

  for (final product in products) {
    await db.insert(
      'products',
      product,
      conflictAlgorithm: ConflictAlgorithm.ignore,
    );
```

```
  }
}
```

2. Product model

File: `lib/features/products/data/models/product.dart`

```
class Product {
  const Product({
    required this.id,
    required this.barcode,
    required this.name,
    required this.priceCents,
    required this.taxRate,
    required this.stockQuantity,
  });

  final String id;
  final String barcode;
  final String name;
  final int priceCents;
  final int taxRate;
  final int stockQuantity;
}
```

3. Product mapper

File: `lib/features/products/data/models/product_mapper.dart`

```
import 'product.dart';

class ProductMapper {
  static Product fromMap(Map<String, Object?> map) {
    return Product(
      id: map['id'] as String,
      barcode: map['barcode'] as String,
      name: map['name'] as String,
      priceCents: map['price_cents'] as int,
      taxRate: map['tax_rate'] as int,
      stockQuantity: map['stock_quantity'] as int,
    );
  }

  static Map<String, Object?> toMap(Product product) {
    return {
      'id': product.id,
      'barcode': product.barcode,
      'name': product.name,
      'price_cents': product.priceCents,
      'tax_rate': product.taxRate,
      'stock_quantity': product.stockQuantity,
```

```
      'is_active': 1,
    };
  }
}
```

4. Product local service

This class performs the actual SQLite queries.

File: `lib/features/products/data/services/product_local_service.dart`

```
import 'package:sqflite/sqflite.dart';

import '../../../../core/database/app_database.dart';
import '../models/product.dart';
import '../models/product_mapper.dart';

class ProductLocalService {
  ProductLocalService({
    required this.database,
  });

  final AppDatabase database;

  Future<List<Product>> getAllProducts() async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'is_active = ?',
      whereArgs: [^29_1],
      orderBy: 'name COLLATE NOCASE ASC',
    );

    return rows.map(ProductMapper.fromMap).toList();
  }

  Future<List<Product>> searchProducts(String query) async {
    final searchText = query.trim();

    if (searchText.isEmpty) {
      return getAllProducts();
    }

    final db = await database.database;

    final rows = await db.query(
      'products',
      where: '''
        is_active = ?
        AND (
          name LIKE ?
          OR barcode LIKE ?
        )
```

```
      ''',
      whereArgs: [
        1,
        '%$searchText%',
        '%$searchText%',
      ],
      orderBy: 'name COLLATE NOCASE ASC',
    );

    return rows.map(ProductMapper.fromMap).toList();
  }

  Future<Product?> getProductById(String id) async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'id = ? AND is_active = ?',
      whereArgs: [id, 1],
      limit: 1,
    );

    if (rows.isEmpty) {
      return null;
    }

    return ProductMapper.fromMap(rows.first);
  }

  Future<Product?> getProductByBarcode(String barcode) async {
    final db = await database.database;

    final rows = await db.query(
      'products',
      where: 'barcode = ? AND is_active = ?',
      whereArgs: [barcode.trim(), 1],
      limit: 1,
    );

    if (rows.isEmpty) {
      return null;
    }

    return ProductMapper.fromMap(rows.first);
  }

  Future<void> insertProduct(Product product) async {
    final db = await database.database;

    await db.insert(
      'products',
      ProductMapper.toMap(product),
      conflictAlgorithm: ConflictAlgorithm.abort,
    );
  }
}
```

The `db.query()` calls are the read operations that load products from SQLite. This follows the normal SQLite query pattern: open the database, query rows, and map rows into application models. [281]

5. Product repository

File: `lib/features/products/data/repositories/product_repository.dart`

```
import '../models/product.dart';
import '../services/product_local_service.dart';

class ProductRepository {
  ProductRepository({
    required this.localService,
  });

  final ProductLocalService localService;

  Future<List<Product>> getAllProducts() {
    return localService.getAllProducts();
  }

  Future<List<Product>> searchProducts(String query) {
    return localService.searchProducts(query);
  }

  Future<Product?> getProductById(String id) {
    return localService.getProductById(id);
  }

  Future<Product?> getProductByBarcode(String barcode) {
    return localService.getProductByBarcode(barcode);
  }

  Future<void> addProduct(Product product) {
    return localService.insertProduct(product);
  }
}
```

6. Product ViewModel

File: `lib/features/products/presentation/view_models/product_view_model.dart`

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../../core/database/app_database.dart';
import '../../data/models/product.dart';
import '../../data/repositories/product_repository.dart';
import '../../data/services/product_local_service.dart';

final productRepositoryProvider = Provider<ProductRepository>((ref) {
```

```
  return ProductRepository(
    localService: ProductLocalService(
      database: AppDatabase.instance,
    ),
  );
});

final productViewModelProvider =
    AsyncNotifierProvider<ProductViewModel, ProductState>(
  ProductViewModel.new,
);

class ProductState {
  const ProductState({
    this.products = const [],
    this.searchText = '',
  });

  final List<Product> products;
  final String searchText;

  ProductState copyWith({
    List<Product>? products,
    String? searchText,
  }) {
    return ProductState(
      products: products ?? this.products,
      searchText: searchText ?? this.searchText,
    );
  }
}

class ProductViewModel extends AsyncNotifier<ProductState> {
  @override
  Future<ProductState> build() async {
    final repository = ref.read(productRepositoryProvider);
    final products = await repository.getAllProducts();

    return ProductState(
      products: products,
    );
  }

  Future<void> loadProducts() async {
    final repository = ref.read(productRepositoryProvider);

    state = const AsyncLoading();

    state = await AsyncValue.guard(() async {
      final products = await repository.getAllProducts();

      return ProductState(
        products: products,
      );
    });
  }
```

```
  Future<void> search(String value) async {
    final repository = ref.read(productRepositoryProvider);

    state = const AsyncLoading();

    state = await AsyncValue.guard(() async {
      final products = await repository.searchProducts(value);

      return ProductState(
        products: products,
        searchText: value,
      );
    });
  }

  Future<void> clearSearch() {
    return search('');
  }
}
```

7. Product search widget

File: `lib/features/products/presentation/widgets/product_search_field.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';
import 'product_list_item.dart';

class ProductSearchField extends ConsumerWidget {
  const ProductSearchField({
    super.key,
  });

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productsAsync = ref.watch(productViewModelProvider);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            decoration: InputDecoration(
              labelText: 'Search product or barcode',
              hintText: 'Type Coffee, Tea, or a barcode',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: IconButton(
                tooltip: 'Clear search',
                icon: const Icon(Icons.clear),
                onPressed: () {
```

```
                ref
                    .read(productViewModelProvider.notifier)
                    .clearSearch();
              },
            ),
          ),
          onChanged: (value) {
            ref
                .read(productViewModelProvider.notifier)
                .search(value);
          },
        ),
      ),
      Expanded(
        child: productsAsync.when(
          loading: () => const Center(
            child: CircularProgressIndicator(),
          ),
          error: (error, stackTrace) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    const Icon(
                      Icons.error_outline,
                      size: 48,
                      color: Colors.red,
                    ),
                    const SizedBox(height: 12),
                    Text(
                      'Unable to load products',
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                    const SizedBox(height: 8),
                    Text(
                      '$error',
                      textAlign: TextAlign.center,
                    ),
                    const SizedBox(height: 16),
                    OutlinedButton.icon(
                      onPressed: () {
                        ref
                            .read(productViewModelProvider.notifier)
                            .loadProducts();
                      },
                      icon: const Icon(Icons.refresh),
                      label: const Text('Retry'),
                    ),
                  ],
                ),
              ),
            );
          },
          data: (productState) {
```

```
              if (productState.products.isEmpty) {
                return const Center(
                  child: Text('No products found'),
                );
              }

              return RefreshIndicator(
                onRefresh: () {
                  return ref
                      .read(productViewModelProvider.notifier)
                      .loadProducts();
                },
                child: ListView.builder(
                  physics: const AlwaysScrollableScrollPhysics(),
                  padding: const EdgeInsets.only(
                    left: 8,
                    right: 8,
                    bottom: 16,
                  ),
                  itemCount: productState.products.length,
                  itemBuilder: (context, index) {
                    return ProductListItem(
                      product: productState.products[index],
                    );
                  },
                ),
              );
            },
          ),
        ),
      ],
    );
  }
}
```

8. Product list item

File: `lib/features/products/presentation/widgets/product_list_item.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../cart/presentation/view_models/cart_view_model.dart';
import '../../data/models/product.dart';

class ProductListItem extends ConsumerWidget {
  const ProductListItem({
    super.key,
    required this.product,
  });

  final Product product;

  String formatMoney(int cents) {
```

```
    return '€${(cents / 100).toStringAsFixed(2)}';
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final colorScheme = Theme.of(context).colorScheme;
    final isOutOfStock = product.stockQuantity <= 0;

    return Card(
      margin: const EdgeInsets.symmetric(
        horizontal: 8,
        vertical: 4,
      ),
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: colorScheme.primaryContainer,
          child: Icon(
            Icons.inventory_2,
            color: colorScheme.onPrimaryContainer,
          ),
        ),
        title: Text(
          product.name,
          style: const TextStyle(
            fontWeight: FontWeight.bold,
          ),
        ),
        subtitle: Text(
          'Barcode: ${product.barcode}\n'
          'Stock: ${product.stockQuantity}',
        ),
        isThreeLine: true,
        trailing: SizedBox(
          width: 125,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            crossAxisAlignment: CrossAxisAlignment.end,
            children: [
              Text(
                formatMoney(product.priceCents),
                style: TextStyle(
                  color: colorScheme.primary,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 4),
              SizedBox(
                height: 32,
                child: FilledButton.icon(
                  onPressed: isOutOfStock
                      ? null
                      : () {
                          ref
                              .read(cartProvider.notifier)
                              .addProduct(product);
```

```
                          ScaffoldMessenger.of(context).showSnackBar(
                            SnackBar(
                              content: Text(
                                '${product.name} added to cart',
                              ),
                              duration: const Duration(seconds: 1),
                            ),
                          );
                        },
                    icon: const Icon(
                      Icons.add_shopping_cart,
                      size: 16,
                    ),
                    label: const Text('Add'),
                  ),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

9. Product panel in the PoS screen

File: `lib/features/cart/presentation/screens/pos_screen.dart`

The product section must give `ProductSearchField` a bounded height:

```
class _ProductsPanel extends StatelessWidget {
  const _ProductsPanel();

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Theme.of(context).colorScheme.surface,
      child: const Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Padding(
            padding: EdgeInsets.fromLTRB(16, 16, 16, 0),
            child: Text(
              'Products',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          Expanded(
            child: ProductSearchField(),
          ),
        ],
```

```
        ),
      );
    }
  }
```

The `Expanded` is important because the search widget contains a `ListView`. Without a bounded height, Flutter can produce layout or hit-test errors.

10. Ensure database initialization happens before loading

File: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app.dart';
import 'core/database/database_initializer.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await initializeDatabase();

  runApp(
    ProviderScope(
      child: const PosBillingApp(),
    ),
  );
}
```

If you are running Windows desktop, initialize the FFI factory in `database_initializer_io.dart`. If you are running Chrome, use the web implementation. The database factory must be set before the first database call.

11. Reset the database once if necessary

If the database was created before the `products` table was added, its `onCreate` callback will not run again. Temporarily call:

```
await AppDatabase.instance.deleteDatabaseForDevelopment();
```

inside `main()` before `runApp()`:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await initializeDatabase();

  // Use this once only if the schema is old.
  // await AppDatabase.instance.deleteDatabaseForDevelopment();
```

```
  runApp(
    ProviderScope(
      child: const PosBillingApp(),
    ),
  );
}
```

Uncomment it once, run the app, then comment it again. Otherwise, your local products will be deleted on every startup.

12. Run and test

```
dart format lib
flutter pub get
flutter analyze
flutter run
```

Expected result:

1. The Product ViewModel opens the database through `AppDatabase`.
2. The repository calls `ProductLocalService`.
3. The service runs `SELECT` queries against `products`.
4. The rows are converted to `Product` objects.
5. `ProductSearchField` displays them.
6. Searching by `Coffee` or `8900000000001` filters the SQLite results.
7. Clicking **Add** sends the product to the cart provider.

If the screen displays `No products found`, check whether the database was created before the product table or whether the product rows were never inserted.

⁂

are the below already included in the previous implementation

Display all products.
Search by name or barcode.
Add a new product.
Edit an existing product.
Delete or deactivate a product.
Keep the PoS product list synchronized with the database.

No. The previous implementation includes only part of the requested functionality.

Requirement	Current status	Details
Display all products	✅ Included	`getAllProducts()` loads active products from SQLite and `ProductSearchField` displays them.
Search by name or barcode	✅ Included	`searchProducts()` uses `name LIKE` and `barcode LIKE`.
Add a new product	⚠ Partially included	The repository and local service have `insertProduct()`, but there is no completed Product screen form or Add Product UI.
Edit an existing product	❌ Not included	No update method, form, or edit action exists.
Delete/deactivate a product	❌ Not included	No soft-delete method or delete action exists.
Synchronize PoS product list	⚠ Partially included	Both screens can read from the same repository, but there is no refresh/invalidation mechanism after add, edit, or deactivate.

SQLite supports separate insert, update, and delete operations; only the insert/read parts were implemented so far. Shared repository state is also the right approach when multiple ViewModels depend on the same data.[296] [297]

Already implemented

These parts are present:

```
Future<List<Product>> getAllProducts()
```

```
Future<List<Product>> searchProducts(String query)
```

```
Future<Product?> getProductByBarcode(String barcode)
```

```
Future<void> insertProduct(Product product)
```

The first three support loading and searching. The last one supports database insertion, but there is currently no screen that calls it.

Missing implementation

We still need to add:

```
Future<void> updateProduct(Product product)
```

```
Future<void> deactivateProduct(String productId)
```

```
Future<void> addProduct(Product product)
```

```
Future<void> updateProduct(Product product)
```

```
Future<void> deactivateProduct(String productId)
```

We also need:

- Product form screen or dialog.
- Add button.
- Edit button.
- Deactivate/delete button.
- Product list refresh after mutations.
- PoS refresh after product mutations.
- Database schema migration if existing databases use an older schema.

Current capability

At this stage, the application can:

1. Open SQLite.
2. Read products.
3. Display them in the PoS product panel.
4. Search by name or barcode.
5. Add an existing product to the cart.
6. Save checkout transactions and reduce stock.

It cannot yet manage the product catalog from the Product screen.

Recommended next step

We should implement the missing Product CRUD functionality in this order:

1. Add `updateProduct()` to `ProductLocalService`.
2. Add `deactivateProduct()` to `ProductLocalService`.
3. Expose both methods through `ProductRepository`.
4. Add `addProduct()`, `updateProduct()`, and `deactivateProduct()` to `ProductViewModel`.
5. Create a reusable `ProductFormDialog`.
6. Create the full `ProductsScreen`.
7. Refresh the PoS list after changes.

8. Add database migration/version handling.

The next implementation should therefore be the **Product CRUD screen**, rather than changing the existing loading code.

⁂

Let us complete the functionality

Adding a new product.

We will implement **Add New Product** using a validated `Form`, then insert the product into SQLite through the repository. Flutter's recommended form flow is `Form` → `TextFormField.validator` → validate → submit, and `sqflite` provides the database `insert` operation.[311] [312]

1. Add a product ID helper

No extra package is required. Use a timestamp-based ID for now.

2. Add product support to the ViewModel

File: `lib/features/products/presentation/view_models/product_view_model.dart`

Add this method inside `ProductViewModel`:

```
Future<bool> addProduct({
  required String barcode,
  required String name,
  required int priceCents,
  required int taxRate,
  required int stockQuantity,
}) async {
  final repository = ref.read(productRepositoryProvider);

  final product = Product(
    id: 'p-${DateTime.now().microsecondsSinceEpoch}',
    barcode: barcode.trim(),
    name: name.trim(),
    priceCents: priceCents,
    taxRate: taxRate,
    stockQuantity: stockQuantity,
  );

  state = const AsyncLoading();

  final result = await AsyncValue.guard(() async {
    await repository.addProduct(product);

    final products = await repository.getAllProducts();

    return ProductState(
      products: products,
```

```
    );
  });

  state = result;

  return result.hasValue;
}
```

Your complete ViewModel should contain this method along with the existing `build`, `loadProducts`, `search`, and `clearSearch` methods.

3. Add duplicate barcode protection

Update `ProductLocalService` so duplicate barcodes produce a useful error.

File: `lib/features/products/data/services/product_local_service.dart`

Replace `insertProduct` with:

```
Future<void> insertProduct(Product product) async {
  final db = await database.database;

  final existingProduct = await db.query(
    'products',
    columns: ['id'],
    where: 'barcode = ?',
    whereArgs: [product.barcode],
    limit: 1,
  );

  if (existingProduct.isNotEmpty) {
    throw Exception(
      'A product with barcode ${product.barcode} already exists.',
    );
  }

  await db.insert(
    'products',
    ProductMapper.toMap(product),
    conflictAlgorithm: ConflictAlgorithm.abort,
  );
}
```

The database's `barcode UNIQUE` constraint remains important, but validating before insertion gives the user a readable error.

4. Create the product form dialog

File: `lib/features/products/presentation/widgets/product_form_dialog.dart`

Create this file:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../view_models/product_view_model.dart';

class ProductFormDialog extends ConsumerStatefulWidget {
  const ProductFormDialog({
    super.key,
  });

  @override
  ConsumerState<ProductFormDialog> createState() {
    return _ProductFormDialogState();
  }
}

class _ProductFormDialogState
    extends ConsumerState<ProductFormDialog> {
  final _formKey = GlobalKey<FormState>();

  final _barcodeController = TextEditingController();
  final _nameController = TextEditingController();
  final _priceController = TextEditingController();
  final _taxController = TextEditingController(text: '19');
  final _stockController = TextEditingController();

  bool _isSaving = false;

  @override
  void dispose() {
    _barcodeController.dispose();
    _nameController.dispose();
    _priceController.dispose();
    _taxController.dispose();
    _stockController.dispose();
    super.dispose();
  }

  Future<void> _saveProduct() async {
    if (!_formKey.currentState!.validate()) {
      return;
    }

    final priceCents = _parsePriceToCents(
      _priceController.text,
    );

    final taxRate = int.parse(_taxController.text.trim());
    final stockQuantity = int.parse(
      _stockController.text.trim(),
    );
```

```
    setState(() {
      _isSaving = true;
    });

    final success = await ref
        .read(productViewModelProvider.notifier)
        .addProduct(
          barcode: _barcodeController.text,
          name: _nameController.text,
          priceCents: priceCents,
          taxRate: taxRate,
          stockQuantity: stockQuantity,
        );

    if (!mounted) {
      return;
    }

    setState(() {
      _isSaving = false;
    });

    if (success) {
      Navigator.of(context).pop(true);
      return;
    }

    final error = ref.read(productViewModelProvider).error;

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          error?.toString() ?? 'Unable to add product.',
        ),
      ),
    );
  }

  int _parsePriceToCents(String value) {
    final normalized = value.trim().replaceAll(',', '.');
    final price = double.parse(normalized);
    return (price * 100).round();
  }

  String? _requiredValidator(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'This field is required.';
    }

    return null;
  }

  String? _positiveIntegerValidator(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'This field is required.';
    }
```

```
    final parsed = int.tryParse(value.trim());

    if (parsed == null || parsed < 0) {
      return 'Enter a valid non-negative number.';
    }

    return null;
  }

  String? _priceValidator(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Price is required.';
    }

    final normalized = value.trim().replaceAll(',', '.');
    final parsed = double.tryParse(normalized);

    if (parsed == null || parsed < 0) {
      return 'Enter a valid price.';
    }

    return null;
  }

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: const Text('Add product'),
      content: SizedBox(
        width: 450,
        child: Form(
          key: _formKey,
          child: SingleChildScrollView(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                TextFormField(
                  controller: _nameController,
                  autofocus: true,
                  decoration: const InputDecoration(
                    labelText: 'Product name',
                    prefixIcon: Icon(Icons.label_outline),
                  ),
                  validator: _requiredValidator,
                  textInputAction: TextInputAction.next,
                ),
                const SizedBox(height: 12),
                TextFormField(
                  controller: _barcodeController,
                  decoration: const InputDecoration(
                    labelText: 'Barcode',
                    prefixIcon: Icon(Icons.qr_code),
                  ),
                  validator: _requiredValidator,
                  textInputAction: TextInputAction.next,
```

```
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _priceController,
                decoration: const InputDecoration(
                  labelText: 'Price',
                  hintText: 'Example: 2.99',
                  prefixIcon: Icon(Icons.euro),
                ),
                keyboardType: const TextInputType.numberWithOptions(
                  decimal: true,
                ),
                validator: _priceValidator,
                textInputAction: TextInputAction.next,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _taxController,
                decoration: const InputDecoration(
                  labelText: 'Tax rate (%)',
                  prefixIcon: Icon(Icons.percent),
                ),
                keyboardType: TextInputType.number,
                validator: _positiveIntegerValidator,
                textInputAction: TextInputAction.next,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _stockController,
                decoration: const InputDecoration(
                  labelText: 'Stock quantity',
                  prefixIcon: Icon(Icons.inventory_2_outlined),
                ),
                keyboardType: TextInputType.number,
                validator: _positiveIntegerValidator,
                textInputAction: TextInputAction.done,
                onFieldSubmitted: (_) {
                  if (!_isSaving) {
                    _saveProduct();
                  }
                },
              ),
            ],
          ),
        ),
      ),
    ),
    actions: [
      TextButton(
        onPressed: _isSaving
            ? null
            : () {
                Navigator.of(context).pop(false);
              },
        child: const Text('Cancel'),
      ),
```

```
        FilledButton.icon(
          onPressed: _isSaving ? null : _saveProduct,
          icon: _isSaving
              ? const SizedBox(
                  width: 18,
                  height: 18,
                  child: CircularProgressIndicator(
                    strokeWidth: 2,
                  ),
                )
              : const Icon(Icons.save),
          label: const Text('Save'),
        ),
      ],
    );
  }
}
```

The form validates all fields before calling the ViewModel, consistent with Flutter's standard form-validation flow. [311]

5. Create the Product screen

File: `lib/features/products/presentation/screens/products_screen.dart`

Create this file:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../../../../app/widgets/side_navigation_rail.dart';
import '../view_models/product_view_model.dart';
import '../widgets/product_form_dialog.dart';
import '../widgets/product_list_item.dart';

class ProductsScreen extends ConsumerWidget {
  const ProductsScreen({
    super.key,
  });

  Future<void> _showAddProductDialog(
    BuildContext context,
    WidgetRef ref,
  ) async {
    final added = await showDialog<bool>(
      context: context,
      builder: (context) {
        return const ProductFormDialog();
      },
    );

    if (added == true && context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
```

```
        const SnackBar(
          content: Text('Product added successfully'),
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productsAsync = ref.watch(productViewModelProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Products'),
        actions: [
          IconButton(
            tooltip: 'Refresh products',
            icon: const Icon(Icons.refresh),
            onPressed: () {
              ref
                  .read(productViewModelProvider.notifier)
                  .loadProducts();
            },
          ),
        ],
      ),
      body: Row(
        children: [
          SideNavigationRail(
            selectedIndex: 2,
            onDestinationSelected: (index) {
              _navigateToDestination(context, index);
            },
          ),
          const VerticalDivider(
            width: 1,
            thickness: 1,
          ),
          Expanded(
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Row(
                    children: [
                      Text(
                        'Product catalog',
                        style: Theme.of(context)
                            .textTheme
                            .headlineSmall
                            ?.copyWith(
                              fontWeight: FontWeight.bold,
                            ),
                      ),
                      const Spacer(),
```

```
      FilledButton.icon(
        onPressed: () {
          _showAddProductDialog(context, ref);
        },
        icon: const Icon(Icons.add),
        label: const Text('Add product'),
      ),
    ],
  ),
  const SizedBox(height: 16),
  const _ProductSearchInput(),
  const SizedBox(height: 16),
  Expanded(
    child: productsAsync.when(
      loading: () => const Center(
        child: CircularProgressIndicator(),
      ),
      error: (error, stackTrace) {
        return Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text('Unable to load products: $error'),
              const SizedBox(height: 12),
              OutlinedButton(
                onPressed: () {
                  ref
                      .read(
                        productViewModelProvider.notifier,
                      )
                      .loadProducts();
                },
                child: const Text('Retry'),
              ),
            ],
          ),
        );
      },
      data: (productState) {
        if (productState.products.isEmpty) {
          return const Center(
            child: Text('No products found'),
          );
        }

        return ListView.builder(
          itemCount: productState.products.length,
          itemBuilder: (context, index) {
            return ProductListItem(
              product: productState.products[index],
            );
          },
        );
      },
    ),
  ),
```

```
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }

  void _navigateToDestination(
    BuildContext context,
    int index,
  ) {
    switch (index) {
      case 0:
        context.go('/dashboard');
        break;
      case 1:
        context.go('/pos');
        break;
      case 2:
        context.go('/products');
        break;
      case 3:
        context.go('/inventory');
        break;
      case 4:
        context.go('/sales');
        break;
      case 5:
        context.go('/customers');
        break;
      case 6:
        context.go('/reports');
        break;
      case 7:
        context.go('/settings');
        break;
    }
  }
}

class _ProductSearchInput extends ConsumerWidget {
  const _ProductSearchInput();

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productState = ref.watch(productViewModelProvider).valueOrNull;

    return TextField(
      decoration: InputDecoration(
        labelText: 'Search products',
        hintText: 'Search by name or barcode',
        prefixIcon: const Icon(Icons.search),
        suffixIcon: IconButton(
          tooltip: 'Clear search',
```

```
            icon: const Icon(Icons.clear),
            onPressed: () {
              ref
                  .read(productViewModelProvider.notifier)
                  .clearSearch();
            },
          ),
        ),
        controller: TextEditingController(
          text: productState?.searchText ?? '',
        ),
        onChanged: (value) {
          ref
              .read(productViewModelProvider.notifier)
              .search(value);
        },
      );
    }
}
```

6. Add the route

File: `lib/app/router.dart`

Import the screen:

```
import '../features/products/presentation/screens/products_screen.dart';
```

Replace the current `/products` route:

```
GoRoute(
  path: '/products',
  builder: (context, state) => const ProductsScreen(),
),
```

7. Keep the PoS screen synchronized

The PoS product list and Product screen use the same `productViewModelProvider`. After saving a product, the ViewModel reloads the products from SQLite:

```
await repository.addProduct(product);

final products = await repository.getAllProducts();

return ProductState(
  products: products,
);
```

Therefore, the Product screen updates immediately. When navigating to the PoS screen afterward, it reads the same provider and displays the new product.

For an already-mounted PoS screen, add a refresh when the screen resumes or expose a shared refresh method. The simplest immediate approach is to make the PoS screen refresh products when it is built:

```
class PosScreen extends ConsumerStatefulWidget {
  const PosScreen({super.key});

  @override
  ConsumerState<PosScreen> createState() {
    return _PosScreenState();
  }
}

class _PosScreenState extends ConsumerState<PosScreen> {
  @override
  void initState() {
    super.initState();

    Future.microtask(() {
      ref.read(productViewModelProvider.notifier).loadProducts();
    });
  }

  @override
  Widget build(BuildContext context) {
    // Move the existing PosScreen build code here.
    return const SizedBox();
  }
}
```

However, if the PoS screen is rebuilt after navigation, the existing `AsyncNotifier` state may already contain the new product. We can improve this synchronization when implementing edit and deactivate.

8. Verify the product database

If the database already existed before the `products` table was added, temporarily reset it:

```
await AppDatabase.instance.deleteDatabaseForDevelopment();
```

Run the app once, then remove or comment out that line. Otherwise, every application start will delete all products.

9. Run and test

```
dart format lib
flutter pub get
flutter analyze
flutter run
```

Test:

1. Open `/products`.
2. Click **Add product**.
3. Enter:
    - Name: `Juice`
    - Barcode: `890000000005`
    - Price: `1.99`
    - Tax: `19`
    - Stock: `25`
4. Click **Save**.
5. Confirm `Juice` appears in the Product catalog.
6. Search for `Juice`.
7. Search for `890000000005`.
8. Open `/pos`.
9. Confirm `Juice` is available.
10. Click **Add** and confirm it appears in the cart.

The next remaining catalog functionality is **editing existing products**, followed by **deactivating products**.

⁂

1. https://www.replay.build/blog/beyond-the-screenshot-why-replay-s-video-workflow-capture-is-the-new-standard-for-ui-extraction
2. https://www.replay.build/blog/why-replay-screenscraping-recording-beats-ocr-for-modernizing-legacy-apps
3. https://medium.com/data-science-collective/turning-screensots-int-data-html-126bdcaa4821
4. https://codia.ai/screenshot-to-figma
5. https://www.replay.build/blog/what-is-the-difference-between-ocr-based-ui-generators-and-replay-s-visual-extraction
6. https://www.replay.build/blog/how-to-reconstruct-interative-ui-elements-from-mp4-or-mov-files-the-definitive-guide
7. https://screph.com/en/use-cases.html
8. https://ui.vision/rpa/x/desktop-automation/screen-scraping
9. https://codia.ai/visual-struct
10. https://www.replay.build/blog/how-to-extract-svg-icons-and-assets-from-video-files-using-ai

11. https://www.durgamsoft.com/bill_master.html
12. https://www.durgamsoft.in/billing-software.html
13. https://play.google.com/store/apps/details?id=com.durgamsoft.com.dsmartpos&hl=pt
14. https://sourceforge.net/software/product/BillMaster/
15. https://play.google.com/store/apps/details?id=com.durgamsoft.com.dsmartpos&hl=fa
16. https://www.durgamsoft.com/
17. https://www.durgamsoft.in/barcode-scanner.html
18. https://www.durgamsoft.in/thermal-receipt-printer.html
19. https://billmaster.co.in/
20. https://www.justdial.com/Rangareddy/Durgam-Soft-Pvt-Ltd-Silver-Oak-Bunglows-Cherlapalli/040PXX40-XX40-250109134840-M7D6_BZDET
21. https://docs.flutter.dev/app-architecture/guide
22. https://docs.flutter.dev/app-architecture/recommendations
23. https://docs.flutter.dev/app-architecture/concepts
24. https://docs.flutter.dev/app-architecture/design-patterns/offline-first
25. https://docs.flutter.dev/data-and-backend/state-mgmt/simple
26. https://docs.flutter.dev/app-architecture
27. https://docs.flutter.dev/data-and-backend/state-mgmt
28. https://docs.flutter.dev/data-and-backend/state-mgmt/options
29. https://docs.flutter.dev/data-and-backend/state-mgmt/intro
30. https://docs.flutter.dev/resources/architectural-overview
31. https://docs.flutter.dev/app-architecture/guide
32. https://docs.flutter.dev/reference/create-new-app
33. https://docs.flutter.dev/reference/flutter-cli
34. https://docs.flutter.dev/cookbook/persistence
35. https://docs.flutter.dev/cookbook/persistence/sqlite
36. https://docs.flutter.dev/app-architecture/design-patterns/offline-first
37. https://docs.flutter.dev/platform-integration/web/building
38. https://stackoverflow.com/questions/49047411/flutter-how-to-create-a-new-project
39. https://docs.flutter.dev/learn/pathway/tutorial/create-an-app
40. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
41. https://docs.flutter.dev/reference/create-new-app.md
42. https://docs.flutter.dev/app-architecture/guide
43. https://docs.flutter.dev/app-architecture/case-study
44. https://docs.flutter.dev/cookbook/persistence/sqlite
45. https://dev.to/leehack/journey-to-the-clean-architecture-for-my-flutter-app-138n
46. https://medium.com/@31carlton7/how-to-structure-your-flutter-projects-3939784d7d04

47. https://ottomancoder.medium.com/the-flutter-folder-structure-that-scales-and-the-one-that-quietly-doesnt-eafaadcf87c5
48. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
49. https://docs.flutter.dev/app-architecture/recommendations
50. https://mailharshkhatri.medium.com/understanding-flutters-project-structure-e45cc7140ddb
51. https://flutteroriginals.com/flutter-clean-architecture-folder-structure/
52. https://harshvardhan082.hashnode.dev/project-structure-in-flutter
53. https://docs.flutter.dev/learn/pathway/tutorial/widget-fundamentals
54. https://medium.com/@oindrila.das2708/how-to-make-custom-reusable-widgets-in-flutter-b21c5ef9aee8
55. https://stackoverflow.com/questions/60384439/the-method-isnt-defined-for-the-class-flutter
56. https://flutter.design/article/How_to_Create_Custom_Widgets_in_Flutter_for_Better_UI_Design.html
57. https://fluttermasterylibrary.com/3/3/1/4/
58. https://flutter.design/article/Creating_Custom_Widgets_in_Flutter.html
59. https://flutter.dev/blog/how-to-create-stateless-widgets
60. https://medium.com/@ugamakelechi501/10-common-flutter-errors-and-how-to-fix-them-64ed5e1eeb21
61. https://stackoverflow.com/questions/69409725/i-have-an-error-for-import-package-in-flutter
62. https://docs.flutter.dev/ui/interactivity
63. https://api.flutter.dev/flutter/material/ColorScheme-class.html
64. https://docs.flutter.dev/cookbook/design/themes
65. https://docs.flutter.dev/release/breaking-changes/material-3-migration
66. https://api.flutter.dev/flutter/material/MaterialColor-class.html
67. https://api.flutter.dev/flutter/material/
68. https://api.flutter.dev/flutter/material/ThemeData/useMaterial3.html
69. https://docs.flutter.dev/release/breaking-changes/material-3-default
70. https://m3.material.io/develop/flutter
71. https://api.flutter.dev/flutter/material/ThemeData/ThemeData.html
72. https://www.christianfindlay.com/blog/flutter-mastering-material-design3
73. https://api.flutter.dev/flutter/material/NavigationDrawer-class.html
74. https://docs.flutter.dev/ui/widgets/material
75. https://api.flutter.dev/flutter/material/Drawer-class.html
76. https://m3.material.io/components/navigation-drawer/overview
77. https://m3.material.io/components/navigation-rail/overview
78. https://api.flutter.dev/flutter/material/NavigationRail/NavigationRail.html
79. https://binoo11.medium.com/material-3-bottomnavigation-and-navigationdrawer-flutter-428e00d80435
80. https://github.com/flutter/flutter/issues/123113
81. https://flutter.dev/blog/material-3-for-flutter
82. https://pub.dev/packages/navigation_rail_flutter/versions/0.0.11

83. https://codelabs.developers.google.com/codelabs/flutter-animated-responsive-layout
84. https://m3.material.io/components/navigation-rail/overview
85. https://pub.dev/packages/navigation_rail_flutter/versions/0.0.11
86. https://api.flutter.dev/flutter/material/NavigationRail/NavigationRail.html
87. https://api.flutter.dev/flutter/material/Drawer-class.html
88. https://api.flutter.dev/flutter/material/Scaffold/drawer.html
89. https://docs.flutter.dev/
90. https://pub.dev/packages/adaptive_scaffold_router
91. https://api.flutter.dev/flutter/material/NavigationDrawer-class.html
92. https://pub.dev/packages/responsive_scaffold_navigation
93. https://pub.dev/packages/nova_drawer
94. https://stackoverflow.com/questions/51659805/persisting-appbar-drawer-across-all-pages-flutter/51661319
95. https://docs.flutter.dev/app-architecture/guide
96. https://docs.flutter.dev/app-architecture/case-study/data-layer
97. https://docs.flutter.dev/app-architecture/case-study/dependency-injection
98. https://docs-v2.riverpod.dev/docs/providers/change_notifier_provider
99. https://pub.dev/documentation/flutter_riverpod/latest/flutter_riverpod/NotifierProvider-class.html
100. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
101. https://docs.flutter.dev/app-architecture/recommendations
102. https://pub.dev/documentation/riverpod/latest/riverpod/NotifierProvider-class.html
103. https://docs.flutter.dev/app-architecture/case-study
104. https://docs-v2.riverpod.dev/docs/providers/notifier_provider
105. https://api.flutter.dev/flutter/material/TextField/onChanged.html
106. https://docs.flutter.dev/cookbook/forms/text-field-changes
107. https://pub.dev/documentation/flutter_riverpod/latest/flutter_riverpod/NotifierProvider-class.html
108. https://docs-v2.riverpod.dev/docs/providers/notifier_provider
109. https://docs-v2.riverpod.dev/docs/providers/state_provider
110. https://pub.dev/documentation/riverpod/latest/legacy/StateNotifierProvider-class.html
111. https://docs.flutterflow.io/resources/forms/textfield/
112. https://api.flutter.dev/flutter/cupertino/CupertinoTextField/onChanged.html
113. https://stackoverflow.com/questions/52234378/flutter-textfield-in-listview-builder
114. https://docs.flutter.dev/cookbook/forms
115. https://docs-v2.riverpod.dev/docs/providers/state_notifier_provider
116. https://pub.dev/documentation/riverpod/latest/riverpod/StateNotifierProvider-class.html
117. https://dart.dev/tools/diagnostics
118. https://dart.dev/tools/diagnostics/undefined_class
119. https://dart.dev/tools/diagnostics/undefined_identifier/index.html.md

120. https://github.com/flutter/flutter-intellij/issues/1313
121. https://stackoverflow.com/questions/79673206/flutter-undefined-name-error-despite-correct-import-path-and-clean-build
122. https://github.com/dart-lang/sdk/issues/44321
123. https://stackoverflow.com/questions/71494812/flutter-error-isnt-a-type-with-imported-class
124. https://dart.dev/tools/diagnostics/redirect_to_non_class
125. https://medium.com/@ugamakelechi501/10-common-flutter-errors-and-how-to-fix-them-64ed5e1eeb21
126. https://github.com/dart-lang/sdk/issues/44433
127. https://docs.flutter.dev/testing/common-errors
128. https://docs.flutter.dev/ui/layout/constraints
129. https://api.flutter.dev/flutter/rendering/RenderBox/hitTest.html
130. https://pub.dev/documentation/flutter_constraintlayout/latest/
131. https://stackoverflow.com/questions/66547537/flutter-cannot-hit-test-a-render-box-that-has-never-been-laid-out
132. https://stackoverflow.com/questions/54420788/listbuilder-in-separate-widget-not-working/54436003
133. https://stackoverflow.com/questions/52801201/flutter-renderbox-was-not-laid-out
134. https://api.flutter.dev/flutter/rendering/RenderProxyBoxMixin/hitTest.html
135. https://groups.google.com/g/flutter-dev/c/x62n_2t0dOg
136. https://stackoverflow.com/questions/70077249/how-to-lay-out-rows-and-columns-in-flutter-widget
137. https://pub.dev/documentation/riverpod_annotation/latest/riverpod_annotation/Notifier-class.html
138. https://pub.dev/documentation/riverpod/latest/riverpod/Notifier-class.html
139. https://dev.to/the_flutter_sensei/flutter-state-management-101-building-an-interactive-counter-55oj
140. https://docs.flutter.dev/data-and-backend/state-mgmt/intro
141. https://api.flutter.dev/flutter/material/FloatingActionButton/onPressed.html
142. https://docs.flutter.dev/data-and-backend/state-mgmt
143. https://github.com/flutter/website/blob/main/src/content/get-started/fundamentals/state-management.md
144. https://api.flutter.dev/flutter/material/SimpleDialogOption/onPressed.html
145. https://pub.dev/documentation/riverpod/latest/riverpod/StateNotifier-class.html
146. https://docs.flutter.dev/data-and-backend/state-mgmt/options
147. https://docs.flutter.dev/cookbook/persistence/sqlite.md
148. https://docs.flutter.dev/app-architecture/design-patterns/offline-first
149. https://gist.github.com/nhancv/d263af195116149d1f5ab93338011ee8
150. https://docs.flutter.dev/cookbook/persistence
151. https://docs.flutter.dev/app-architecture/design-patterns/sql
152. https://flutteruniversity.gitbook.io/docs/learn-flutter/intermediate/persistence
153. https://www.kodeco.com/9376629-sqlite-in-flutter/page/2
154. https://saai.pro/blog/flutter-offline-first-architecture
155. https://777genius.medium.com/offline-first-flutter-a-practical-guide-to-data-synchronization-5c37ee657755

156. https://github.com/thisissandipp/flutter-sqflite-example/blob/main/README.md
157. https://pub.dev/documentation/sqflite/latest/sqflite/databaseFactory.html
158. https://github.com/tekartik/sqflite/issues/1003
159. https://stackoverflow.com/questions/76158800/databasefactory-not-initialized-when-using-sqflite-in-flutter
160. https://blog.csdn.net/ZHANXIANG100/article/details/140386372
161. https://stackoverflow.com/questions/78329221/sqflite-package-problem-while-making-a-simple-app-for-implementing-crud-operatio
162. https://www.volcengine.com/article/1494585
163. https://stackoverflow.com/questions/77494137/databasefactory-not-initialized-sqflite-error-on-flutter-web
164. https://pub.dev/documentation/sqlite_flutter_pcl/latest/sqlite_flutter_pcl/databaseFactory.html
165. https://community.flutterflow.io/ask-the-community/post/sqlite-integration-having-troubles-on-desktop-RZOa8IEYUsleVxr
166. https://github.com/tekartik/sqflite/issues/1087
167. https://dart.dev/tools/diagnostics/undefined_identifier
168. https://dart.dev/tools/diagnostics/unused_import
169. https://deepsource.com/directory/dart-analyze/issues/DRT-W1655
170. https://medium.com/@arte.bi/dart-import-16323ead7d39
171. https://dart.dev/tools/diagnostics/undefined_getter
172. https://dart.dev/tools/diagnostics/unnecessary_import
173. https://stackoverflow.com/questions/67941696/how-to-access-another-class-from-a-different-file-in-the-same-flutter-project
174. https://dart.cn/tools/diagnostic-messages
175. https://deepsource.com/directory/dart-analyze/issues/DRT-W1671
176. https://api.flutter.dev/flutter/dart-io/File-class.html
177. https://docs.flutter.dev/app-architecture/guide
178. https://docs.flutter.dev/app-architecture/case-study/dependency-injection
179. https://mark-story.com/posts/view/flutter-mvvm-application-architecture
180. https://stackoverflow.com/questions/70344611/how-to-make-a-textfield-search-in-listview-builder-update-with-setstate-inside
181. https://www.reddit.com/r/flutterhelp/comments/z570nx/filter_listview_from_state/
182. https://gist.github.com/mercykip/72e398505fecd3221eaba4ba7b01b170
183. https://docs.flutter.dev/app-architecture/case-study
184. https://github.com/flutter/skills/blob/main/skills/flutter-apply-architecture-best-practices/SKILL.md
185. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
186. https://github.com/Robbendebiene/flutter-mvvm-architecture/blob/main/README.md
187. https://www.atlassian.com/git/tutorials/syncing
188. https://github.com/git-guides/git-init
189. https://docs.github.com/en/get-started/using-git/pushing-commits-to-a-remote-repository

190. https://github.com/github/gitignore/blob/main/Flutter.gitignore
191. https://github.com/firebase/quickstart-flutter/blob/main/.gitignore
192. https://support.atlassian.com/bitbucket-cloud/docs/git-and-mercurial-commands/
193. https://github.com/tendermint/flutter/blob/main/.gitignore
194. https://stackoverflow.com/questions/2337281/how-do-i-do-an-initial-push-to-a-remote-repository-with-git
195. https://gitignore.org/Flutter
196. https://github.com/flutter/flutter/blob/master/.gitignore
197. https://docs.flutter.dev/app-architecture/guide
198. https://docs.flutter.dev/app-architecture/case-study
199. https://m3.material.io/components/navigation-rail/overview
200. https://m3.material.io/components/navigation-drawer/overview
201. https://github.com/dinethsiriwardana/Flutter-Folder-And-File-Structure
202. https://github.com/Robbendebiene/flutter-mvvm-architecture/blob/main/README.md
203. https://github.com/loisenjoki/FlutterStructure
204. https://github.com/NemesisX1/flutter-starter-architecture
205. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
206. https://dev.to/leehack/journey-to-the-clean-architecture-for-my-flutter-app-138n
207. https://docs.flutter.dev/app-architecture/guide
208. https://docs.flutter.dev/app-architecture/recommendations
209. https://docs.flutter.dev/app-architecture/case-study/dependency-injection
210. https://dev.to/leehack/journey-to-the-clean-architecture-for-my-flutter-app-138n
211. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
212. https://github.com/flutter/skills/blob/main/skills/flutter-apply-architecture-best-practices/SKILL.md
213. https://www.blup.in/blog/learn-how-to-build-scalable-testable-flutter-apps-using-mvvm-architecture
214. https://docs.flutter.dev/app-architecture/design-patterns
215. https://docs.flutter.dev/app-architecture/case-study
216. https://medium.com/@mohamedyousufdev/understanding-mvvm-architecture-in-flutter-a-practical-guide-3719cfc3e2c3
217. https://dart.dev/tools/diagnostics/uri_does_not_exist
218. https://dart.dev/tools/diagnostics/prefer_relative_imports
219. https://github.com/dart-lang/sdk/issues/42999
220. https://dart.dev/tools/diagnostics/uri_has_not_been_generated
221. https://dart.dev/tools/diagnostics/invalid_uri
222. https://flutterinterview.dev/questions/135/
223. https://api.flutter.dev/flutter/package-path_path/relative.html
224. https://developermemos.com/posts/absolute-relative-imports-flutter/
225. https://cgrant.io/notebook/flutter---packages-and-imports/

226. https://groups.google.com/a/dartlang.org/g/analyzer-discuss/c/Z8hIZDYvUxk
227. https://dart.dev/tools/diagnostics/undefined_class
228. https://developermemos.com/posts/absolute-relative-imports-flutter/
229. https://dart.dev/tools/diagnostics/uri_does_not_exist
230. https://github.com/flutter/flutter/pull/51/files
231. https://dart.dev/tools/diagnostics/duplicate_import
232. https://stackoverflow.com/questions/55679918/how-to-fix-undefined-file-class-and-its-methods-in-dart
233. https://zenn.dev/k9i/articles/34dab7d09b29a1?locale=en
234. https://api.flutter.dev/flutter/package-path_path/relative.html
235. https://flutterinterview.dev/questions/135/
236. https://www.oreilly.com/library/view/flutter-for-beginners/9781788996082/3912703d-d4d0-4fbd-8a59-fe2c09e2afee.xhtml
237. https://stackoverflow.com/questions/79406549/flutter-import-export
238. https://dart.dev/tools/diagnostics
239. https://dart.dev/tools/diagnostics/const_with_non_const
240. https://dart.dev/tools/diagnostics/const_with_non_constant_argument
241. https://dart.dev/tools/diagnostics/const_initialized_with_non_constant_value
242. https://www.reddit.com/r/flutterhelp/comments/oxyr6c/how_to_resolve_invalid_constant_value_error/
243. https://dart.dev/tools/diagnostics/const_constructor_with_field_initialized_by_non_const
244. https://github.com/dart-lang/sdk/issues/46355
245. https://stackoverflow.com/questions/70462026/invalid-constant-value-in-flutter-dart-const-constructor
246. https://dart.dev/tools/diagnostics/const_constructor_param_type_mismatch
247. https://deepsource.com/directory/dart-analyze/issues/DRT-W1265
248. https://stackoverflow.com/questions/69527724/the-name-myapp-isnt-a-class-try-correcting-the-name-to-match-an-existing-cla
249. https://github.com/flutter/flutter/issues/64399
250. https://docs.flutter.dev/testing/overview
251. https://dart.dev/tools/diagnostics/undefined_class
252. https://codelabs.developers.google.com/codelabs/flutter-app-testing/
253. https://docs.flutter.dev/cookbook/testing/unit/introduction
254. https://github.com/iampawan/FlutterExampleApps/issues/46
255. https://github.com/dart-lang/sdk/issues/31224
256. https://github.com/flutter/flutter-intellij/issues/1429
257. https://pub.dev/packages/rename_app
258. https://docs.github.com/en/repositories/creating-and-managing-repositories/troubleshooting-cloning-errors
259. https://www.geeksforgeeks.org/git/how-to-fix-git-remote-repository-not-found-error/
260. https://github.com/orgs/community/discussions/45993

261. https://stackoverflow.com/questions/10116373/git-push-error-repository-not-found
262. https://www.anygen.io/showcase/remote-write-access-to-repository-not-granted/index.html
263. https://oneuptime.com/blog/post/2026-01-24-git-unable-to-access-remote/view
264. https://github.com/orgs/community/discussions/45698
265. https://stackoverflow.com/questions/10184433/repository-not-found-when-pushing-to-github-remote
266. https://stackoverflow.com/questions/37813568/git-remote-repository-not-found
267. https://learn.microsoft.com/en-us/answers/questions/1467218/git-remote-repository-not-found
268. https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility
269. https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories
270. https://docs.github.com/en/enterprise-server@3.17/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility
271. https://docs.github.com/en/organizations/managing-organization-settings/restricting-repository-visibility-changes-in-your-organization
272. https://docs.github.com/en/enterprise-server@3.7/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility
273. https://docs.github.com/en/issues/planning-and-tracking-with-projects/managing-your-project/managing-visibility-of-your-projects
274. https://docs.github.com/api/article?pathname=/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility
275. https://docs.github.com/en/enterprise-cloud@latest/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility
276. https://docs.github.com/en/enterprise-server@3.15/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility
277. https://stackoverflow.com/questions/57836411/how-can-i-switch-a-public-repo-to-private-and-vice-versa-on-github
278. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
279. https://dart.dev/tools/diagnostics/undefined_identifier/index.html.md
280. https://docs.flutter.dev/app-architecture
281. https://blog.flutteragency.com/database-query-with-sqflite/
282. https://www.youtube.com/watch?v=ckXSR79AACg
283. https://geekyants.com/blog/offline-first-flutter-implementation-blueprint-for-real-world-apps
284. https://www.youtube.com/watch?v=q8UXj-44dk8
285. https://docs.flutter.dev/app-architecture/design-patterns/sql
286. https://docs.flutter.dev/cookbook/persistence/sqlite.md
287. https://docs.flutter.dev/app-architecture/guide
288. https://docs.flutter.dev/app-architecture/design-patterns
289. https://api.flutter.dev/flutter/material/NavigationDrawer-class.html
290. https://fluttergems.dev/sql-database/
291. https://github.com/Raz4492/Flutter-Clean-Code-Architecture-with-Sqflite-Local-DB

292. https://medium.com/@moyeen_haider/building-a-professional-flutter-app-a-step-by-step-guide-to-local-storage-with-sqlite-bfdb00825a71
293. https://www.youtube.com/watch?v=bihC6ou8FqQ
294. https://www.youtube.com/watch?v=YXPksR7xjpw
295. https://dev.to/anurag_dev/implementing-offline-first-architecture-in-flutter-part-1-local-storage-with-conflict-resolution-4mdl
296. https://www.kodeco.com/9376629-sqlite-in-flutter
297. https://github.com/Robbendebiene/flutter-mvvm-architecture/
298. https://docs.flutter.dev/app-architecture/design-patterns/offline-first
299. https://github.com/flutter/skills/blob/main/skills/flutter-apply-architecture-best-practices/SKILL.md
300. https://mark-story.com/posts/view/flutter-mvvm-application-architecture
301. https://github.com/Robbendebiene/flutter-mvvm-architecture/blob/main/README.md
302. https://dev.to/leehack/journey-to-the-clean-architecture-for-my-flutter-app-138n
303. https://dev.to/techwithsam/clean-architecture-in-flutter-2026-practical-implementation-guide-1dfb
304. https://stackoverflow.com/questions/75994975/simplest-implementation-of-mvvm-repository-in-flutter
305. https://asoasis.tech/articles/2026-06-04-0838-flutter-offline-first-architecture-guide/
306. https://www.reddit.com/r/FlutterDev/comments/1jgu3t7/in_clean_architecture_ddd_do_i_pass_repositories/
307. https://medium.com/@jiteshyadav5019/crud-operations-with-sqlite-in-flutter-bloc-d21f6b782079
308. https://docs.flutter.dev/app-architecture/guide
309. https://docs.flutter.dev/app-architecture/recommendations
310. https://docs.flutter.dev/app-architecture/case-study
311. https://docs.flutter.dev/cookbook/forms/validation/index.html.md
312. https://pub.dev/packages/sqflite
313. https://dev.to/jige2025/flutter-lesson-10-forms-and-inputs-2l8a
314. https://medium.com/@dpatel312002/guide-for-sqflite-in-flutter-59e429db1088
315. https://medium.com/@samra.sajjad0001/comprehensive-guide-to-sqlite-in-flutter-with-sqflite-b2b301c1f244
316. https://dev.to/adityasubrahmanyabhat/flutter-textformfield-validation-made-easy-3e61
317. https://docs.flutter.dev/cookbook/forms
318. https://docs.flutter.dev/cookbook/persistence/sqlite.md
319. https://stackoverflow.com/questions/54097326/how-to-do-a-database-insert-with-sqflite-in-flutter
320. https://pub.dev/packages/sqflite/versions/1.3.2+4
321. https://pub.dev/packages/sqflite/versions/0.13.0+1
322. https://pub.dev/documentation/sqflite/latest/
323. https://pub.dev/documentation/text_form_field_flutter/latest/
324. https://deku.posstree.com/en/flutter/widget/sqflite/
325. https://gist.github.com/bizz84/cefdf4d717150397401cfe345ab6f383